

EMBEDDED SYSTEMS / BEGINNER SERIES

PIC ASSEMBLY FROM SCRATCH

A Beginner's Guide with MPLAB XC8

PIC16F17146 • Simulator-first workflow
From first instruction to programmed silicon

PIC Assembly from Scratch: A Beginner's Guide with MPLAB XC8

First book edition, July 2026.

This educational text focuses on the PIC16F17146 enhanced mid-range device. Verify electrical limits, register definitions, configuration settings, and programming procedures against the current Microchip device documentation before use in production hardware.

MPLAB, MPLAB XC8, and PIC are trademarks of Microchip Technology Incorporated. All other marks belong to their respective owners.

Contents

Part I: Foundations	1
Chapter 1 — What Is a Microcontroller?	3
1.1 A whole computer on one chip	3
1.2 Two separate memories: the Harvard design	3
1.3 Inside the CPU: the working register	4
1.4 What an “instruction set” is	5
1.5 The shape of a PIC program	5
1.6 Common misconceptions	6
1.7 Try it yourself	7
1.8 Reference bridge	7
Chapter 2 — Meet the PIC Cores	8
2.1 Why there are “cores” at all	8
2.2 The cores, one at a time	9
2.3 Where the PIC16F17146 sits — and why it’s a good teacher	10
2.4 “Which core is my chip?”	10
2.5 Common misconceptions	11
2.6 Try it yourself	11
2.7 Reference bridge	11
Chapter 3 — Installing the Toolchain	13
3.1 The pieces you need	13
3.2 The tool you <i>don’t</i> need yet: hardware	14
3.3 Creating your first project	14
3.4 A minimal source file that builds	15
3.5 Build it	15
3.6 Run it on the simulator	16
3.7 What just happened	16
3.8 If you use MPLAB X IDE instead	16
3.9 Common problems	17
3.10 Try it yourself	18
3.11 Reference bridge	18
Part II: Your First Programs	19
Chapter 4 — Anatomy of an Assembly Source File	21
4.1 The idea: an assembly file is just a list of <i>statements</i>	21
4.2 The whole program, up front	22

4.3 Line by line	23
Comments — ;	23
PROCESSOR 16F17146 — naming the target	23
#include <xc.inc> — getting the register names	23
PSECT resetVec,class=CODE,delta=2 — declaring a program section	24
resetVec: and the reset vector	24
PSECT code and main:	25
END resetVec — that’s all, folks	25
4.4 The rules for names (identifiers)	25
4.5 Numbers, when you need them	26
4.6 Build & run it	26
On the command line	26
In your IDE	27
Watch it do nothing (productively)	27
4.7 What just happened	27
4.8 Common mistakes	28
4.9 Try it yourself	28
4.10 Reference bridge	29
Chapter 5 — Configuration Bits & the Reset Vector	30
5.1 The idea: settings you burn in <i>before</i> your code runs	30
5.2 Get the tokens from the selected DFP (then understand them)	31
5.3 The five decisions that matter for a first program	31
Decision 1 — What clock runs the chip? (CONFIG1)	31
Decision 2 — Can the watchdog reset me? (CONFIG3)	32
Decision 3 — What does the MCLR pin do, and how do I program the chip? (CONFIG2 + CONFIG4)	33
Decision 4 — Should the chip protect itself from brown-outs and bad startups? (CONFIG2)	33
Decision 5 — Lock the memory down? (CONFIG4 + CON- FIG5)	34
5.4 The upgraded program	35
5.5 Build & run it	37
5.6 What just happened	37
5.7 Common mistakes	38
5.8 Try it yourself	38
5.9 Reference bridge	39
Chapter 6 — Moving Data: W, Literals, and File Registers	40
6.1 The idea: almost everything flows through one register	40
6.2 Two kinds of “value”: literals and file registers	40
6.3 What counts as a “file register”?	41
6.4 The three instructions you need today	42
The ,w / ,f destination operand — don’t leave it off	42

6.5 The whole program	43
6.6 Build & run it — and actually watch	44
6.7 What just happened	45
6.8 Common mistakes	45
6.9 Try it yourself	45
6.10 Reference bridge	46
Chapter 7 — Your First Blink	47
7.1 The idea: direction, latch, and input mode	47
7.2 Which pin? Meet LED0	48
7.3 Configuring the pin, step by step	48
7.4 Making it blink: the delay	49
7.5 The whole program	50
7.6 Build & run it	52
7.7 What just happened	52
7.8 Common mistakes	53
7.9 Try it yourself	53
7.10 Reference bridge	54
 Part III: Memory: The Hard Part, Made Gentle	 55
Chapter 8 — Data Memory & Banking	57
8.1 The idea: an instruction can only point 128 bytes	57
8.2 What's in a bank	58
8.3 BANKSEL: setting the bank	58
8.4 BANKMASK: making the address fit	59
8.5 Placing your own variables in a bank	60
8.6 The whole program — blink, banked by hand	61
8.7 Build & run it — no crutch this time	62
8.8 What just happened	63
8.9 Common mistakes	63
8.10 Try it yourself	63
8.11 Reference bridge	64
 Chapter 9 — Program Memory & Paging	 65
9.1 The idea: goto and call can't reach the whole program	65
9.2 Why you often don't notice paging	66
9.3 PAGESEL and PAGEMASK: the manual fix	66
9.4 fcall and ljmp: let the assembler do it	67
9.5 The whole program — a genuine cross-page call	68
9.6 See the failure (do this once)	69
9.7 What just happened	69
9.8 Common mistakes	70
9.9 Try it yourself	70

9.10 Reference bridge	71
Chapter 10 — Psects: Organizing Code & Data	72
10.1 The idea: you write sections, the linker places them	72
10.2 A psect declaration, dissected	73
class= — <i>which</i> memory	73
delta= — how wide is an address	74
space= — resolving the great address collision	74
reloc= — alignment (you rarely touch it on mid-range)	74
10.3 Predefined vs. your own psects	74
10.4 Placing a psect: class vs. -P	75
10.5 The program — roll your own psects	75
10.6 Build it and read the map file	76
10.7 What just happened	77
10.8 Common mistakes	77
10.9 Try it yourself	78
10.10 Reference bridge	78
Chapter 11 — Linear Memory & Larger Variables	79
11.1 The idea: bytes are small, so glue them together	79
11.2 Incrementing a 16-bit value	79
11.3 The whole program — count to 65535 and watch it	80
11.4 Reaching data indirectly: the FSRs	81
The auto-increment moves	82
11.5 Walking a buffer with a pointer	83
11.6 When one bank isn't enough: linear memory	83
Reserving a buffer that really crosses banks	84
11.7 What just happened	85
11.8 Common mistakes	85
11.9 Try it yourself	86
11.10 Reference bridge	86
Part IV: Building Real Programs	87
Chapter 12 — Directives You'll Actually Use	89
12.1 The idea: directives talk to the assembler, not the chip	89
12.2 Named constants: EQU and SET	89
12.3 Reserving vs. initializing memory	90
DS — reserve RAM (you've used this)	90
DB/DW — bake constant data into the program	91
12.4 The program — a DB lookup table you read at run time	91
12.5 Build-time decisions: conditional assembly	93
Fail the build on purpose: ERROR and MESSG	93
12.6 A quick tour of the rest	94

12.7 What just happened	94
12.8 Common mistakes	95
12.9 Try it yourself	95
12.10 Reference bridge	96
Chapter 13 — Macros & Multiple Source Files	97
13.1 The idea: two very different kinds of reuse	97
13.2 Macros: patterns the assembler pastes in	98
13.3 Splitting code across files: GLOBAL and EXTRN	99
13.4 The main file — using both	100
13.5 Building a multi-file project	101
Single-step — list all the source files	101
Multi-step — assemble once, link once (incremental builds)	101
13.6 Bundling modules into a library: the archiver	102
13.7 What just happened	102
13.8 Common mistakes	102
13.9 Try it yourself	103
13.10 Reference bridge	104
Chapter 14 — Interrupts on the PIC16F17146	105
14.1 The idea: let hardware interrupt you	105
14.2 Turning an interrupt on	106
The PEIE subtlety worth knowing	106
14.3 The gift of automatic context saving	106
14.4 The rule you must never break: clear the flag	107
One vector means you must dispatch	107
14.5 Setting up TMR0 to overflow periodically	108
14.6 The whole program — a timer-driven blink	108
14.7 What just happened	110
14.8 Common mistakes	110
14.9 Try it yourself	111
14.10 Reference bridge	112
Chapter 15 — Moving Up a Core: PIC18 Interrupts	113
15.1 Why move up — and what stays the same	113
15.2 The assembler-level differences	113
The delta/reloc swap	114
The Access bank — PIC18's "common RAM," bigger	114
movff — copy anywhere to anywhere	115
15.3 PIC18 interrupts: two models	115
15.4 The vectored interrupt controller, by example	116
15.5 What actually changed, at a glance	119
15.6 What just happened	119
15.7 Common mistakes (porting from mid-range)	120
15.8 Try it yourself	120

15.9 Reference bridge	121
Chapter 16 — The Compiled Stack	122
16.1 The idea: locals without a data stack	122
16.2 The tools: the <code>FN</code> directives	123
16.3 A worked example	124
16.4 Seeing the overlap in the map file	125
16.5 On the PIC16F17146	126
16.6 What just happened	126
16.7 Common mistakes	127
16.8 Try it yourself	127
16.9 Reference bridge	128
Part V: Toolchain Mastery and Going Further	129
Chapter 17 — The Linker & Map Files	131
17.1 The idea: the linker places, the map reports	131
17.2 Getting a map file	131
17.3 What's in a map file	132
17.4 The psect listing — and the columns that matter	132
17.5 Unused ranges and the symbol table	133
Free memory (§6.3.2.5)	133
Where your symbols went (§6.3.2.6)	134
17.6 Hands-on: read your blink's map	134
17.7 Diagnosing placement problems	135
17.8 What just happened	135
17.9 Common mistakes	135
17.10 Try it yourself	136
17.11 Reference bridge	136
Chapter 18 — Utilities, Hex Files & Programming the Chip	138
18.1 The idea: the HEX file is the deliverable	138
18.2 The gotcha: HEX addresses are byte addresses	139
18.3 Hexmate: massaging the HEX file	139
18.4 Programming the PIC16F17146 Curiosity Nano	140
Method 1 — from your IDE (the normal way)	140
Method 2 — Drag-and-drop (no IDE needed)	140
18.5 Run your blink on real silicon	141
18.6 What just happened	141
18.7 Common mistakes	142
18.8 Try it yourself	142
18.9 Reference bridge	143

Chapter 19 — Baseline and Enhanced-Baseline Devices (Optional Finale)	144
19.1 The idea: the same skills, a smaller box	144
19.2 The baseline call restriction	145
19.3 The ENTRY class: letting the linker handle it	146
19.4 A baseline example	146
19.5 Seeing ENTRY in the map file	148
19.6 What just happened (and a look back)	149
19.7 Common mistakes	149
19.8 Try it yourself	150
19.9 Reference bridge	150
Appendix A — Instruction Set Quick Reference	151
A.1 Byte-oriented operations	151
Byte-oriented skip	152
A.2 Bit-oriented operations	152
A.3 Literal operations	152
A.4 Control operations	153
A.5 Inherent operations	153
A.6 Indirect / FSR operations	153
A.7 Assembler conveniences (not part of the 50)	154
A.8 Cross-core delta cards	154
A.9 Reference	155
Appendix B — Directive Reference Card	156
B.1 The directives you'll use constantly	156
B.2 The rest of the directives (reference)	157
B.3 PSECT flags (the ones that matter)	158
Common psect declarations, ready to copy	158
B.4 Number formats (quick reminder)	159
B.5 Reference	159
Appendix C — Decoding Error & Warning Messages	160
C.1 How to read a message	160
C.2 How to look one up	161
C.3 The fifteen most common beginner problems	161
Build won't even start	162
Assembler errors (your .s)	162
Linker errors (placement & symbols)	162
Config & runtime (builds, but doesn't run right)	163
C.4 A worked lookup	163
C.5 Reference	164
Appendix D — MPLAB X & pic-as Option Reference	165
D.1 Options this book actually uses	165

D.2 The broader option set (from Table 3-1)	166
D.3 Long command lines (@file)	167
Reproducible pack-specific examples	167
D.4 Reference	168
Appendix E — MPASM → XC8 Migration Notes	169
E.1 The high-frequency changes	169
E.2 Things that trip up MPASM porters	170
E.3 A tiny before/after	170
E.4 Reference	171
Appendix F — Glossary	172
Appendix G — From Assembly to C	176
G.1 Why C, and why assembly still matters	176
G.2 The same tool underneath	176
G.3 Inline assembly inside C	177
G.4 Mixing whole assembly modules with C	177
G.5 A suggested path forward	178
G.6 Reference	178

PART I

Foundations

Chapter 1 — What Is a Microcontroller?

***What you'll take away:** a clear mental picture of what's actually inside the chip you're about to program — the CPU, the two kinds of memory, the working register, and the idea of an instruction set. No code yet; this chapter builds the map you'll navigate for the rest of the book. It ends with a labeling exercise, not a build.*

1.1 A whole computer on one chip

A **microcontroller** (MCU) is a complete little computer squeezed onto a single chip: a processor, its memory, and a set of built-in peripherals, all together. That's the difference from the CPU in a laptop — a desktop processor needs external RAM chips, storage, and support hardware, while a microcontroller has everything it needs to run a program on board the moment you power it up.

Our chip for this book, the **PIC16F17146**, is a textbook example. Its block diagram (data sheet DS40002343F, Figure 1) shows three things clustered together (data sheet §“Block Diagram”):

- a **CPU** — the part that executes instructions,
- **memory** — Program Flash Memory, Data Memory (RAM), and Data EEPROM,
- **peripherals** — timers, an analog-to-digital converter, serial ports, I/O ports, and more.

You write a program, it gets stored in the chip's flash, and on power-up the CPU starts executing it — driving pins, reading sensors, talking to other devices. That's embedded programming: software that reaches out and touches the physical world.

1.2 Two separate memories: the Harvard design

Here's the first idea that shapes *everything* in PIC assembly. On a PIC, **program memory and data memory are physically separate**, on their own buses. The data sheet states it plainly (§9):

“In Harvard architecture devices, the data and program memories use separate buses that allow for concurrent access of the two memory spaces.”

This is the **Harvard architecture**, and it has two big consequences you’ll feel throughout the book:

- **Your instructions live in one place, your variables in another.** Program Flash holds the code; Data RAM holds the variables your code manipulates. They’re different memories with their *own address 0* — program address 0 and data address 0 are completely different physical locations. (This is why Chapter 10’s space flag and Chapter 17’s map-file “Space” column exist — the tools must always know *which* memory you mean.)
- **The chip can fetch the next instruction while working on data**, because the two buses don’t compete. That’s part of why PICs are efficient at their clock speed.

The PIC16F17146 actually has **three** memory regions (data sheet §9):

Memory	Holds	Volatile?
Program Flash Memory	your program (instructions) + constants	no — keeps its contents with power off
Data Memory (RAM)	your variables while the program runs	yes — loses its contents when power is removed
Data EEPROM	small amounts of data you want to survive power-off	no

For now, remember the headline: **code and variables live in separate worlds**. Keeping that straight is half of understanding PIC assembly.

1.3 Inside the CPU: the working register

The CPU is where instructions execute. The PIC16F17146 uses what Microchip calls an **enhanced mid-range 8-bit core** with **50 instructions** and, at its center, a single 8-bit scratch register called **W — the Working register** (data sheet §7). A great many operations flow through W: you load a value into it, the arithmetic-logic unit (ALU) combines it with something, and the result can go back to W or to a file register.

Three pieces of the CPU are worth naming now (you'll meet them all again):

- **W (Working register)** — the 8-bit accumulator that most instructions read from or write to.
- **The ALU** — the arithmetic-logic unit that adds, subtracts, and does bitwise operations.
- **The STATUS register** — a set of flags (like “the last result was zero” or “there was a carry”) that the ALU updates as a side effect.

“8-bit” means the natural unit of work is a **byte** — a value from 0 to 255. When you need bigger numbers you'll glue bytes together (Chapter 11), but the CPU handles them one byte at a time.

1.4 What an “instruction set” is

The CPU only understands a fixed, finite menu of very small operations — its **instruction set**. Each instruction does one tiny thing: “load this constant into W,” “add this register to W,” “if this bit is set, skip the next instruction,” “jump to that address.” The PIC16F17146's menu has exactly **50 instructions** (data sheet §7.4).

That's the whole vocabulary. There's no `print`, no `if/else`, no `while` — you *build* those out of the primitive instructions. Assembly programming is the craft of expressing what you want using only that small menu. It feels restrictive at first and then, surprisingly quickly, becomes a kind of clarity: nothing is hidden, and you know exactly what the chip does on every cycle.

Each instruction is stored as a number (an **opcode**) in program memory. The **assembler** — the tool this book is about — is what translates the human-readable mnemonics you type (`movlw`, `addwf`, `goto`) into those opcodes. You write `movlw 0x2A`; the assembler emits the bits the CPU recognizes.

1.5 The shape of a PIC program

Putting it together, here's the life of a PIC program, which the rest of the book fills in:

1. You write assembly instructions in a text file.

2. The **assembler** turns them into opcodes and the **linker** places them at real addresses in program flash, producing a **HEX file** (Chapters 17–18).
3. A **programmer** burns the HEX into the chip’s flash (Chapter 18).
4. On power-up or reset, the CPU begins fetching and executing instructions from a fixed starting point — the **reset vector** (Chapter 4).
5. The firmware remains in control until a reset or loss of power; it normally loops, waits, or sleeps rather than “exiting” to an operating system.

You now have the map: a CPU with a working register and a 50-instruction menu, two separate memories for code and data, peripherals to reach the outside world, and a toolchain that turns your text into something the chip runs.

1.6 Common misconceptions

Belief	Reality
“Variables and code share one memory like on a PC”	PICs are Harvard: program and data memory are separate, with their own address spaces (§9)
“The CPU can do big operations in one step”	it’s 8-bit — it works a byte at a time; larger values are built up (Chapter 11)
“Assembly has loops and if-statements”	it has only ~50 primitive instructions; you build control flow from them
“The chip needs external RAM/storage”	a microcontroller has its memory and peripherals on board

Volatile does **not** mean “automatically initialized to zero.” General-purpose RAM does not have a defined power-up value on this device; startup code must initialize every variable before relying on it. Volatile means only that RAM does not retain its state when power is removed.

1.7 Try it yourself

1. **Label the block diagram.** Open the PIC16F17146 data sheet to its Block Diagram (Figure 1) and identify, in your own words, the CPU, the three memories, and three peripherals. Sketch it.
 2. **Sort the list.** For each item — “an LED blink pattern,” “the current sensor reading,” “a calibration constant that must survive power-off” — decide whether it belongs in Program Flash, Data RAM, or Data EEPROM, and why.
 3. **Count to 255.** Explain in one sentence why an 8-bit register can hold values only from 0 to 255, and predict what happens when you add 1 to 255. (You’ll confirm it in Chapter 11.)
-

1.8 Reference bridge

- **Data sheet §7 “Enhanced Mid-Range CPU”** — the core, the W register, the ALU, and the 50-instruction set at a glance.
- **Data sheet §9 “Memory Organization”** — the three memories and the Harvard split.
- **Data sheet “Block Diagram” (Figure 1)** — the whole chip on one page.

Next chapter: the PIC16F17146 is one of several PIC “cores.” Chapter 2 places it in the family — baseline, mid-range, enhanced mid-range, and PIC18 — so you understand what’s specific to our chip and what carries across the whole 8-bit PIC line.

Chapter 2 — Meet the PIC Cores

*What you’ll take away: a map of the 8-bit PIC family — **baseline**, **enhanced baseline**, **mid-range**, **enhanced mid-range**, and **PIC18** — so you know exactly where our PIC16F17146 sits and what you’re learning that transfers to every other PIC. Knowing the cores tells you which chapters of this book apply to which chips.*

2.1 Why there are “cores” at all

Microchip’s 8-bit PICs span everything from a 6-pin chip costing pennies to a feature-packed 40-pin controller. To cover that range they share a handful of **cores** — CPU designs that differ mainly in the **width of their instruction set** and the features built around it. Pick a core and you’ve picked the instruction menu, the memory style, and the interrupt model.

The assembler groups every device into one of these cores, recorded as an **ARCH value** in the device’s INI file (data sheet info comes from your Device Family Pack; User’s Guide §2.1). Here’s the whole family, straight from the User’s Guide §2.1:

Core	Instruction width	Notes	ARCH
Baseline	12-bit	smallest, fewest instructions (PIC10/12/16)	PIC12
Enhanced baseline	12-bit	baseline + extra instructions; <i>some</i> support interrupts	PIC12E / PIC12IE
Mid-range	14-bit	more instructions, larger banks and pages	PIC14
Enhanced mid-range	14-bit	mid-range + more instructions and features	PIC14E / PIC14EX

Core	Instruction width	Notes	ARCH
PIC18	16-bit	expanded register set; extended data memory and vectored interrupts on some parts	PIC18 / PIC18XV

Read it as a ladder. As you climb, the instruction set widens (12 → 14 → 16 bits), more instructions appear, memory grows, and the interrupt system gets richer.

2.2 The cores, one at a time

Baseline (12-bit). The floor of the range — tiny PIC10, PIC12, and small PIC16 parts. A 12-bit instruction set means the fewest instructions and the tightest memory. Most baseline parts have **no interrupts**, and their `call` instruction has a placement quirk you’ll meet in Chapter 19. Great for the smallest, cheapest jobs.

Enhanced baseline (12-bit). Still a 12-bit instruction set, but with additional instructions; some of these chips add interrupt support (the PIC12IE ARCH), which basic baseline lacks (User’s Guide §2.1).

Mid-range (14-bit). The classic PIC middle ground — a 14-bit instruction set with more instructions than baseline, plus larger data-memory banks and program-memory pages (User’s Guide §2.1). This is the world of banking and paging you’ll spend Part III mastering.

Enhanced mid-range (14-bit). Mid-range plus extra instructions and features — and **this is our core**. The data sheet confirms the PIC16F17146 “contains an enhanced mid-range 8-bit CPU core” with 50 instructions, automatic interrupt context saving, a 16-level stack, and two File Select Registers (data sheet §7). It has the modern conveniences — a single vectored interrupt with hardware context saving (Chapter 14), linear data memory (Chapter 11), and a rich peripheral set — while keeping the 14-bit instruction set.

PIC18 (16-bit). The top of the 8-bit line: a 16-bit instruction set and an expanded register set. Interrupt hardware varies by device: traditional PIC18 parts use fixed high/low interrupt vectors, while some

newer parts add extended data memory and a **vectored interrupt controller** with programmable priorities (User’s Guide §2.1). Chapters 15 and 16 visit PIC18 so your skills carry upward.

2.3 Where the PIC16F17146 sits — and why it’s a good teacher

Our chip is **enhanced mid-range**, deliberately chosen as the spine of this book because it’s the sweet spot for learning:

- It’s **modern and full-featured** — the peripherals, interrupt model, and memory layout match what you’ll meet on current parts, not a 1990s design.
- It still has the **core challenges that make you a real PIC programmer** — banking and paging (Part III) — which the simplest chips hide and the biggest chips paper over.
- What you learn **transfers both directions**: down to baseline (Chapter 19) and up to PIC18 (Chapters 15–16), because every core shares the same fundamentals — a W register, file registers, psects, a reset vector, and the same assembler.

Learn this one core deeply and you can sit down with almost any 8-bit PIC data sheet and be productive.

2.4 “Which core is my chip?”

When you pick up any PIC, identify its core first — it tells you which rules apply:

1. **Check the data sheet’s CPU chapter.** It will name the core (e.g. “enhanced mid-range 8-bit CPU,” as ours does in §7).
2. **Check the ARCH value** in the device’s INI file if you’re unsure (User’s Guide §2.1) — PIC14E means enhanced mid-range, PIC18 means PIC18, and so on.
3. **Match it to this book:** the main chapters target enhanced mid-range; Chapter 19 covers baseline differences; Chapters 15–16 cover PIC18 differences.

A rough field guide by part number (always confirm against the data sheet): **PIC10/PIC12** are often baseline; many **PIC16F1xxxx** parts

(like ours) are enhanced mid-range; **PIC18F...** parts are PIC18. But the number alone isn't definitive — the data sheet is the authority.

2.5 Common misconceptions

Belief	Reality
"All PICs use the same assembly"	the cores differ in instruction width and features; syntax is shared but capabilities aren't (§2.1)
"A bigger part number means a different language"	it means a different <i>core</i> ; the fundamentals (W, psects, reset vector) are the same
"Enhanced mid-range is just mid-range"	it adds instructions, features, automatic interrupt context saving, and linear memory (§7)
"Every PIC18 has the same interrupt system"	PIC18 devices range from fixed high/low vectors to newer vectored, prioritized controllers; check the exact device (§2.1; Chapter 15)

2.6 Try it yourself

1. **Place three chips.** Look up a PIC10F200, a PIC16F17146, and a PIC18F57Q43, and state each one's core. Which chapters of this book apply most directly to each?
2. **Find the ARCH.** In the PIC16F17146 data sheet §7, find the sentence that names its core. Confirm it's enhanced mid-range.
3. **Predict the transfer.** Name one skill from later chapters (say, banking) and argue whether it applies to baseline, to PIC18, or to both.

2.7 Reference bridge

- **User's Guide §2.1 "Device Description"** — the authoritative descriptions of all five cores and their ARCH values.

- **Data sheet §7 “Enhanced Mid-Range CPU”** — confirms our chip’s core and its capabilities.

Next chapter: you know what a microcontroller is and which core we’re using. Chapter 3 gets the tools onto your computer — MPLAB X IDE, the XC8 assembler, and the Device Family Pack for the PIC16F17146 — and builds your first (empty) project, so you’re ready to write real code in Part II.

Chapter 3 — Installing the Toolchain

***What you'll build:** a working development setup — **VS Code** with the **MPLAB Extension Pack**, the **MPLAB XC8** assembler, and the **Device Family Pack** for the PIC16F17146 — and an empty project that builds cleanly. No hardware required: you'll run it on the **simulator** via the **MPLAB Debugger**. By the end you have a green build and are ready to write real code in Part II.*

***Which tools?** Microchip now delivers its 8-bit workflow as **MPLAB Extensions for Visual Studio Code** (its modern, recommended path), alongside the older **MPLAB X IDE**. This book uses the **VS Code** workflow for the GUI steps and a **command-line** `pic-as` invocation for every code listing (so each example is exactly reproducible in any terminal). If you prefer **MPLAB X IDE**, a short mapping is in §3.8 — the concepts are identical.*

3.1 The pieces you need

Four things turn your typing into a program on a chip:

1. **Visual Studio Code** — the editor. Free from code.visualstudio.com/download
2. **The MPLAB Extension Pack** — Microchip's suite of VS Code extensions (project wizard, CMake build, the **MPLAB Debugger** debug adapter, toolchain manager, Hexmate, MCC). Install it from the VS Code **Marketplace**: open Extensions (**Ctrl+Shift+X**), search *MPLAB Extension Pack*, and install (verified: Microchip developerhelp, "Get Started for MPLAB X IDE Users New to VS Code").
3. **MPLAB XC8** — the compiler package that includes the **PIC assembler**, whose command-line driver is `pic-as` (User's Guide §3). This is the tool that turns your `.s` files into a HEX file; the extensions call it for you.
4. **The Device Family Pack (DFP)** — a package that teaches the tools your specific chip's registers, config bits, and memory. The assembler's device info "typically comes from your selected Device Family Pack" (User's Guide §2.1). You need the pack covering the PIC16F17146 (the `PIC16F1xxxx_DFP`).

The commands and UI names here were verified with the **MPLAB Extension Pack** (extensions `mplab-core-da`, `runcmake`, `toolchains`), **MPLAB XC8 4.00**, and **PIC16F1xxx_DFP 1.31.465**. Record the XC8 and DFP versions you actually select — later versions can move a control or change a config-token spelling (Chapter 5).

3.2 The tool you *don't* need yet: hardware

You can write, build, and run this book's early chapters **without buying any hardware**, because the MPLAB Debugger includes a **simulator** — a software model of the chip. It lets you single-step instructions, watch registers change, and confirm your logic, all on your PC (User's Guide §2.2 lists the simulator among the compatible tools).

The simulator proves CPU state and the peripherals it models; it does **not** prove board wiring, voltage levels, oscillator tolerance, or LED current. Treat a simulator pass as a *logic* check and do a hardware check before making electrical or timing claims.

When you *are* ready for real silicon, the tools work with all Microchip programmers and boards — including the **PIC16F17146 Curiosity Nano**, whose on-board debugger programs the chip over USB with no separate programmer (Curiosity Nano guide §3.1).

This book's approach: simulator-first. Every example runs on the simulator; the Curiosity Nano is an optional “run it on hardware” step (Chapter 18).

3.3 Creating your first project

VS Code drives everything through the **Command Palette** — press **Ctrl+Shift+P** (a fast shortcut is typing `>` in the search bar). Every MPLAB command starts with `MPLAB:.` To make a project (verified: `developerhelp` “Workflow for MPLAB Extensions for VS Code”):

1. Run **MPLAB: Create New Project** and follow the wizard.
2. **Device:** enter `PIC16F17146` and select it. (If it's not offered, install its DFP — the extension's pack manager fetches it.)
3. **Toolchain:** choose the **XC8 (pic-as)** assembler toolchain — we're writing assembly, not C.

4. Name the project and finish.

You now have a project folder targeting the PIC16F17146, ready for source files. (Already have an MPLAB X IDE project? Just **File** → **Open Folder** on it and accept the import suggestion.)

3.4 A minimal source file that builds

Add a source file named with an uppercase **.S** (the uppercase matters — Chapter 4 explains why) and put in the smallest thing that assembles — the idle loop you’ll dissect next chapter:

```
PROCESSOR 16F17146
#include <xc.inc>

PSECT resetVec,class=CODE,delta=2
resetVec:
    goto    $          ; loop here forever

    END        resetVec
```

3.5 Build it

In VS Code, build with **Ctrl+Shift+B**, or run **MPLAB CMake: Full Build** from the Command Palette, or click the **Hammer** in the status bar (verified: developerhelp workflow page — the extensions drive the build through CMake). A successful build produces the object, ELF, and HEX files in the project’s build output.

Command-line equivalent (used for every listing in this book). So each example is reproducible in any terminal, the book gives the raw `pic-as` command:

```
DFP=/path/to/Microchip/PIC16F1xxx_DFP/1.31.465/xc8
pic-as -mcpu=16f17146 -mdfp="$DFP" -Wl,-presetVec=0h spin.S
```

XC8 4.00 note: a command-line build needs `-mdfp=<DFP-xc8-directory>` — point it at the pack’s `xc8` subdirectory (not the pack root), or the driver reports “no device-support files” (errors 2103/2104). **VS Code and MPLAB X supply this automatically.** The `pic-as` binary lives under your XC8 install (e.g. `.../xc8/vN.NN/pic-as/bin/pic-as`);

the DFP lives under the MPLAB packs directory or your ~/.mchp_packs. Later chapters assume the DFP variable above is set.

3.6 Run it on the simulator

Now step it in the simulator via the **MPLAB Debugger** (verified: developerhelp workflow page):

1. Run **Debug: Add Configuration** and choose **MPLAB Debugger** — this creates a debug configuration (a `launch.json`) that defaults to the simulator when no board is connected.
2. Press **F5** (or use the **Run and Debug** view) to start the session. Execution stops at the program's start.
3. **Step** one instruction (**F10**): you land on `goto $`. Step again — the Program Counter doesn't move. That frozen PC is your idle loop working.

Use the **Run and Debug** view's **Variables/Watch** panels to watch `w` (as `WREG`), registers, and memory change as you step. These are the panels you'll live in for the next several chapters. You've closed the loop: **write** → **build** → **simulate**, no hardware, no risk.

3.7 What just happened

You installed the modern toolchain — **VS Code + the MPLAB Extension Pack**, the **XC8/pic-as assembler**, and the **PIC16F17146 DFP** — created an assembly project, built a minimal source file, and stepped it on the **simulator** with the MPLAB Debugger. You confirmed the whole pipeline works before writing a single “real” line. That's the professional order of operations: get a trivial thing building and running first, *then* add complexity.

3.8 If you use MPLAB X IDE instead

The older MPLAB X IDE does the same jobs; the command names differ (both paths are supported by Microchip):

Task	VS Code + MPLAB Extensions	MPLAB X IDE
New project	MPLAB: Create New Project (Ctrl+Shift+P)	File → New Project → Standalone
Pick device	in the wizard: PIC16F17146	wizard: device PIC16F17146
Toolchain	XC8 (pic-as)	pic-as toolchain (not the C compiler)
Build	Ctrl+Shift+B / MPLAB CMake: Full Build	the Hammer (Build) button
Simulate/debug	Debug: Add Configuration → MPLAB Debugger, F5	set Tool = Simulator , press Debug
Watch registers	Run and Debug → Variables/Watch	Window → Debugging → Watches

Everything else in this book — the assembly, the pic-as command lines, the map/HEX files — is identical either way.

3.9 Common problems

Symptom	Cause	Fix
No MPLAB: commands in the palette	Extension Pack not installed/enabled	install <i>MPLAB Extension Pack</i> (Ctrl+Shift+X) and reload VS Code
PIC16F17146 not offered	its DFP isn't installed	let the extension's pack manager fetch the PIC16F1xxx_DFP
Build produces C, not assembly	picked the C toolchain	choose the XC8 (pic-as) assembler toolchain
#include <xc.inc> not found	file saved as lowercase .s (no preprocessing)	use an uppercase .S extension (Chapter 4)
Terminal build: error 2103/2104	XC8 4.00 not given its DFP	pass <code>-mdfp="\$DFP"</code> pointing at the pack's xc8 directory

Symptom	Cause	Fix
Debug session won't start	no debug configuration, or backend not ready	run Debug: Add Configuration → MPLAB Debugger first; ensure the MPLAB platform backend finished installing

3.10 Try it yourself

1. **Green light.** Get §3.4's file to build successfully — in VS Code (**Ctrl+Shift+B**) *and* from the terminal with the `pic-as` command. This working toolchain is the milestone everything else builds on.
2. **Step the loop.** Start the MPLAB Debugger (**F5**), step three times, and watch the Program Counter freeze on `goto $`. Explain what you're seeing.
3. **Find the versions.** Note your XC8 version and DFP version (the Command Palette's MPLAB: Edit Project Properties shows them). You'll need the DFP version in Chapter 5.

3.11 Reference bridge

- **Microchip developerhelp** — “Workflow for MPLAB Extensions for VS Code” and “Get Started for MPLAB X IDE Users New to VS Code” — the authoritative command list (create/build/debug) this chapter follows.
- **User's Guide §2.2 / §3.5** — compatible tools, and how project properties map to `pic-as` options.
- **XC8 4.00 Release Notes** — the installed tool version and the command-line DFP requirement.
- **Curiosity Nano guide §3.1** — the on-board debugger, for when you move to hardware (Chapter 18).

Next chapter — and Part II: your tools work. Chapter 4 dissects that minimal source file line by line — statements, labels, directives, the reset vector, and the `goto $` idle loop — so every character means something. From here on, you write real code.

PART II

Your First Programs

Chapter 4 — Anatomy of an Assembly Source File

What you'll build: *a complete, buildable PIC16F17146 assembly program whose only job is to start up and spin forever in a one-line endless loop. It does nothing useful — and that is exactly the point. By the end of this chapter you will recognize every single character in a .s file and know why it's there.*

4.1 The idea: an assembly file is just a list of *statements*

A C file is made of statements terminated by semicolons and grouped with braces. An assembly file is simpler and flatter: it is a **list of lines**, and each line is one *statement*. The assembler reads them top to bottom. An actual instruction mnemonic normally becomes one machine instruction; assembler pseudo-instructions, directives, and macros can emit zero, one, or several instructions or data bytes.

The MPLAB XC8 PIC Assembler recognizes exactly five statement shapes. Everything you will ever write is one of these:

Shape	Looks like	Purpose
A label alone	loop:	names the current spot in memory
Label + instruction	loop: goto loop	a named machine instruction
A directive	PSECT code	an order to the <i>assembler</i> , not the chip
A comment alone	; spin forever	a note for humans
A blank line		breathing room

That's the whole grammar. There is **no required column layout** — you can indent however you like; whitespace and tabs are just separators. (Source: XC8 PIC Assembler User's Guide §4.2, "Statement Formats.")

Instruction vs. directive — the distinction that trips up every beginner. A machine instruction (`goto`, `movlw`, `clrf`) becomes *bits*

inside the chip's program memory and executes at run time. A directive (PSECT, PROCESSOR, CONFIG, END) is an instruction to the assembler program on your PC. Most directives emit no executable instruction, although data directives emit data and pseudo-ops such as BANKSEL/PAGESEL can emit instructions. Same file, two audiences. Keep asking "who is this line talking to — the chip, or the assembler?"

4.2 The whole program, up front

Here it is — the entire file. Save it as `spin.S` (note the **uppercase .S**; we'll explain why in §4.7). Read past it; every line is dissected below.

```
; -----
; spin.S — the smallest honest PIC16F17146 program
; Starts up, then loops forever doing nothing.
; -----

    PROCESSOR 16F17146          ; tell the assembler which chip we're targeting
#include <xc.inc>              ; pull in the register/SFR names for this chip

; --- Reset entry point -----
; The PIC16F17146 begins executing at program address 0x0000
; after any Reset (verified: data sheet DS40002343F §9.1).
    PSECT resetVec,class=CODE,delta=2
resetVec:
    goto     main              ; jump to our real code

; --- Main program -----
    PSECT code
main:
    goto     $                 ; "$" means "this instruction" → endless loop

    END         resetVec      ; end of source; entry point is resetVec
```

Build it (we'll walk through this command in §4.6):

```
pic-as -mcpu=16f17146 -mdfp="$DFP" -WL,-presetVec=0h \
      -Wa,-a -WL,-Map=spin.map spin.S
```

If it produces `spin.hex` with no errors, you have just assembled a PIC program. Everything from here is understanding *why each line was necessary*.

4.3 Line by line

Comments — ;

```
; Starts up, then loops forever doing nothing.
```

A semicolon that isn't inside a string begins a comment; everything to its end-of-line is ignored by the assembler (User's Guide §4.4). Comments can sit on their own line or trail an instruction:

```
goto    $    ; endless loop
```

Trap: because our file is preprocessed (uppercase `.S`), you may also use C-style `//` and `/* ... */` comments. But **never** put a `;` comment on a `#define` line — the preprocessor doesn't strip `;` comments, so the text leaks into your macro and breaks the build. On preprocessor lines, use `//`. (User's Guide §4.4.)

PROCESSOR 16F17146 — naming the target

This directive tells the assembler which device's rules and register set to use. It must match the chip. We verified the exact part is the **PIC16F17146**, one of the PIC16F17126/46 family, an *enhanced mid-range* core (data sheet DS40002343F, cover & §9.1).

#include <xc.inc> — getting the register names

Out of the box, the assembler does **not** know that `LATA`, `PORTC`, or `TRISA` mean anything — Special Function Register names are not built in (User's Guide §4.6.4). The line

```
#include <xc.inc>
```

pulls in the header that *equates* every SFR name to its hardware address for your selected chip. After this line, writing `LATA` in an instruction refers to the real Port A latch. The header also defines handy per-bit symbols (e.g. `RB5`) and field equates of the form `REGISTERNAME_FIELDNAME_POSN`. We use none of them in `spin.S`, but every later program needs this line, so it's a habit worth forming now.

PSECT resetVec,class=CODE,delta=2 — declaring a program section

A **psect** (“program section”) is a named box that your code and data live in; the linker later places each box at a real address. This is *the* concept the official guides assume you already know — Chapter 10 is devoted to it. For now, absorb just three things about this line:

- **resetVec** is the name we chose for this box.
- **class=CODE** says “this box holds executable instructions,” so the linker puts it in program memory.
- **delta=2** says “each address in this memory space is represented by 2 data bytes.” On the PIC16F17146, program memory is word-addressed and each implemented word is **14 bits** wide, so a custom program-space psect must use **delta=2** (User’s Guide §4.9.48.4). The device implements 16,384 words at 0x0000–0x3FFF even though its 15-bit PC can address a 32K-word architectural space (data sheet §9.1). The assembler-provided code psect already has the correct flags. PIC18 program memory is byte-addressed (**delta=1**) and executable psects normally use **reloc=2** for instruction alignment; do not copy this declaration to PIC18 unchanged.

resetVec: and the reset vector

```
resetVec:
    goto    main
```

resetVec: is a **label** — a name for “the address of the next instruction” (User’s Guide §4.6.5). Why does this one matter so much? Because the PIC16F17146 **always begins executing at program address 0x0000 after a Reset** (verified: data sheet DS40002343F §9.1 — “The Reset vector is at 0000h”). In §4.6 we tell the linker to place the **resetVec** psect *at* address 0. So the very first thing the chip does on power-up is run this **goto main**.

Why jump instead of putting **main** here directly? Because address 0x0004 is the **interrupt vector** (also §9.1). Real programs need those first few locations kept clear so an interrupt can land at 0x0004 without colliding with your main code. Jumping out immediately is the standard, safe pattern — you’ll reuse it in every program.

PSECT code and main:

```
PSECT code
main:
    goto    $
```

A second box, named `code`, holding the body of the program. `main:` labels its first instruction.

The star of the show is `goto $`. The symbol **\$ means “the address of this very instruction”** (User’s Guide §4.6.3). So `goto $` jumps to itself, forever — a deliberate, one-line infinite loop. On a real chip the CPU would sit here spinning; on the simulator you’ll watch the program counter freeze at this address. It’s the assembly equivalent of `while(1);`.

***Why \$ counts differently on different chips.** Any offset you add to \$ is measured in the chip’s native addressing. On mid-range parts like ours, program memory is word-addressable, so `$+1` means “one instruction later.” On PIC18 (byte-addressable) `$+2` is the next instruction. You won’t hit this in `spin.S`, but file it away (User’s Guide §4.6.3).*

END resetVec — that’s all, folks

```
END    resetVec
```

The `END` directive marks the end of the source file. Naming `resetVec` after it identifies the program’s entry point. Lines after `END` are ignored.

4.4 The rules for names (identifiers)

You invented three names above: `resetVec`, `main`, and the `psect` names. The assembler’s rules for legal names (User’s Guide §4.6):

- Made of letters, digits, and the special characters `$`, `?`, `_`.
- **Cannot start with a digit or \$.**
- **Case-sensitive** — `Main`, `main`, and `MAIN` are three different symbols. (Note that *instruction mnemonics and directives* are **not** case-sensitive: `goto`, `GOTO`, and `GoTo` are the same instruction. Only your identifiers care about case.)

- A name can't collide with a mnemonic (goto), a directive (PSECT), or an operator (mod, and).

Legal: resetVec, _temp, loop1, ?flag. Illegal: 1st (starts with digit), \$x (starts with \$), goto (reserved).

4.5 Numbers, when you need them

spin.S uses only the literal 0h, but you'll need the number rules immediately after this chapter. The assembler's default radix is **decimal**, and other bases use a suffix or the 0x prefix (User's Guide §4.5):

Base	How to write 255	Rule
Decimal	255 or 255D	default; no suffix needed
Hexadecimal	0xFF or 0FFh	0x prefix or h suffix; a h-suffixed number must start with a digit, so 0FFh, not FFh
Binary	11111111B	uppercase B suffix
Octal	377o or 377q	o/q suffix

Trap: FFh is an error — the assembler thinks it's an identifier. Write 0FFh. This is the single most common “why won't my constant assemble?” mistake.

4.6 Build & run it

On the command line

```
pic-as -mcpu=16f17146 -mdfp="$DFP" -Wl,-presetVec=0h \  
-Wa,-a -Wl,-Map=spin.map spin.S
```

Decoding the options (patterned on the Embedded Engineers guide §4.4):

- **-mcpu=16f17146** — the target device. Must match your PROCESSOR directive.

- `-mdfp="$DFP"` — the XC8 4.00 command-line path to the selected DFP's `xc8` directory; Chapter 3 shows how to set `DFP`. MPLAB X supplies it from the project pack.
- `-Wl,-presetVec=0h` — a *linker* option (`-Wl`, = “pass to linker”). It places our `resetVec` psect at address 0, so our `goto main` lands on the reset vector the chip jumps to. This is the line that makes the whole thing actually run.
- `-Wa,-a` — a *assembler* option (`-Wa`, = “pass to assembler”) requesting a **list file** (`spin.lst`) showing the machine code generated for each source line. Open it — it’s the best way to *see* your instructions become bits.
- `-Wl,-Map=spin.map` — produce a **map file** showing where every psect ended up. We’ll live in this file in Chapter 17.

In your IDE

Create the project (Chapter 3): in VS Code run **MPLAB: Create New Project**, choose device **PIC16F17146** and the **XC8 (pic-as)** toolchain, add `spin.S`, and build with **Ctrl+Shift+B**. The reset-vector placement and list/map options are set through the project’s build settings (in MPLAB X IDE they go in Project Properties → pic-as Linker/Global Options, without the leading `-Wl,-Wa,;` Embedded Engineers guide §4.4).

Watch it do nothing (productively)

Start the **MPLAB Debugger** simulator (Chapter 3: Debug: Add Configuration → MPLAB Debugger, then **F5**), step once — you land on `resetVec`’s `goto main`. Step again — you’re on `main`’s `goto $`. Step a third time — the Program Counter doesn’t move. That frozen PC is your endless loop working. No hardware required.

4.7 What just happened

1. On Reset the PIC16F17146 fetched the instruction at **0x0000** (data sheet §9.1).
2. We had placed `goto main` there, so control jumped to `main`.
3. `main` ran `goto $`, jumping to itself forever.

Two instruction words of program memory, and you’ve exercised the reset vector, two psects, a label, the location counter, and the whole build pipeline.

Why “uppercase .s”? *The uppercase extension tells the driver to run the **C preprocessor** over your file first, which is what makes `#include` and `#define` work (User’s Guide §4.6.4). A lowercase `.s` skips preprocessing and your `#include <xc.inc>` would fail. When in doubt, use `.S`.*

4.8 Common mistakes

Symptom	Cause	Fix
<code>#include <xc.inc></code> not found / SFR names undefined	file saved as lowercase <code>.s</code> , so no preprocessing	rename to uppercase <code>.S</code>
Code assembles but chip never runs your program	forgot <code>-Wl,-presetVec=0h</code> , so <code>resetVec</code> isn’t at <code>0x0000</code>	add the linker option (or set psect origin in IDE)
Fixup/overflow or “won’t fit” on a custom program-space psect	<code>delta</code> not set to 2 on a baseline/mid-range psect	add <code>delta=2</code> to that custom psect; provided psects already carry their flags
A hex constant like <code>FFh</code> reported as an unknown symbol	hex needs a leading digit	write <code>0FFh</code> or <code>0xFF</code>
“duplicate symbol” between your name and a keyword	you named something <code>and</code> , <code>mod</code> , <code>goto</code> ...	rename the identifier

(Cross-reference: Appendix C decodes the exact wording of these assembler messages.)

4.9 Try it yourself

1. **Break it on purpose.** Change `goto $` to `goto here` with no `here`: label. Build. Read the exact error text, then fix it by adding the label. (Learning to read errors is half of assembly.)

2. **Prove the reset vector matters.** Remove `-Wl,-presetVec=0h` from the build command, rebuild, and inspect `spin.map` — note where `resetVec` landed and predict whether the chip would run your code. Put the option back.
 3. **Read the machine code.** Open `spin.lst` and find the opcode bytes generated for `goto main`. You don't need to decode them yet — just confirm that one source line became one instruction word.
-

4.10 Reference bridge

You're now ready to read these sections of the official **MPLAB XC8 PIC Assembler User's Guide** without getting lost:

- **§4.2 Statement Formats** — the five shapes, formally.
- **§4.4 Comments** and **§4.5 Constants** — the full comment and number rules.
- **§4.6 Identifiers** — including the location counter `$` and how labels work.

Next chapter: `spin.S` runs on the simulator's defaults, but a *real* PIC16F17146 needs its **Configuration bits** set — the oscillator, watchdog, and MCLR choices burned in alongside your code. Chapter 5 adds the `CONFIG` block and explains why the chip won't reliably start without it.

Chapter 5 — Configuration Bits & the Reset Vector

***What you'll build:** spin.S from Chapter 4, upgraded with a proper **Configuration Word block** so it will start reliably on real PIC16F17146 silicon — not just on the simulator's forgiving defaults. You'll choose an oscillator, switch the watchdog off, and settle the MCLR/programming pins on purpose instead of by accident.*

5.1 The idea: settings you burn in *before* your code runs

Your program isn't the first thing that decides how the chip behaves. Before a single instruction executes, the PIC16F17146 reads a handful of **Configuration Words** out of a special nonvolatile configuration region and uses them to wire itself up: which clock to run from, whether the watchdog timer can reset you, what the MCLR pin does, whether the memory is code-protected.

Get these wrong and the failure is baffling for a beginner — the chip resets in a loop, or runs at the wrong speed, or requires a different programming entry mode. Get them right once and you rarely think about them again.

The PIC16F17146 has **five Configuration Words**, physically located at program addresses **0x8007-0x800B**, and they “select the device oscillator, Reset and memory protection options” (verified: data sheet DS40002343F §8.1). You set them from source with the assembler's `CONFIG` directive; the assembler places the resulting bits at those addresses and the programmer burns them in alongside your code.

***The reset vector connection.** Chapter 4 relied on the fact that the chip begins executing at **0x0000** after a Reset (data sheet §9.1). Config bits decide what kind of resets can happen and what clock is running when execution reaches 0x0000. Reset behavior and the reset vector are two halves of the same story — that's why they share this chapter.*

5.2 Get the tokens from the selected DFP (then understand them)

Here is the professional workflow, and it matters:

The hardware meaning of each field comes from the data sheet. The accepted symbolic value tokens (ON, OFF, HFINTOSC_1MHz, ...) come from the exact Device Family Pack (DFP) selected for the build. Open that pack’s `xc8/docs/pic_chipinfo.html`, follow the PIC16F17146 link, and use the page’s **PIC-AS config Usage** and field tables. The block in §5.4 was assembled with XC8 4.00 and PIC16F1xxxx_DFP 1.31.465.

A **Configuration Bits** view (MPLAB X: *Window* → *Target Memory Views* → *Configuration Bits*; in VS Code the MPLAB Code Configurator / project config UI serves the same role) is useful for exploring the choices. Depending on the IDE/project, generated output can use C’s `#pragma config` syntax; do not paste that syntax into a `.s` file and assume it is `pic-as` syntax. For assembly-only source, copy the same setting/value decisions into quoted `pic-as CONFIG` directives using the selected DFP’s `chipinfo` page. In all cases, inspect the final map and HEX rather than treating a generated block as self-validating.

5.3 The five decisions that matter for a first program

Everything in the config block is one of five kinds of decision. For a simple blink we make the simplest safe choice for each; every meaning here is verified against DS40002343F §8.6.

Decision 1 — What clock runs the chip? (CONFIG1)

We want zero external parts, so we run from the **internal oscillator (HFINTOSC)** and use no external crystal.

Field	Our choice	Why (data sheet §8.6.1)
FEXTOSC	off / not enabled	value 100 = “Oscillator not enabled” — we use no external crystal

Field	Our choice	Why (data sheet §8.6.1)
RSTOSC	HFINTOSC	value 110 selects HFINTOSC at 1 MHz on power-up (value 000 would give 32 MHz)
CLKOUTEN	off	value 1 = CLKOUT disabled, so that pin stays a normal I/O
CSWEN	on	value 1 = software is allowed to switch clocks later if it wants
FCMEN	on	value 1 enables the monitor; it applies to external oscillator modes and is dormant while HFINTOSC is selected
VDDAR	high range for the board's default 3.3 V	value 1 = analog calibrated for VDD 2.3–5.5 V; value 0 = 1.8–3.6 V

The internal oscillator is the beginner's best friend: nothing to wire, no crystal to get wrong, and it's plenty accurate for blinking an LED.

VDDAR = HI matches the Curiosity Nano's factory-default 3.3 V target supply. If you change the board supply below 2.3 V, select VDDAR = LO and recheck every peripheral's operating limits.

Decision 2 — Can the watchdog reset me? (CONFIG3)

The **Windowed Watchdog Timer (WWDT)** resets the chip if your code doesn't periodically "pet" it. That's a great safety feature for shipping firmware and a constant nuisance while learning. Turn it **off** for now.

Field	Our choice	Why (§8.6.3)
WDTE	off	value 00 = "WDT disabled, the SEN bit is ignored"

(The other WDT fields — WDTCPs, WDTcWS, WDTCCS — set the period and window. They have no run-time effect with WDTE off, but §5.4 still programs their erased/default selections explicitly so the complete intent is visible.)

Decision 3 — What does the MCLR pin do, and how do I program the chip? (CONFIG2 + CONFIG4)

This is the pairing that traps the most beginners, so read carefully. Two fields interact:

- **LVP** (CONFIG4, §8.6.4) — Low-Voltage Programming. Value 1 = “Low-Voltage Programming is enabled. MCLR/VPP pin function is MCLR. **The MCLRRE bit is ignored.**”
- **MCLRRE** (CONFIG2, §8.6.2) — Master Clear Enable. Only has an effect *when* LVP = 0.

Read that carefully: **if LVP = 1, the MCLR pin is forced to be MCLR and your MCLRRE setting does nothing.** For a beginner using the Curiosity Nano’s on-board ICSP debugger, **keep LVP = 1** — low-voltage programming keeps MCLR working as a reset pin automatically.

Field	Our choice	Why
LVP	on	value 1 = low-voltage programming enabled; MCLR pin is MCLR; MCLRRE ignored (§8.6.4)
MCLRRE	on	value 1 = MCLR pin is MCLR — matters only if you ever set LVP off (§8.6.2)

***Programming-entry guardrail.** With LVP = 0, subsequent programming must enter with **high voltage** on MCLR (§8.6.4). A low-voltage-only debugger then cannot reconnect. The device is recoverable with suitable HV-capable hardware, but the Curiosity Nano guide warns that an external tool’s high voltage can damage board resistor R110. A low-voltage session cannot itself clear LVP to zero (§8.6.4 note 1), which prevents accidentally dropping out of LVP while that session is active.*

Decision 4 — Should the chip protect itself from brown-outs and bad startups? (CONFIG2)

Reasonable safety defaults that won’t get in your way:

Field	Our choice	Why (§8.6.2)
BOREN	on while running	value 10, DFP token NSLEEP, enables BOR while running and disables it in Sleep
BORV	low trip point	value 1 = VBOR nominally 1.9 V (value 0 is nominally 2.65 V; see electrical limits)
PWRTS	your call	Power-up Timer; e.g. value 10 = 64 ms startup delay for supply to settle
STVREN	on	value 1 = a stack overflow/underflow causes a Reset — catches runaway recursion
LPBOREN	off	value 1 = Low-Power BOR disabled (fine for a mains/USB-powered board)
PPS1WAY	on	value 1 = PPS can be locked once; safe default (we cover PPS in Chapter 7)
DACAUTOEN	off	value 1 leaves DAC buffer range under REFRNG software control; the DAC is unused here

Decision 5 — Lock the memory down? (CONFIG4 + CONFIG5)

While learning you want the memory **open** so you can reprogram and read back freely. Every one of these is the “not protected / disabled” choice:

Field	Our choice	Why (§8.6.4–8.6.5)
CP	off	value 1 = user program-flash code protection disabled (§8.6.5)
CPD	off	value 1 = Data EEPROM code protection disabled (§8.6.5)

Field	Our choice	Why (§8.6.4–8.6.5)
WRTAPP,WRTB,WRTC,WRTD,WRTSAF	off	value 1 = the corresponding block is <i>not</i> write-protected (§8.6.4)
BBEN	off	value 1 = Boot Block disabled — we don't need a bootloader region (§8.6.4)
SAFEN	off	value 1 = Storage Area Flash disabled (§8.6.4)
BBSIZE	512 words	explicit but inactive while BBEN is off

Note the polarity. For most protection bits, **1 means “unlocked/disabled protection.”** This is deliberate: an erased flash cell reads as 1, so a blank device powers up wide open, and you opt into protection by writing 0. Once you write a protection bit to 0, the data sheet warns it can typically only be undone with a **Bulk Erase** (§8.6.4 note 2, §8.6.5 note 2) — so don't enable protection casually while learning.

5.4 The upgraded program

Below is `spin.S` with a config block added. **These tokens are verified against PIC16F1xxxx_DFP 1.31.465 and assemble cleanly with XC8 4.00.** If you select another DFP, check its `pic_chipinfo.html` before building. Every implemented field is stated explicitly except `DEBUG`, which the development tools own.

```
; -----
; spin.S - minimal PIC16F17146 program, now board-ready
; -----

PROCESSOR 16F17146
#include <xc.inc>

; --- Configuration Words (0x8007-0x800B, data sheet §8.1) ---
; These tokens are verified against PIC16F1xxxx_DFP 1.31.465.
; If your selected DFP differs, check xc8/docs/pic_chipinfo.html.
; The *intent* of each line is explained in §5.3.
```

```

; CONFIG1 - clock: run from internal HFINTOSC, no external crystal
CONFIG "FEXTOSC = OFF"           // no external oscillator
CONFIG "RSTOSC = HFINTOSC_1MHz" // start on HFINTOSC, 1 MHz
CONFIG "CLKOUTEN = OFF"         // CLKOUT pin is normal I/O
CONFIG "CSWEN = ON"             // allow software clock switching
CONFIG "VDDAR = HI"             // board-default 3.3 V range
CONFIG "FCMEN = ON"             // fail-safe monitor enabled

; CONFIG2 - resets & MCLR
CONFIG "MCLRE = EXTMCCLR"        // MCLR pin is MCLR (ignored when LVP=ON)
CONFIG "PWRTS = PWRT_64"        // 64 ms power-up timer
CONFIG "LPBORN = OFF"           // low-power BOR off
CONFIG "BORN = NSLEEP"          // BOR on while running, off in Sleep
CONFIG "DACAUTOEN = OFF"        // REFRNG controls range; DAC unused
CONFIG "BORV = LO"              // VBOR = 1.9 V
CONFIG "ZCD = OFF"              // zero-cross detect off by default
CONFIG "PPS1WAY = ON"           // PPS one-way lock
CONFIG "STVREN = ON"            // stack over/underflow → reset

; CONFIG3 - watchdog OFF while learning; inactive fields explicit
CONFIG "WDTCPSS = WDTCPSS_31"   // erased/default period selection
CONFIG "WDTE = OFF"             // watchdog disabled
CONFIG "WDTCCS = WDTCCS_7"      // erased/default window selection
CONFIG "WDTCCS = SC"            // erased/default clock selection

; CONFIG4 - programming & write protection
CONFIG "BBSIZE = BB512"         // inactive because BBEN=OFF
CONFIG "LVP = ON"               // low-voltage programming (keep ON)
CONFIG "BBEN = OFF"             // no boot block
CONFIG "SAFEN = OFF"            // no Storage Area Flash
CONFIG "WRTAPP = OFF"           // application not write-protected
CONFIG "WRTB = OFF"             // boot block not write-protected
CONFIG "WRTC = OFF"             // config not write-protected
CONFIG "WRTD = OFF"             // data EEPROM not write-protected
CONFIG "WRTSAF = OFF"           // SAF not write-protected

; CONFIG5 - code protection OFF while learning
CONFIG "CP = OFF"               // program flash readable
CONFIG "CPD = OFF"              // data EEPROM readable

; --- Reset entry point (chip starts at 0x0000, §9.1) -----
PSECT resetVec,class=CODE,delta=2
resetVec:
    goto    main

; --- Main program -----
PSECT code
main:
    goto    $                ; endless loop

END        resetVec

```

5.5 Build & run it

Same command as Chapter 4 (with DFP set as in Chapter 3):

```
pic-as -mcpu=16f17146 -mdfp="$DFP" -Wl,-presetVec=0h \  
-Wa,-a -Wl,-Map=spin.map spin.S
```

The build now also programs the five Configuration Words. If you flash this to a real PIC16F17146 (Chapter 18 covers the mechanics), it comes up running from the internal 1 MHz oscillator, watchdog off, MCLR working, memory open for reprogramming — a clean, predictable starting state.

DEBUG bit — leave it alone. *You won't see a DEBUG line in generated code, and that's correct: the data sheet says the DEBUG bit "is managed automatically by device development tools" and must stay 1 for normal operation (§8.1, §8.6.2 note 1). MPLAB X flips it for you when you launch a debug session. Never set it by hand.*

Errata check. *Exact-device errata DS80001009E §1.3.1 reports unexpected wake-from-Sleep behavior only for B0 silicon. This example never executes SLEEP; check the revision ID and current errata before adding low-power behavior. The current B4 column does not mark that issue.*

5.6 What just happened

You told the chip, in permanent-until-reprogrammed flash, *how to be itself* before your code gets a vote: internal clock, no watchdog surprises, MCLR as reset, low-voltage programming so your tool can always reconnect, and no memory locks to trip over. That's the boring, essential groundwork every PIC program stands on.

5.7 Common mistakes

Symptom	Cause	Fix
A low-voltage debugger cannot enter programming after an image was written with another tool	that image set <code>LVP = OFF</code>	recovery requires suitable HV entry; consider board limits and keep <code>LVP = ON</code> (§8.6.4)
Chip resets periodically on its own	watchdog left enabled and never serviced	set <code>WDTE = OFF</code> (§8.6.3)
CONFIG token rejected: “unknown/invalid setting”	value token doesn’t match your DFP’s spelling	consult the selected DFP’s chipinfo page; MPLAB X can help identify the same setting/value pair
MCLR pin won’t reset the chip / behaves oddly	assumed <code>MCLRE</code> controls it, but <code>LVP = 1</code> overrides it	with <code>LVP = ON</code> , MCLR is MCLR automatically; <code>MCLRE</code> is ignored (§8.6.2, §8.6.4)
Can’t read program back after flashing	accidentally set <code>CP = 0</code> (protection on)	remember <code>1 = disabled</code> ; recover with a Bulk Erase (§8.6.5)

(Appendix C decodes the exact assembler message wording.)

5.8 Try it yourself

1. **Audit the block yourself.** Open your selected DFP’s `xc8/docs/pic_chipinfo.htm` follow the PIC16F17146 link, and compare every token with §5.4. If MPLAB X generates `#pragma config` output, compare the setting/value pairs but retain `pic-as CONFIG` syntax in the `.s` file.
2. **Predict a failure.** Without building, write down what you’d expect if you set `WDTE = ON` with a period of ~ 2 s and never petted the watchdog. Then try it on the simulator and confirm.
3. **Trace the polarity.** Look up `CP` in the data sheet (§8.6.5) and confirm from the bit table that a blank (all-ones) device is *not* code-protected. Explain in one sentence why erased-means-unlocked is a sensible design.

5.9 Reference bridge

- **Data sheet DS40002343F §8 “Device Configuration”** — the authoritative field-by-field reference for all five Configuration Words. You can now read every register table in it.
- **Embedded Engineers guide §3.2 / §4** — shows CONFIG blocks in context for other devices; compare their choices to ours.
- **User’s Guide** — search for the CONFIG directive for its exact syntax and the “NAME = VALUE” string form.
- **Selected DFP** xc8/docs/pic_chipinfo.html — accepted PIC16F17146 setting/value tokens and PIC-AS examples; the data sheet remains authoritative for hardware meaning.
- **Exact-device errata DS80001009E** — revision-specific exceptions, including B0 Sleep wake-up.

Next chapter: with the chip configured and starting cleanly, we finally make it *do* something. Chapter 6 introduces the working register *w*, literals, and file registers — how data actually moves inside a PIC — building toward the first blink in Chapter 7.

Chapter 6 — Moving Data: W, Literals, and File Registers

***What you'll build:** a tiny program that loads a number, stashes it in a memory location, clears its working copy, then reads the number back — and you'll watch each value move in the simulator's Watch window. No blinking yet; this is the chapter where “data” stops being abstract and becomes three registers you can see change.*

6.1 The idea: almost everything flows through one register

Look at the PIC16F17146's core data-path diagram in the data sheet (DS40002343F §7, Figure 7-1). Nearly every arrow into and out of the ALU passes through a single 8-bit register called **W — the Working register** (shown as “W Register” in that figure). W is the chip's scratch register. You load a value into W, the ALU operates using W, and an instruction can place its result in W or back in a file register.

This is the mental shift from high-level languages. In C you write $x = a + b$ and the compiler finds registers for you. In PIC assembly *you* are the compiler: to add two numbers you load one into W, then add the other to W. W is the crossing point that almost all data traffic must go through. Get comfortable with it and the rest of the instruction set falls into place.

***Fact check.** The enhanced mid-range CPU in this chip has **50 instructions** and W sits at the center of the data path alongside the STATUS register and ALU (data sheet §7, §7.4). A great many of those 50 instructions are just different ways of moving data into W, out of W, or combining something with W.*

6.2 Two kinds of “value”: literals and file registers

Most instructions that move or combine byte values use one of two kinds of operand:

- A **literal** — a constant number baked into the instruction itself. `movlw 0x2A` means “move the *literal* 0x2A into W.” The *l* in `movlw` stands for *literal*. The value lives inside the program, not in RAM.
- A **file register** — any byte of the chip’s **data memory**. “File register” is the classic PIC term for “an addressable byte of RAM.” `movwf myByte` means “move W into the *file register* named `myByte`.” The *f* stands for *file register*.

So `movlw` brings a fixed constant in from your code, and the file-register instructions shuffle data between W and RAM. That’s the whole game at this level.

6.3 What counts as a “file register”?

Here’s the part that surprises newcomers: on a PIC, **almost everything is a file register**. The data memory isn’t just your variables — the ports, the timers, the configuration of every peripheral, and even W itself all live in the same addressable data space. The data sheet lays out how each 128-byte **bank** of data memory is divided (§9.2):

Region	Size per bank	What lives there
Core registers	12 bytes (offsets 0x00–0x0B)	INDF0/1, PCL, STATUS, FSR0L/H, FSR1L/H, BSR, WREG , PCLATH, INTCON
Special Function Registers	up to 20 bytes	peripheral controls: PORTA, TRISC, LATA, timer registers...
General Purpose RAM (GPR)	up to 80 bytes (0x20–0x6F)	<i>your</i> variables
Common RAM	16 bytes (0x70–0x7F)	<i>your</i> variables, reachable from every bank

Two facts from that table matter enormously right now:

1. **W is mapped into the file-register address space.** The dedicated CPU working register appears as `WREG` at offset 0x09 of every bank (data sheet §9.2.2), so file-register instructions such as `clrf WREG` can access it. That does not make W ordinary SRAM: it is a CPU register with special instruction encodings and ALU behavior.
2. **Common RAM (0x70–0x7F) is reachable from every bank** (§9.2.5). This is the beginner’s escape hatch from the banking

headache we'll tackle in Chapter 8: put a variable there and you can touch it without any bank-selection ceremony.

Why banks exist at all (the one-paragraph version). A file-register instruction only has room to name a **7-bit** offset (0-127) — enough for one 128-byte bank. To reach more RAM, the chip keeps a **Bank Select Register (BSR)** that picks which bank those 7 bits refer to; the full data address is 6 bank bits + 7 offset bits (§9.2.1). Common RAM cheats: it's mirrored into all banks, so its offset means the same thing no matter what BSR holds. Chapter 8 is devoted to the banks; for now we simply sidestep them.

6.4 The three instructions you need today

All three appear verbatim in Microchip's own examples (data sheet §9.1.1; User's Guide §4.1), so these are the real mnemonics, not simplifications:

Instruction	Meaning	Example
<code>movlw k</code>	W ← literal k (an 8-bit constant)	<code>movlw 0x2A</code> → W becomes 0x2A
<code>movwf f</code>	file register f ← W	<code>movwf myByte</code> → myByte becomes whatever W holds
<code>movf f,w</code>	W ← file register f	<code>movf myByte,w</code> → W becomes myByte's value

The `,w / ,f` destination operand — don't leave it off

Many byte instructions can send their result to *either* W or back to the file register, and you choose with a destination operand (User's Guide §4.1.1):

- `,w` → result goes to **W**.
- `,f` → result goes back to the **file register**.

So `movf myByte,w` copies myByte *into* W, while `movf myByte,f` reads myByte and writes it back to *itself* (useful only for its side effect on the STATUS flags — a trick for later).

Trap — the silent default. If you omit the destination, the assembler assumes `,f` (User's Guide §4.1.1). That means a careless `movf`

myByte *does not* load W — it writes myByte back to itself and leaves W untouched. The guide “highly recommends” you always write the destination explicitly, and so do I. Nearly every beginner loses an hour to this once.

Trap — `movfw` *doesn’t exist here*. If you’ve seen older MPASM code use `movfw foo`, that pseudo-instruction is **not** implemented by the XC8 assembler. Write the standard `movf foo,w` instead (User’s Guide §4.1.4).

6.5 The whole program

Save as `move.s`. Add your Chapter 5 CONFIG block at the top for real hardware; it’s omitted here so the data movement stays in focus.

```
; -----
; move.S — watch a value travel: literal → RAM → W
; -----

PROCESSOR 16F17146
#include <xc.inc>
; (CONFIG block from Chapter 5 goes here for real silicon)

; --- one byte of storage in COMMON RAM (0x70-0x7F) -----
; Common RAM is reachable from every bank, so NO BANKSEL is
; needed to touch myByte. (Assembler psect: udata_shr, class
; COMMON — User's Guide §5.2; common RAM — data sheet §9.2.5.)
PSECT udata_shr
myByte:
    DS            1                ; reserve 1 byte

; --- reset entry point (chip starts at 0x0000, §9.1) -----
PSECT resetVec,class=CODE,delta=2
resetVec:
    goto    main

; --- the code -----
PSECT code
main:
    clrf    myByte                ; define RAM before observing it (reset value is
    ↪      undefined)
    movlw   0x2A                  ; (1) W ← literal 0x2A (decimal 42)
    movwf   myByte                ; (2) myByte ← W (myByte = 0x2A)
    movlw   0x00                  ; (3) W ← 0 (wipe W on purpose)
    movf    myByte,w              ; (4) W ← myByte (W = 0x2A again)
```

```
goto    $          ; (5) stop here forever  
  
END     resetVec
```

Notice there is **no BANKSEL** and **no BANKMASK** anywhere. That's the payoff of putting `myByte` in common RAM — its address (somewhere in 0x70–0x7F) has no bank bits to worry about, so every instruction just works. Enjoy it now; Chapter 8 explains what you're being spared.

6.6 Build & run it — and actually watch

Build exactly as before, with DFP set as in Chapter 3:

```
pic-as -mcpu=16f17146 -mdfp="$DFP" -Wl,-presetVec=0h \  
      -Wa,-a -Wl,-Map=move.map move.S
```

Now the important part — *observe the moves*:

1. Start the **MPLAB Debugger** simulator (Chapter 3: **F5** on the debug configuration).
2. In the **Run and Debug** view open **Variables**, or add a **Watch** on `myByte` and on `WREG`. (W shows up as `WREG` because, as §6.3 noted, the working register is mapped into the file-register address space under that name.)
3. **Step one instruction at a time** and watch. The initial `clrf` makes the otherwise-undefined RAM byte deterministic:
 - After line (1): `WREG = 0x2A`, `myByte` remains 0.
 - After line (2): `myByte = 0x2A`.
 - After line (3): `WREG = 0x00` — proof the copy in `myByte` is independent.
 - After line (4): `WREG = 0x2A` again — you just read RAM back into W.
 - Line (5): the PC freezes (your endless loop from Chapter 4).

Seeing those two values change on separate steps is the entire point of this chapter. Data movement is no longer a concept — it's two numbers you watched move.

6.7 What just happened

You exercised the fundamental cycle of all PIC assembly: **literal** → **W** → **file register** → **W**. Every larger program is this pattern repeated and combined. Loading a port value, saving a sensor reading, setting up a timer — all of it is `movlw/movwf/movf` with the occasional arithmetic instruction mixed in.

You also met W wearing both of its hats: the ALU’s dedicated working register *and* a CPU register mapped into the file space as `WREG`. And you got your first, painless taste of data memory by hiding in common RAM.

6.8 Common mistakes

Symptom	Cause	Fix
<code>movf myByte</code> doesn’t change W	destination defaulted to <code>,f</code> , writing <code>myByte</code> to itself	write <code>movf myByte,w</code> explicitly
<code>movfw</code> reported as unknown	that MPASM pseudo-instruction isn’t in XC8	use <code>movf foo,w</code> (User’s Guide §4.1.4)
Fixup/overflow once you move <code>myByte</code> out of common RAM	a GPR variable needs bank selection + masking	that’s Chapter 8; keep it in <code>udata_shr</code> for now
Loaded a constant but W is wrong	<code>movwf k</code> used where you meant <code>movlw k</code>	<code>movlw</code> = load literal; <code>movwf</code> = store W to a file register
<code>movlw 0FFh</code> fine but <code>movlw FFh</code> errors	hex constant must start with a digit	write <code>0FFh</code> or <code>0xFF</code> (Chapter 4 §4.5)

(Appendix C decodes the exact assembler message text.)

6.9 Try it yourself

1. **Add a second variable.** Reserve `myByte2` in the same `udata_shr` psect, and write code that copies `myByte` into `myByte2` through W. Step through and confirm both hold `0x2A`.

2. **Prove the default bites.** Change line (4) to `movf myByte (no ,w)`, rebuild, and step through. Watch `WREG` *not* update. Then fix it and watch the difference. Feel the trap so you never fall in it later.
 3. **Clear W two ways.** Replace `movlw 0x00` with `clrf WREG` and confirm that both make W zero. Then inspect `STATUS.Z`: `clrf` sets Z, while `movlw` does not affect STATUS. Equal data results do not imply equal side effects (data sheet §9.2.2 and §41).
-

6.10 Reference bridge

- **Data sheet §7 “Enhanced Mid-Range CPU”** — the core data path, W, STATUS, and the 50-instruction set at a glance.
- **Data sheet §9.2 “Data Memory Organization”** — banks, core registers, GPR, and common RAM, now that you’ve used them.
- **User’s Guide §4.1 “Assembly Instruction Deviations”** — the `,w`, `,f` operand styles and the `movfw` note, formally.
- **User’s Guide §5.2 “Assembler-provided Psects”** — where `udata_shr`, `udata`, and `code` come from.

Next chapter: you can now move bytes around at will. Chapter 7 points those bytes at the outside world — choosing the input-buffer mode with `ANSEL`, enabling the output driver with `TRIS`, and driving it with `LAT` to make an LED blink. That’s the first program that does something you can *see*.

Chapter 7 — Your First Blink

***What you'll build:** the “hello, world” of hardware — a physically blinking LED on the PIC16F17146 Curiosity Nano. You'll configure a real pin, drive it, and add a delay so your eye can see it. This is the chapter where the chip finally does something in the world.*

7.1 The idea: direction, latch, and input mode

Every I/O pin on this chip is controlled by a small stack of registers. Two registers actually control a simple digital output; a third controls the pin's input buffer. The data sheet lists all eight port-control registers in §16.1; these are the ones that matter today:

Register	Job	The rule (data sheet)
TRISx	direction	bit = 0 → output ; bit = 1 → input (§16.4)
ANSELx	input-buffer mode	bit = 1 → analog input (reset default); bit = 0 → digital input (§16.5)
LATx	output value	write here to drive the pin high/low (§16.3)

The x is the port letter — TRISC, ANSELC, LATC for Port C, and so on. Three things to burn into memory right now, because each one is a classic first-blink failure:

1. **TRIS is backwards from what you'd guess.** 0 means output. Mnemonic: TRIS bit = “is it an input? 1 = yes.” Clearing the bit lets the pin drive out (§16.4).
2. **Pins wake up with their input buffers in analog mode.** An analog-mode pin reads as zero through PORTx and cannot serve as a digital peripheral input. But the data sheet is explicit: ANSELx has **no effect on digital output functions**; with TRIS clear, the latch still drives the pin (§16.5). We clear ANSEL so RC1 is fully defined as digital GPIO and can be read later, not because the LED output requires it.
3. **Drive LAT, never PORT.** Writing the pin's value through PORTx invites a read-modify-write glitch; “as a general rule, output operations to

a port must use the LAT register” (§16.3). Writing LATx sets the same pin, safely.

What about PPS? *You may have heard the PIC16F17146 needs Peripheral Pin Select to use a pin. Not for this. Every pin’s output defaults to its own data latch after Reset (PPS output code 0x00 = LATxy, data sheet §16.12 and Table 18-2), so driving LATC reaches the pin with **no PPS setup at all**. PPS matters when you route a peripheral (like a PWM or UART) onto a pin — we’ll use it then. A software-driven LED needs none of it.*

7.2 Which pin? Meet LED0

On the PIC16F17146 Curiosity Nano, the yellow user LED (**LED0**) is wired to pin **RC1** (Curiosity Nano user guide DS50003388B §4.2.1). One more crucial detail from that same section:

“Driving the connected I/O line to GND can also activate the LED.”

That means LED0 is **active-low**: the LED lights when RC1 is driven **low**, and turns off when RC1 is **high**. So in our code, `bcf LATC,1` (drive low) = **LED on**, and `bsf LATC,1` (drive high) = **LED off**. Don’t let the inversion trip you up — if your first build seems “on when it should be off,” this polarity is why.

(No Curiosity Nano? Any LED from a GPIO through a ~330 Ω resistor to ground works; then the logic is normal — high = on. Adjust the two bit instructions accordingly.)

7.3 Configuring the pin, step by step

We set up RC1 in three moves. First select digital input mode, then preload the output latch **while the pin is still an input**, and only then enable the output driver. Preloading avoids a brief active-low LED pulse: after Reset the latch is zero, and clearing TRIS before writing LAT would momentarily drive RC1 low.

Each access uses `BANKSEL` to select the register’s bank and `BANKMASK()` to put only the register’s within-bank address into the instruction. Chapter 8 explains both; using the pair now keeps the default linker’s overflow checks strict.

```

BANKSEL ANSEL
bcf     BANKMASK(ANSEL),ANSEL_ANSC1_POSN ; RC1 digital input mode
BANKSEL LATC
bsf     BANKMASK(LATC),LATC_LATC1_POSN   ; preload LED OFF (high)
BANKSEL TRISC
bcf     BANKMASK(TRISC),TRISC_TRISC1_POSN ; enable RC1 output last

```

After these three writes RC1 is a digital output, driven high, LED dark — without an output glitch.

7.4 Making it blink: the delay

If we just toggled the pin in a tight loop it would switch millions of times a second — far too fast to see. We need to *waste time* between toggles. The classic tool is a **nested decrement-and-skip loop** built from DECFSZ (“decrement f, skip next instruction if it hit zero,” data sheet §41 Table 41-3):

```

delay:
    movlw    0xFF
    movwf    d1                ; outer counter = 255
d_outer:
    movlw    0xFF
    movwf    d2                ; inner counter = 255
d_inner:
    decfsz   d2,f              ; d2--, skip the goto when d2 reaches 0
    goto     d_inner           ; ...otherwise keep looping
    decfsz   d1,f              ; d1--, skip when d1 reaches 0
    goto     d_outer
    return

```

How long is that? One instruction cycle is **four oscillator cycles** (data sheet §41). Our Chapter 5 config selects FOSC = 1 MHz, so one instruction cycle is nominally **4 μs**. Counting the one-cycle decrements, two-cycle goto instructions, two-cycle taken skips, setup, and two-cycle return, the routine body takes 196,098 instruction cycles. The call adds two more: $196,100 \times 4 \mu\text{s} = \mathbf{0.7844 \text{ second}}$ per call. Including the few LED and loop instructions, the nominal full on/off period is about **1.569 seconds** (about 0.637 Hz). Real timing follows the internal oscillator’s tolerance; this is a busy-wait, not a precision timer.

d1 and d2 are two bytes we’ll keep in **common RAM** so the delay never has to think about banks (Chapter 6’s trick).

7.5 The whole program

Save as `blink.S`. This listing includes the complete Chapter 5 configuration so the hardware clock and delay agree; if your selected DFP differs, re-audit the tokens as described there.

```
; -----
; blink.S  -  blink LED0 (RC1) on the PIC16F17146 Curiosity Nano
; LED0 is active-low: drive RC1 low = ON, high = OFF.
; -----

        PROCESSOR 16F17146
#include <xc.inc>

; --- Configuration Words: XC8 4.00 / PIC16F1xxx_DFP 1.31.465 ---
CONFIG "FEXTOSC = OFF"
CONFIG "RSTOSC = HFINTOSC_1MHz"
CONFIG "CLKOUTEN = OFF"
CONFIG "CSWEN = ON"
CONFIG "VDDAR = HI"
CONFIG "FCMEN = ON"

CONFIG "MCLRE = EXTMCLR"
CONFIG "PWRTS = PWRT_64"
CONFIG "LPBOREN = OFF"
CONFIG "BOREN = NSLEEP"
CONFIG "DACAUTOEN = OFF"
CONFIG "BORV = LO"
CONFIG "ZCD = OFF"
CONFIG "PPS1WAY = ON"
CONFIG "STVREN = ON"

CONFIG "WDTCPSS = WDTCPSS_31"
CONFIG "WDTE = OFF"
CONFIG "WDTCS = WDTCS_7"
CONFIG "WDTCCS = SC"

CONFIG "BBSIZE = BB512"
CONFIG "LVP = ON"
CONFIG "BBEN = OFF"
CONFIG "SAFEN = OFF"
CONFIG "WRTAPP = OFF"
CONFIG "WRTB = OFF"
CONFIG "WRTC = OFF"
CONFIG "WRTD = OFF"
CONFIG "WRTSAF = OFF"
```

```

CONFIG "CP = OFF"
CONFIG "CPD = OFF"

; --- two delay counters in COMMON RAM (no banking needed) ----
PSECT udata_shr
d1: DS 1
d2: DS 1

; --- reset entry point (chip starts at 0x0000, §9.1) -----
PSECT resetVec,class=CODE,delta=2
resetVec:
    goto    main

; --- main -----
PSECT code
main:
    ; --- prepare RC1, preload OFF, then enable its output driver ---
    BANKSEL ANSEL
    bcf     BANKMASK(ANSEL),ANSEL_ANSC1_POSN
    BANKSEL LATC
    bsf     BANKMASK(LATC),LATC_LATC1_POSN
    BANKSEL TRISC
    bcf     BANKMASK(TRISC),TRISC_TRISC1_POSN

    ; --- blink forever ---
loop:
    bcf     BANKMASK(LATC),LATC_LATC1_POSN ; drive RC1 low → LED ON
    call    delay
    bsf     BANKMASK(LATC),LATC_LATC1_POSN ; drive RC1 high → LED OFF
    call    delay
    goto    loop

; --- nominal 0.7844 s busy-wait delay at FOSC = 1 MHz -----
delay:
    movlw   0xFF
    movwf   d1
d_outer:
    movlw   0xFF
    movwf   d2
d_inner:
    decfsz  d2,f
    goto    d_inner
    decfsz  d1,f
    goto    d_outer
    return

END        resetVec

```

TRISC and LATC are both in bank 0, so the final BANKSEL TRISC leaves the correct bank selected for LATC. The delay counters are in common RAM and

delay never changes BSR, so that bank-state assumption holds across both calls. If a future callee or edit can change BSR, reselect LATC before accessing it; bank state is part of a routine's contract, not a global guarantee.

7.6 Build & run it

Set DFP as shown in Chapter 3, then build with the default strict operand-overflow policy. The source's `BANKMASK()` operands make the intended truncation explicit, so no fixup-warning or fixup-ignore option is needed:

```
pic-as -mcpu=16f17146 -mdfp="$DFP" -Wl,-presetVec=0h \  
      -Wa,-a -Wl,-Map=blink.map blink.S
```

On the Curiosity Nano: create the project for **PIC16F17146** (Chapter 3), add `blink.S` with the same DFP and the `-presetVec=0h` linker option, build (**Ctrl+Shift+B**), and program the board over USB via the MPLAB Debugger (Chapter 18). The yellow LED0 should complete one on/off cycle about every 1.57 seconds.

On the simulator (no board): run to the loop, put a Watch on LATC, and step — you'll see bit 1 of LATC flip between the `bcf` and `bsf`. (You can't see an LED in the simulator, but you can watch the latch that would drive it.)

7.7 What just happened

You reached out of the CPU and touched the physical world. The safe output sequence was: choose the input-buffer mode with **ANSEL**, preload the inactive level in **LAT**, then enable the output driver with **TRIS**. The loop changed LAT only after the driver was active, with a delay between changes so they were visible.

You also used four instruction *categories* for real: literal (`movlw`), byte (`movwf`), bit (`bcf/bsf`), and control (`call/goto/return`) — the backbone of the whole 50-instruction set.

7.8 Common mistakes

Symptom	Cause	Fix
LED stays off	TRISC bit left set (pin is an input)	clear the masked TRISC1 bit to enable the output driver (§16.4)
LED on when you expect off (and vice-versa)	LED0 is active-low	clear the masked LATC1 bit for ON; set it for OFF (CNANO §4.2.1)
A later read of RC1 is always zero even while the output is high	ANSELC1 was left in analog input mode	clear ANSELC1; ANSEL affects the input buffer, not the output driver (§16.5)
Output behaves unpredictably after other port operations	code used read-modify-write on PORTC	write or modify LATC instead (§16.3)
Build error: fixup overflow on ANSELC	high-bank operand was not masked	keep BANKSEL ANSELC and use BANKMASK(ANSELC) as the instruction operand
Loop stops toggling after one pass	an errant BANKSEL inside the loop moved the bank off bank 0	keep the loop touching only LATC; select its bank once before the loop

(Appendix C decodes the exact assembler/linker message text.)

7.9 Try it yourself

1. **Change the rate.** In `delay`, change the outer-counter load from `0xFF` to `0x80`. Predict, then observe, that both on-time and off-time become roughly half as long.
2. **Make a heartbeat.** Copy `delay` to `delay_short`, use `0x40` for its outer count, and call the short version after LED ON while retaining the long version after LED OFF.
3. **Blink a second pin.** Pick another free Port C pin routed to the edge connector, wire an external LED + resistor, configure it exactly like RC1, and blink both — one on while the other is off. (Watch the polarity: your external LED to ground is active-*high*.)
4. **Prove the ANSEL distinction.** Comment out the masked `bcf ANSELC, ...` line. The physical LED still blinks because ANSEL does not affect output, but a simulator Watch/read of PORTC bit 1 reports zero. Restore the line before using RC1 as a digital input.

5. **Read the switch (stretch).** SW0 is on RC0 (CNANO §4.2.2) with no external pull-up. Keep `TRISC0` set for input, clear `ANSELC0`, enable `WPUC0` (§16.6), and read `PORTC` bit 0. We'll do inputs properly later; try "LED on only while the button is pressed."
-

7.10 Reference bridge

- **Data sheet §16 “I/O Ports”** — every port register (`TRIS`, `LAT`, `ANSEL`, `WPU`, `INLV`, `SLRCON`, `ODCON`), including the output and input-buffer distinctions used here.
- **Data sheet §41 “Instruction Set Summary”** — the full 50 instructions; you've now used `movlw`, `movwf`, `bcf`, `bsf`, `decfsz`, `call`, `return`, `goto`.
- **Curiosity Nano guide §4.2** — the board's LED, switch, and crystal connections.

Next chapter (and Part III): you've been typing `BANKSEL` and `BANKMASK()` as a verified pair. Chapter 8 finally opens up **data memory banking** — what both forms do, why the chip needs banks, and how to place and reach your own variables anywhere in RAM.

PART III

Memory: The Hard Part, Made Gentle

Chapter 8 — Data Memory & Banking

***What you'll build:** the same blinking LED as Chapter 7 — but this time with the training wheels off. You'll replace the `--fixupoverflow` linker crutch with the real `BANKSEL` and `BANKMASK` you write yourself, and you'll finally understand what every one of those lines has been doing. This is the chapter that turns “cargo-cult `BANKSEL`” into knowledge.*

8.1 The idea: an instruction can only point 128 bytes

Here is the whole reason banking exists, in one fact. A file-register instruction on this chip — `movwf`, `clrf`, `bcf`, and friends — reserves only a **7-bit field** for the address of the byte it touches. Seven bits count from 0 to 127. So a single instruction can only name one location inside a **128-byte window**.

But the PIC16F17146 has far more than 128 bytes of data memory. The data sheet spells out the solution (§9.2): the data memory is split into **up to 64 banks of 128 bytes each**, and a separate register — the **Bank Select Register (BSR)** — chooses *which* bank those 7 address bits refer to. Put formally (§9.2.1):

“Data memory uses a 13-bit address. The upper six bits of the address define the Bank Address and the lower seven bits select the registers/RAM in that bank.”

So a full data address is **6 bank bits + 7 offset bits = 13 bits**. The instruction supplies the 7 offset bits; **BSR supplies the 6 bank bits**. That division of labor is the entire banking system. Everything else is bookkeeping to keep BSR pointing where you think it is.

full 13-bit data address: bbbbbb ooooooo
 └──┬──┘ └──┬──┘
 BSR (bank) instruction (offset 0-127)

8.2 What’s in a bank

The architecture gives each of the 64 banks the same 128-byte *address layout*, although a particular bank can implement fewer SFRs or GPR bytes (data sheet §9.2, Figure 9-2):

Offset in bank	Region	Notes
0x00–0x0B	12 core registers	WREG, STATUS, BSR, FSR0/1, INDF0/1, PCL, PCLATH, INTCON — mirrored in every bank
0x0C–0x1F	Special Function Registers	up to 20 peripheral-control bytes (varies by bank)
0x20–0x6F	SFR or General Purpose RAM	up to 80 bytes of <i>your</i> variables; high SFR banks use this range too
0x70–0x7F	Common RAM	16 bytes, mirrored into every bank

Two features fall out of this layout that you’ve already been leaning on:

- **Core registers and common RAM appear in every bank.** That’s why WREG and your `udata_shr` variables (Chapter 6) never needed a `BANKSEL` — no matter what BSR holds, offsets 0x00–0x0B and 0x70–0x7F mean the same thing. Common RAM is the banking escape hatch precisely because it’s un-banked.
- **Everything else moves.** `PORTC`, `TRISC`, `ANSEL`, and your GPR variables live in specific banks, and you must set BSR before touching them.

***Reality check on “64 banks.”** The address space allows 64 banks, but this chip doesn’t implement GPR in all of them — high banks are used heavily for SFRs, and unimplemented locations read as 0 (data sheet §9, Figures 9-6 through 9-10). The banks that matter hold the SFRs (spread across banks 0–61) and the GPR/common RAM your program actually uses.*

8.3 BANKSEL: setting the bank

To point BSR at the right bank, you *could* compute the bank number and load it yourself. On this enhanced mid-range core there’s a dedicated

instruction for it — **MOVLB k** (“Move literal to BSR,” data sheet §41). But you rarely write it by hand. Instead you use the assembler’s **BANKSEL** directive and name the object you’re about to access:

```
BANKSEL TRISC           ; assembler figures out TRISC's bank, emits movlb
↪ <that bank>
   bcf      TRISC,1
```

BANKSEL TRISC asks the linker “which bank is TRISC in?” and generates the instruction to load BSR with that number (User’s Guide §4.1.2 — on enhanced mid-range and PIC18 devices it expands to a single **movlb**). You get portability for free: move to a chip where TRISC lives elsewhere and **BANKSEL** still emits the right thing.

Trap — never put **BANKSEL right after a skip.** On some cores **BANKSEL** can expand to more than one instruction, so it must not immediately follow a skip instruction like **btfscc/btfs** (User’s Guide §4.1.2). On enhanced mid-range it’s a single **movlb**, but forming the habit now saves you on other PICs.

8.4 BANKMASK: making the address fit

Setting the bank is only half the job. Consider **ANSELC**. Its full address on this chip is **0x1EA0** (data sheet §9, Figure 9-12). Split that into the 13-bit form:

```
0x1EA0 = 11 1101 010 0000 (binary)
          └──┬──┘ └──┬──┘
          bank 0x3D  offset 0x20
          (= 61)
```

So **ANSELC** is **offset 0x20 in bank 61**. Now here’s the problem: an instruction’s address field is only 7 bits, but the symbol **ANSELC** carries the *full* 13-bit value **0x1EA0**. If you wrote **bcf ANSELC,1**, the assembler would try to stuff **0x1EA0** into a 7-bit slot — far too big — and the linker would raise a **fixup overflow error**.

BANKMASK() fixes this by ANDing away the upper (bank) bits, leaving just the 7-bit offset the instruction can hold (User’s Guide §4.1.3):

```
BANKSEL ANSEL    ; BSR <- 61 (the bank bits)
bcf    BANKMASK(ANSEL),1 ; use offset 0x20 (the low 7 bits) - fits!
```

This is exactly the two-part dance banking always requires: 1. **BANKSEL** puts the *bank* into BSR. 2. **BANKMASK** trims the *operand* down to the 7-bit offset.

They are independent and you almost always need both. **BANKSEL** alone without masking → fixup overflow. **BANKMASK** alone without **BANKSEL** → the instruction reaches offset 0x20 of *whatever bank BSR happened to hold* — a silent, nasty bug.

***The Chapter 7 mystery, solved.** Back in the blink program we added `-Wl,--fixupoverflow=warn` and skipped **BANKMASK**. That option told the linker “truncate every over-large operand to fit automatically” — i.e., do the masking for me (User’s Guide §4.1.3). It’s convenient, but it masks **every** overflowing operand globally, which can hide an unrelated addressing mistake. Writing **BANKMASK** only at the operands that intentionally need truncation leaves the linker’s default overflow checks active everywhere else. From here on we do it by hand.*

8.5 Placing your own variables in a bank

For your variables, you pick where they live with the psect you reserve them in (the assembler-provided psepts from Chapter 6, User’s Guide §5.2):

Psect	Where the variable lands	Needs BANKSEL ?
udata_shr	common RAM (0x70–0x7F)	no — mirrored in all banks
udata	any free GPR bank (linker chooses)	yes
udata_bank0 ... udata_bankn	GPR in <i>that specific</i> bank	yes (that bank)

Most of the time `udata` is right: you don’t care which bank a variable is in, so let the linker place it and let **BANKSEL** name it. Reach for `udata_bankn` only when a variable *must* be in a particular bank (rare for beginners).

8.6 The whole program — blink, banked by hand

This is Chapter 7’s blink with the crutch removed. Watch how ANSEL (bank 61) needs BANKMASK, while TRISC/LATC (bank 0, offsets below 0x80) don’t — their bank bits are already zero, so nothing overflows. We also add a real GPR variable, blinks, to make the banking concrete.

```
; -----
; blink8.S — Chapter 7 blink, with banking done manually
; -----

PROCESSOR 16F17146
#include <xc.inc>
; --- paste your Chapter 5 CONFIG block here ---

; a counter in general-purpose RAM (linker picks a bank)
PSECT udata
blinks: DS 1

; delay counters in common RAM (no banking needed)
PSECT udata_shr
d1: DS 1
d2: DS 1

PSECT resetVec,class=CODE,delta=2
resetVec:
    goto    main

PSECT code
main:
    ; count starts at zero
    BANKSEL blinks
    clrf    BANKMASK(blinks)      ; blinks lives in a GPR bank → mask it

    ; configure RC1 as a digital output
    BANKSEL ANSEL                  ; movlb 61 (ANSEL is 0x1EA0)
    bcf     BANKMASK(ANSEL),1      ; offset 0x20, RC1 → digital
    BANKSEL TRISC                  ; movlb 0
    bcf     TRISC,1                ; bank 0 offset 0x14 - no mask needed
    BANKSEL LATC                  ; movlb 0
    bsf     LATC,1                ; LED off (active-low); BSR now = bank 0

loop:
    bcf     LATC,1                ; LED ON (BSR still bank 0 - no BANKSEL)
    call    delay
    bsf     LATC,1                ; LED OFF
    call    delay

    ; tally the blinks (demonstrates banked GPR access)
    BANKSEL blinks
```

```
    incf    BANKMASK(blinks),f      ; blinks++
    BANKSEL LATC                    ; re-select bank 0 for the next loop pass
    goto    loop

delay:
    movlw   0xFF
    movwf   d1
d_outer:
    movlw   0xFF
    movwf   d2
d_inner:
    decfsz  d2,f
    goto    d_inner
    decfsz  d1,f
    goto    d_outer
    return

    END      resetVec
```

Two things to notice, both central to banking discipline:

- **After** `BANKSEL blinks` **we must** `BANKSEL LATC` **again** before the loop repeats, because reading `blinks` moved BSR to `blinks`'s bank. If we forgot, the next `bcf LATC,1` would hit offset 0x1A of the *wrong* bank. Tracking “where is BSR right now?” is the core skill of banked programming.
- **TRISC and LATC skip** `BANKMASK` only because they sit in bank 0, where the bank bits are already 0. It's never *wrong* to wrap them in `BANKMASK` too — many programmers do, for consistency and safety.

8.7 Build & run it — no crutch this time

```
pic-as -mcpu=16f17146 -mdfp=/path/to/PIC16F1xxx_DFP/VERSION/xc8 \
    -Wl,-presetVec=0h -Wa,-a -Wl,-Map=blink8.map blink8.S
```

Notice `--fixupoverflow` is **gone** — because we masked every over-large operand ourselves, the linker has nothing to complain about. The LED blinks exactly as before, but now *you* control the banking. Open the `.lst` file and find your `BANKSEL` lines: you'll see each one became a `movlb` with the bank number the linker computed (e.g. `movlb 0x3D` for `ANSEL`).

8.8 What just happened

You learned the two-part contract behind a direct access to banked data: **BANKSEL loads the bank into BSR; BANKMASK trims the operand to its 7-bit offset.** You saw *why* — the 128-byte reach of a file-register instruction — and you saw the escape hatches (core registers and common RAM) that let you dodge it when you can. Most importantly, you now know what all those BANKSEL lines from Chapters 6 and 7 were really doing, and the `--fixupoverflow` option is a choice you make, not magic you depend on.

8.9 Common mistakes

Symptom	Cause	Fix
Linker: “fixup overflow” on an SFR access	BANKSEL present but operand not masked	wrap the operand in BANKMASK() (§4.1.3)
Wrong RAM byte read/written, no error	BANKMASK used but BANKSEL forgotten	always pair them; select the bank first
Code worked, then broke after adding a RAM access mid-loop	that access moved BSR; later instructions assumed the old bank	re-BANKSEL the register you return to
BANKSEL/BANKMASK reported undefined	forgot #include <xc.inc>	the macros come from the header (§4.1.3)
Variable in udata needs banking but you expected none	only udata_shr (common RAM) is bank-free	use udata_shr for hot variables, or BANKSEL the udata one

(Appendix C decodes the exact message wording.)

8.10 Try it yourself

1. **Read the bank numbers.** Build `blink8.S`, open the `.lst` file, and confirm `BANKSEL ANSEL0` became `movlb 0x3D (61)` and `BANKSEL LATC` became `movlb 0`. Match them to the addresses in data-sheet §9.
2. **Cause the bug on purpose.** Delete the `BANKSEL LATC` that re-selects bank 0 before `goto loop`, rebuild, and run on the simulator. Watch the LED behavior break, and use the Watch window to see which bank BSR is stuck in.

3. **Move a variable into a chosen bank.** Change `blinks` from `PSECT udata` to `PSECT udata_bank0`, rebuild, and confirm from the map file it landed in bank 0. Then try `udata_shr` and confirm `BANKSEL` `blinks` is no longer needed.
 4. **Hand-compute an offset.** Take `LATA` (find its address in §9, Figure 9-3) and work out its bank and 7-bit offset by hand, the way we did for `ANSELC`.
-

8.11 Reference bridge

- **Data sheet §9.2 “Data Memory Organization”** — banks, BSR, the 13-bit address split, and the full per-bank memory maps you can now read.
- **User’s Guide §4.1.2-4.1.3** — `BANKSEL`, `PAGESEL`, `BANKMASK`, `PAGEMASK`, and the `--fixupoverflow` option, formally.
- **Data sheet §41** — `MOVLB` and the file-register instructions whose 7-bit address field started this whole story.

Next chapter: data memory has banks; *program* memory has the same problem one level up. When your code grows past one **page**, `goto` and `call` can’t reach across the boundary on their own. Chapter 9 introduces **paging** and `PAGESEL` — the program-memory twin of everything you just learned.

Chapter 9 — Program Memory & Paging

***What you'll build:** a program that deliberately puts a subroutine in a different page of program memory and calls it correctly across the boundary. You'll see the exact failure that happens when you ignore paging, and the two clean ways to fix it — PAGESEL and the fcall pseudo-instruction. Paging is banking's twin, one level up: same problem (an instruction can't reach far enough), same shape of solution.*

9.1 The idea: goto and call can't reach the whole program

In Chapter 8 you learned a file-register instruction can only address 128 bytes, so RAM is split into banks. Program memory has the identical problem. Look at the opcodes for goto and call (data sheet §41, Table 41-3):

```
GOTO k    -> 10 1kkk kkkk kkkk    (11 address bits)
CALL k     -> 10 0kkk kkkk kkkk    (11 address bits)
```

Each carries an **11-bit** destination. Eleven bits reach $2^{11} = \mathbf{2048}$ **words**. But the PIC16F17146 has **16,384 words** of program memory (addresses 0x0000–0x3FFF; data sheet §9.1). So a single goto or call can only reach one **2048-word page** — one eighth of the chip's code space. Sixteen K words ÷ 2K per page = **8 pages**.

Where do the *missing* upper address bits come from? From **PCLATH**. When a call executes, the data sheet is explicit (§9.4.3):

“PCH[2:0] and PCL registers are loaded with the operand of the CALL instruction. PCH[6:3] is loaded with PCLATH[6:3].”

Decode that: the 11 bits from the instruction fill PC[10:0], and the top of the address — **PC[14:11] — comes from PCLATH**. PCLATH is the *page register* for goto/call, exactly as BSR is the bank register for RAM. If PCLATH points at the wrong page when you call, you land in the wrong page. That's the whole story.

15-bit program address: pppp kkkkkkkkkkkk

PCLATH[6:3] goto/call operand (11 bits, 0-2047)

9.2 Why you often don't notice paging

Here's the good news for beginners: **a program that fits inside one page never needs paging**. Every blink and demo so far has been a few dozen words — comfortably inside page 0 — so every `goto/call` reached its target with `PCLATH` already correct (it's 0 after Reset). You've been paging-safe by accident.

You only have to think about pages when either: - your program grows past **2048 words** and code spills into page 1 or beyond, or - you deliberately place a routine in a specific high page (which we're about to do).

That's why paging feels invisible until a project gets big — and then, for someone who never learned it, becomes a baffling “my working subroutine suddenly jumps to garbage” bug. Let's make it happen on purpose so it never surprises you.

9.3 PAGESEL and PAGEMASK: the manual fix

The tools mirror Chapter 8's banking pair exactly (User's Guide §4.1.2–4.1.3):

- **PAGESEL label** sets the page bits for you. On this enhanced mid-range core it expands to a single **MOVLP k** (“Move literal to PCLATH,” data sheet §41) that loads `PCLATH` with `label`'s page. It's the program-memory twin of `BANKSEL`.
- **PAGEMASK(label)** strips the page bits off the address so the remaining 11 bits fit the `goto/call` operand field — the twin of `BANKMASK`.

So a correct cross-page call, done by hand, is:

```
PAGESEL myFarRoutine          ; PCLATH <- page of myFarRoutine (movlp)
call    PAGEMASK(myFarRoutine) ; 11-bit operand fits the opcode
PAGESEL $                      ; restore PCLATH to THIS page for what follows
```

That last line matters and catches people: after the call returns, PCLATH still points at the callee’s page. Any *later* goto/call in the current routine would use that stale page. Use the location counter \$ — “this page” — to put PCLATH back (User’s Guide §4.1.2 shows exactly this PAGESEL \$ idiom).

Trap — not right after a skip. Like BANKSEL, PAGESEL can expand to more than one instruction on some cores, so never place it immediately after a btfsc/btfss (User’s Guide §4.1.2).

9.4 fcall and ljmp: let the assembler do it

Tracking PCLATH by hand is exactly the kind of bookkeeping that invites bugs. The XC8 assembler offers two pseudo-instructions that handle page selection **and** masking for you (User’s Guide §4.1.7):

Pseudo-instruction	Expands to	Does for you
fcall label	page selection + call + caller-page reselection as needed	page-safe subroutine call and return
ljmp label	page selection + goto as needed	page-safe jump

With these you simply write:

```
fcall    myFarRoutine    ; correct whether the target is near or far
ljmp     someLabel       ; ditto for a jump
```

and the assembler emits a page-safe expansion. For an fcall, that expansion can include page selection before the call and reselection of the caller’s page after it returns. Do not depend on a same-page fcall collapsing to a plain call: XC8 4.00 emits movlp + call + movlp for the PIC16F17146 even when caller and callee ultimately share a page. The guide recommends these mnemonics “where possible” because they make source independent of where routines finally land (User’s Guide §4.1.7). The operand is the *full* address (do **not** mask it — the pseudo-instruction needs the whole address to select and mask the destination).

Two cautions from the guide (§4.1.7). fcall/ljmp can expand to several instructions, so never put one right after a skip. And they assume the containing psect is smaller than a page — which the default linker enforces anyway (it won’t let a code psect exceed one page).

9.5 The whole program — a genuine cross-page call

To demonstrate paging in a *small* program, we force a subroutine into **page 1** by giving it its own psect and telling the linker to position that psect at address **0x0800** (the first address of page 1). Main lives at the reset vector in page 0 and calls across the gap.

```
; -----
; paging.S - call a subroutine that lives in a different page
; -----

PROCESSOR 16F17146
#include <xc.inc>
; --- paste your Chapter 5 CONFIG block here ---

PSECT udata_shr
flag: DS 1

; --- reset entry (page 0) -----
PSECT resetVec,class=CODE,delta=2
resetVec:
    goto    main

; --- main code, page 0 -----
PSECT code,delta=2
main:
    ; Correct cross-page call, the easy way:
    fcall   farRoutine          ; assembler adds PCLATH setup because
                                ; farRoutine is in page 1

    ; Correct cross-page call, the manual way (equivalent):
    PAGESEL farRoutine          ; movlp <page 1>
    call    PAGEMASK(farRoutine)
    PAGESEL $                    ; restore PCLATH to page 0

    goto    $                    ; done - spin

; --- a routine deliberately placed in PAGE 1 -----
; Linked to 0x0800 via: -Wl,-pfarCode=0800h
PSECT farCode,class=CODE,delta=2
farRoutine:
    BANKSEL flag
    bsf     flag,0              ; prove we got here: set a flag bit
    return

END    resetVec
```


Build it, positioning `farCode` at the start of page 1:

```
pic-as -mcpu=16f17146 -mdfp=/path/to/PIC16F1xxxx_DFP/VERSION/xc8 \
      -Wl,-presetVec=0h -Wl,-pfarCode=0800h \
      -Wa,-a -Wl,-Map=paging.map paging.S
```

The `-Wl,-pfarCode=0800h` option pins the `farCode` psect at 0x0800, guaranteeing `farRoutine` is in page 1 while `main` runs from page 0. In the simulator, single-step the `fcall` and watch: you'll see the assembler-inserted `movlp 0x08` load `PCLATH` before the `call`, the PC jump into 0x08xx, flag bit 0 get set, and a second `movlp` reselect page 0 after return. The literal is **0x08**, not the human page number 1: `CALL` takes its upper destination bits from `PCLATH[6:3]`, so those bits must encode page 1.

9.6 See the failure (do this once)

Replace the whole `fcall/PAGESEL` sequence with a naive:

```
call    farRoutine          ; WRONG: no page setup, unmasked 15-bit address
```

Rebuild. The linker raises a **fixup overflow** — `farRoutine`'s address ($\geq 0x0800$) doesn't fit the 11-bit `call` field, and you never told `PCLATH` which page to use. That error is the linker protecting you. Seeing it once, and knowing it means "you crossed a page without saying so," is worth more than any amount of theory.

9.7 What just happened

Program memory is paged the same way data memory is banked: an instruction reaches only part of the space (2048 words), and a page register (`PCLATH`) supplies the rest. You fixed a real cross-page call two ways — the explicit `PAGESEL + PAGEMASK + PAGESEL $` sequence, and the one-line `fcall` that hides all of it. And you saw the fixup-overflow error that flags a forgotten page.

The symmetry is worth memorizing:

	Data memory	Program memory
Reach of one instruction	128 bytes (bank)	2048 words (page)
Page/bank register	BSR	PCLATH
“Set it” directive	BANKSEL ($\rightarrow$ movlb)	PAGESEL ($\rightarrow$ movlp)
“Trim the operand” macro	BANKMASK	PAGEMASK
Auto-everything helper	—	fcall / ljmp

9.8 Common mistakes

Symptom	Cause	Fix
Linker “fixup overflow” on a call/goto	target in another page, address unmasked	use fcall/ljmp, or PAGESEL + PAGEMASK (§4.1.7)
Call reaches the wrong place, no error	PAGEMASK used but PCLATH never set	pair it with PAGESEL (or just use fcall)
A later jump misbehaves after a cross-page call	PCLATH left pointing at the callee’s page	PAGESEL \$ to restore the current page (§4.1.2)
fcall/ljmp target won’t link	you masked the operand	pass the full address; the pseudo-instruction masks it (§4.1.7)
Odd behavior when PAGESEL follows a btfscl	it can expand to multiple instructions	never place PAGESEL/fcall right after a skip (§4.1.2)

(Appendix C decodes the exact message wording.)

9.9 Try it yourself

1. **Read the inserted movlp.** Build paging.S, open the .lst, and find where fcall expanded to page selection + call + caller-page reselection. Confirm the first movlp literal is 0x08 (selecting page 1) and the post-call instruction reselects page 0.
2. **Trigger and read the error.** Do the §9.6 experiment: swap in a plain call farRoutine, build, and read the linker’s fixup-overflow message word for word. Then restore the fix.

3. **Move the routine home.** Delete the `-wl,-pfarCode=0800h` option so `farCode` can link into page 0 alongside `main`. Rebuild and inspect the `.lst`: with XC8 4.00 the `fcall` remains a page-safe multi-instruction expansion even though both page literals now select page 0. This is why you inspect generated code rather than assuming pseudo-instructions are optimized away.
 4. **Restore-page bug.** In the manual sequence, delete `PAGESEL $`, add another `goto` to a page-0 label afterward, and see whether it still lands correctly. Explain what `PCLATH` held.
-

9.10 Reference bridge

- **Data sheet §9.4 “PCL and PCLATH”** and **§9.4.3 “Computed Function Calls”** — how the PC is assembled from the opcode and `PCLATH`, now that you’ve used it.
- **Data sheet §41** — the `goto`, `call`, and `movlp` opcodes whose 11-bit field started this.
- **User’s Guide §4.1.2, §4.1.3, §4.1.7** — `PAGESEL`, `PAGEMASK`, and the `fcall/ljmp` pseudo-instructions, formally.

Next chapter: you’ve now met `psects` a dozen times — `code`, `resetVec`, `udata_shr`, `farCode` — and positioned them with linker options. Chapter 10 finally explains **psects themselves** in full: classes, flags (`delta`, `space`, `reloc`), and how the linker turns your named sections into real addresses. It’s the concept the official guides assume you already know.

Chapter 10 — Psects: Organizing Code & Data

***What you'll build:** nothing new runs — instead you'll finally understand the line you've typed in every program. You'll define your own custom psects for code and data, build, then open the **map file** and watch the linker turn your named sections into real addresses. This is the concept the official guides assume you already know. After this chapter, you do.*

10.1 The idea: you write sections, the linker places them

You never told the assembler “put this instruction at address 0x0123.” You wrote a **psect** — a named container — dropped code or data into it, and let the **linker** decide the final address. That indirection is the heart of how PIC assembly is organized, and it's why your programs are *relocatable*: you name where things belong *categorically* (“this is code,” “this is a bank-0 variable”), and the linker resolves the actual numbers.

A **psect** (“program section”) is a **named block of code or data, normally relocatable**. (The advanced `abs` flag makes one non-relocatable.) The `PSECT` directive “declares or resumes a program section” (User's Guide §4.9.48). Everything emitted or reserved between one `PSECT` line and the next goes into that section. At link time, the linker:

1. gathers every **global** psect of the same name (even across multiple files),
2. finds a memory range appropriate for it (its **class**),
3. assigns it a concrete start address, and
4. resolves every label inside it to a real number.

You've been doing all four, on faith, since Chapter 4. Now let's see the machinery.

10.2 A psect declaration, dissected

Recall the reset-vector line you’ve written five times:

```
PSECT resetVec,class=CODE,delta=2
```

The syntax is PSECT name, flag, flag, ... (User’s Guide §4.9.48). Here resetVec is the name and class=CODE, delta=2 are two **flags**. The full flag list is Table 4-10 in the guide; for beginner work you need exactly four of them. Here they are, each verified against §4.9.48:

Flag	What it controls	The rule for the PIC16F17146
class=	which linker class (memory range) the psect goes in	e.g. CODE for program memory, COMMON for common RAM (§4.9.48.3)
delta=	bytes per address	program memory is word-addressable → delta=2 ; data memory is byte-addressable → delta=1 (the default) (§4.9.48.4)
space=	which memory space when addresses overlap	0 = program memory, 1 = data memory (§4.9.48.18)
reloc=	alignment boundary	default 1 on mid-range (no alignment needed); PIC18 code needs 2 (§4.9.48.16)

class= — which memory

A **class** is a named range of addresses the linker can place a psect into. class=CODE means “this belongs in program memory”; class=COMMON means “put it in common RAM.” Associating a psect with a class is usually all you need — the linker then finds room for it anywhere in that range, and you don’t have to name a specific address (§4.9.48.3). The classes themselves (CODE, RAM, COMMON, BANK0...) are predefined for your device once you #include <xc.inc> (User’s Guide §5.3, covered in Chapter 8).

delta= — how wide is an address

This is the one that feels strange until it clicks. On the PIC16F17146, **program memory stores 14 bits at each address, which takes 2 bytes in the HEX file** — so a code psect uses `delta=2`. The guide states the consequence bluntly: “addresses in the HEX file will not match addresses in the program memory” (§4.9.48.4). Data memory is plain bytes, so data psects use `delta=1` (the default — that’s why your `udata_shr` variables never needed it). **Rule of thumb: `delta=2` on every code psect, nothing on data psects.**

space= — resolving the great address collision

Program address 0 and data address 0 are *different physical places* (Harvard architecture, Chapter 1). The `space` flag tells the linker which one you mean: **space 0 is program memory, space 1 is data memory** (§4.9.48.18). The assembler-provided data psects already set `space=1` for you; you’ll only write it explicitly when you roll your own data psect (as we do below).

reloc= — alignment (you rarely touch it on mid-range)

`reloc=n` forces a psect to start on an address that’s a multiple of `n`. Mid-range code has no alignment requirement, so the default `reloc=1` is fine — which is why our `resetVec` never needed it. (PIC18 code *must* be word-aligned with `reloc=2`; a difference worth knowing when you reach Chapter 15.) (§4.9.48.16.)

Two more flags worth naming now. `global` (the default) concatenates same-named psects across files; `local` keeps a psect private to its module (§4.9.48.5, §4.9.48.9). You’ll use `global` implicitly the moment you split a project in Chapter 13.

10.3 Predefined vs. your own psects

You’ve used both kinds without noticing:

- **Assembler-provided psects** — `code`, `udata`, `udata_shr`, `data` (User’s Guide §5.2, from Chapter 6). Each comes pre-wired to the right class, so PSECT code just works.
- **User-defined psects** — `resetVec`, `farCode` (Chapter 9). You invent the name and supply the flags yourself. You do this when a section

needs to be placed at a *specific* address, because a uniquely-named psect can be positioned with a linker `-P` option — which is exactly how we put `resetVec` at 0 and `farCode` at 0x0800.

That’s the division: use a provided psect when you just need “some code” or “some RAM”; make your own when *where it lands* matters.

10.4 Placing a psect: `class` vs. `-P`

There are two ways a psect gets an address (User’s Guide §4.9.48.3, and the `-P` option):

1. **By class** — “put this anywhere in the `CODE` range.” You just give `class=CODE` and the linker finds room. This is the normal case.
2. **By position (`-P`)** — “put this psect at exactly address X.” You pass a linker option like `-Wl,-presetVec=0h` (reset vector at 0) or `-Wl,-pfarCode=0800h` (Chapter 9). Every build command you’ve run used the first form for `resetVec`.

ORG is not what you think. *Newcomers from other assemblers reach for `ORG` to set an absolute address. On this assembler, “the much-abused `ORG` directive moves the location counter to an offset relative to the base of the current psect, not to an absolute address” (User’s Guide §4.9.42). To place something at a fixed address, give it its own psect and use `-P` — not `ORG`. You will almost never need `ORG`.*

10.5 The program — roll your own psects

Let’s define a custom code psect and a custom data psect, so you can find *your* names in the map file rather than the library’s.

```
; -----
; psects.S - define custom code and data psects, then read
;           the map file to see where the linker put them.
; -----

PROCESSOR 16F17146
#include <xc.inc>
; --- paste your Chapter 5 CONFIG block here ---
```

```

; --- a user-defined DATA psect in bank 0 GPR -----
; space=1 → data memory; class BANK0 → general RAM of bank 0.
    PSECT myVars,class=BANK0,space=1
counter:
    DS      1                ; reserve 1 byte
scratch:
    DS      1                ; reserve another

; --- reset vector (positioned at 0 by the linker option) ----
    PSECT resetVec,class=CODE,delta=2
resetVec:
    goto    main

; --- a user-defined CODE psect -----
    PSECT myCode,class=CODE,delta=2
main:
    BANKSEL counter
    clrf    BANKMASK(counter)
    incf    BANKMASK(counter),f
    goto    $

    END      resetVec

```

Notice the two custom psects: `myVars` (data, `space=1`, class `BANK0`, no delta because data is byte-addressable) and `myCode` (code, class `CODE`, `delta=2`). Everything you learned in §10.2 is on display in four lines.

10.6 Build it and read the map file

```

pic-as -mcpu=16f17146 -mdfp=/path/to/PIC16F1xxx_DFP/VERSION/xc8 \
    -Wl,-presetVec=0h -Wa,-a -Wl,-Map=psects.map psects.S

```

The `-Wl,-Map=psects.map` option produces the **map file** — the linker’s report of where everything ended up. Open `psects.map` and look for a section listing your psects. You’ll find entries for `resetVec`, `myCode`, and `myVars`, each with a **start address**, a **length**, and the **class** it was drawn from. That is the linker showing its work: your named sections, turned into concrete addresses.

Things to confirm in the map: - `resetVec` starts at **0x0000** (because of `-presetVec=0h`). - `myCode` sits somewhere in the `CODE` range (program memory), placed automatically by class. - `myVars` sits in bank-0 general RAM (the `BANK0` class), and `counter/scratch` occupy two consecutive addresses there.

Symbols in the map. By default only GLOBAL symbols appear in the map/symbol files (User's Guide §4.6.5, Chapter 4). Don't be surprised if a plain local label is absent — that's expected, not a bug. Chapter 17 reads the map file in full detail.

10.7 What just happened

You saw the whole pipeline stop being magic. A **psect** is a named container; its **flags** (class, delta, space, reloc) tell the linker what *kind* of memory it needs; the **linker** gathers same-named psects, drops each into its class's address range (or a specific address via -P), and resolves every label. The **map file** is the receipt. Every PSECT, class=CODE, delta=2, and -w1, -p... line from the previous six chapters now has a reason you can articulate.

10.8 Common mistakes

Symptom	Cause	Fix
Code psect won't fit / wrong bytes in HEX	forgot delta=2 on a code psect	mid-range code is word-addressable — always delta=2 (§4.9.48.4)
Data psect placed in program memory (or vice versa)	wrong or missing space flag	data psects need space=1; code uses space=0 (default) (§4.9.48.18)
Linker "can't place psect" / class error	class= names a class that doesn't exist for the device	use a predefined class (CODE, BANK0, COMMON, RAM) (§5.3)
ORG 0x100 didn't put code at 0x100	ORG is psect-relative, not absolute	give it its own psect + -P linker option (§4.9.42)
Two PSECT lines, "contradictory flags" error	re-declared a psect with different flags	flags propagate from the first declaration; don't restate them differently (§4.9.48)

Symptom	Cause	Fix
Your label isn't in the map file	it isn't GLOBAL	that's normal; declare it GLOBAL if you need it listed (§4.6.5)

(Appendix C decodes the exact message wording.)

10.9 Try it yourself

1. **Find your names.** Build `psects.S`, open `psects.map`, and locate `myCode` and `myVars`. Write down each one's start address and length.
2. **Force a location.** Add `-Wl,-pmyCode=0100h` to the build, rebuild, and confirm in the map that `myCode` now starts at `0x0100`. You just positioned a psect by hand.
3. **Break the delta.** Change `myCode's delta=2` to `delta=1`, rebuild, and read the error or the mangled addresses. Restore it — and remember why mid-range code is always `delta=2`.
4. **Same name, two blocks.** Add a second `PSECT myCode,class=CODE,delta=2` later in the file with another instruction, and confirm from the map that the linker concatenated both into one `myCode` section.

10.10 Reference bridge

- **User's Guide §4.9.48 "Psect Directive"** — the complete flag table (Table 4-10) and every flag's detailed description; you can now read all of it.
- **User's Guide §5.2-5.3** — the assembler-provided psects and the predefined linker classes they map to.
- **User's Guide §4.9.42 "Org Directive"** — why `ORG` is relative, and what to use instead.

Next chapter: your variables have so far been single bytes. Chapter 11 uses psects and the enhanced mid-range **linear memory** view to build variables *bigger* than one byte — a 16-bit counter you can increment across its full range — and shows how the FSRs reach contiguous data that spills past a single bank.

Chapter 11 — Linear Memory & Larger Variables

***What you'll build:** a **16-bit counter** — a variable too big for one 8-bit register — that counts from 0 all the way to 65535 and rolls over, incrementing correctly across the byte boundary. Then you'll meet the **FSR/INDF** indirect-addressing machinery and the **linear memory** view that lets a buffer grow past the 80-byte limit of a single bank.*

11.1 The idea: bytes are small, so glue them together

WREG and each directly addressable data-memory register hold 8 bits — 0 to 255. Real programs need bigger numbers: a millisecond timer, an ADC reading, a byte count. The answer is simple and universal: **use more than one byte and treat them as one number**. A 16-bit value is just two adjacent bytes — a **low byte** (bits 0–7) and a **high byte** (bits 8–15) — that you agree to read together.

The only new skill is **carrying** between them. When the low byte counts past 0xFF it wraps back to 0x00, and that overflow has to bump the high byte — exactly like the ones column rolling into the tens column when you count past 9.

11.2 Incrementing a 16-bit value

Reserve two bytes and pick a convention. We'll put the low byte first (the common "little-endian" choice): `counter` is the low byte, `counter+1` is the high byte.

```
PSECT udata_shr          ; common RAM - no banking to worry about
counter:
DS      2                ; 16-bit counter: counter = low, counter+1 = high
```

To add one to the whole 16-bit value, increment the low byte; **only if it wrapped to zero** do we also increment the high byte. The clean way uses `INCFSZ` — “increment f, skip the next instruction if the result is 0” (data sheet §41):

```
incfsz counter,f      ; low++ ; skip the goto if it just wrapped to 0x00
goto    ctr_done      ; no wrap → the high byte is unchanged, done
incf    counter+1,f    ; low wrapped 0xFF→0x00, so carry into the high
↪ byte
ctr_done:
```

Trace it: if counter goes `0x34 → 0x35`, it’s non-zero, so `incfsz` does **not** skip — the `goto ctr_done` runs and we’re finished. If counter goes `0xFF → 0x00`, the result **is** zero, so `incfsz` skips the `goto`, falling through to `incf counter+1,f` — the carry. That’s a full 16-bit increment in three instructions, and it never needs `BANKSEL` because both bytes live in common RAM.

***Why not just check the Carry flag?** `incf` sets the **Z** (zero) flag, not the Carry flag, so the natural test for “did it wrap?” is “did it become zero?” — which is exactly what `INCFSZ` tests in one instruction. (For addition of a value you’d use `ADDWF/ADDWFC`, which do use Carry; we’ll meet those when we do real arithmetic.)*

11.3 The whole program — count to 65535 and watch it

```
; -----
; count16.S - a 16-bit counter that rolls over at 65535
; -----

PROCESSOR 16F17146
#include <xc.inc>
; --- paste your Chapter 5 CONFIG block here ---

PSECT udata_shr
counter:
    DS      2                ; low byte = counter, high byte = counter+1

PSECT resetVec,class=CODE,delta=2
resetVec:
    goto    main
```

```

PSECT code
main:
    clrf    counter           ; start at 0x0000 (common RAM: no BANKSEL)
    clrf    counter+1
loop:
    incfsz  counter,f         ; low++
    goto    loop              ; not zero → keep going (high unchanged)
    incf    counter+1,f       ; low wrapped → bump high byte
    goto    loop              ; ...and continue counting

    END    resetVec

```

Build it and watch the counter climb:

```

pic-as -mcpu=16f17146 -mdfp=/path/to/PIC16F1xxx_DFP/VERSION/xc8 \
      -Wl,-presetVec=0h -Wa,-a -Wl,-Map=count16.map count16.S

```

In the simulator, put a Watch on both `counter` and `counter+1`. Step (or run with a breakpoint on `incf counter+1,f`) and you'll see the low byte race 0x00→0xFF over and over, and each time it rolls over, the high byte ticks up by one. Just before the 256th low-byte rollover the value is 0xFFFF; that rollover changes both bytes to 0 and wraps the whole value to 0x0000. That's a real 16-bit variable working.

(Tip: set a breakpoint on the `incf counter+1,f` line — it only fires once every 256 counts, so you see the carry happen without stepping 65,000 times.)

11.4 Reaching data indirectly: the FSRs

A 16-bit *number* is two glued bytes. A 16-bit *pointer* is something more powerful. The enhanced mid-range core has two **File Select Registers**, FSR0 and FSR1, each a 16-bit FSRnH:FSRnL pair (data sheet §7.3, §9.6). An FSR holds an **address**, and the matching **INDFn** register is a window onto whatever that address points at:

“The INDFn registers are not physical registers. Any instruction that accesses an INDFn register actually accesses the register at the address specified by the FSR.” (data sheet §9.6)

So FSR0 is a pointer and INDF0 is “the byte it points to.” This is how you walk arrays, buffers, and tables without hard-coding each address.

Because the FSR is 16 bits, it names a full **65,536-location** space. The usable windows (plus reserved gaps) are (§9.6):

FSR address range	Region	What it reaches
0x0000–0x1FFF	Traditional/Banked	the absolute address of every SFR, GPR, and common byte (§9.6.1)
0x2000–0x2FEF	Linear Data Memory window	up to bank 50's GPR blocks, laid end-to-end; unimplemented locations read 0 (§9.6.2)
0x2FF0–0x6FFF 0x7000–0x70FF	Reserved Data EEPROM	do not use read-only through <code>INDF</code> ; writes use the NVM interface (§9.6.4)
0x7100–0x7FFF 0x8000–0xFFFF	Reserved Program Flash	do not use lower 8 bits of each program word, read-only through <code>INDF</code> (§9.6.3)

The auto-increment moves

The real power is that reading or writing through an FSR can *advance the pointer in the same instruction* (data sheet §41, `MOVIW/MOVWI`):

Syntax	Meaning
<code>moviw FSR0++</code>	$W \leftarrow *FSR0$, then $FSR0++$ (post-increment)
<code>movwi FSR0++</code>	$*FSR0 \leftarrow W$, then $FSR0++$
<code>moviw ++FSR0 / --FSR0</code>	pre-increment / pre-decrement, then access
<code>moviw k[FSR0]</code>	access $*(FSR0 + k)$, pointer unchanged ($-32 \leq k \leq 31$)

There's also `ADDFSR FSR0,k` to jump the pointer by a signed offset ($-32\dots31$). Together these turn the FSR into a clean array walker.

11.5 Walking a buffer with a pointer

Here's the indirect model in action — fill an 8-byte buffer with 0,1,2,...,7:

```

PSECT udata                ; a GPR buffer (linker picks a bank)
buf:
    DS      8
    PSECT udata_shr
count: DS  1

; ... in code ...
; point FSR0 at buf (its absolute address is in the 0x0000-0x1FFF region)
movlw     low(buf)
movwf     FSR0L
movlw     high(buf)
movwf     FSR0H

movlw     8
movwf     count        ; 8 items to write
clrw      ; W = 0, the first value to store
fill:
    movwi   FSR0++      ; *FSR0 = W, then advance the pointer
    addlw   1           ; next value
    decfsz  count,f     ; done all 8?
    goto    fill

```

The `low()` and `high()` operators (User's Guide §4.7) split `buf`'s address into the two bytes we load into `FSR0L/FSR0H`. Then each `movwi FSR0++` stores a byte *and* steps the pointer forward — no manual address arithmetic. Because `buf` is a GPR variable, its absolute address falls in the **traditional** FSR region (0x0000–0x1FFF), so this “just works.”

11.6 When one bank isn't enough: linear memory

Notice the catch: a single bank holds only **80 bytes** of GPR (0x20–0x6F; Chapter 8). What if you need a 200-byte buffer? In the traditional view, incrementing an FSR past a bank's GPR runs into core registers and SFRs — garbage. That's what **linear data memory** solves (data sheet §9.6.2):

“Use of the linear data memory region allows buffers to be larger than 80 bytes because incrementing the FSR beyond one bank will go directly to the GPR memory of the next bank.”

The architectural window 0x2000–0x2FEF is a *virtual* map that stitches the GPR blocks for banks 0 through 50 into one run: bank 0’s GPR, then bank 1’s, then bank 2’s, and so on. This does **not** mean every device implements that whole window. For the PIC16F17146, the selected DFP defines the implemented BIGRAM range as **0x2000–0x27EF**, corresponding to its implemented GPR in banks 0–25 (bank 25 has only 32 GPR bytes). Point an FSR at an implemented linear address and `movlw FSR0++` sails from one bank’s RAM straight into the next — no SFR/common gaps.

Three caveats worth remembering:

- **Common RAM (0x70–0x7F) is *not* part of the linear region** (§9.6.2). Linear memory is GPR only.
- Unimplemented linear locations read as 0; never size an object from the architectural maximum.
- The linear address of a byte isn’t the same number as its banked address — it’s its position in that stitched-together run.

Reserving a buffer that really crosses banks

A normal `PSECT udata` object must fit one RAM class range, so it cannot cross a bank boundary. For a large enhanced-mid-range object, use `DLABS` (User’s Guide §4.9.12 and Embedded Engineers guide §5.3). Its address argument can be banked or linear; the symbol it creates is always the equivalent **linear** address, and the linker reserves the underlying physical GPR so ordinary variables cannot overlap it:

```
#define BIG_SIZE 200

; Reserve 200 physical GPR bytes beginning at banked address 0x20.
; The generated symbol bigBuf has linear value 0x2000.
DLABS    1, 0x20, BIG_SIZE, bigBuf

; ... in code: point FSR0 at the linear object ...
    movlw    low(bigBuf)
    movwf    FSR0L
    movlw    high(bigBuf)
    movwf    FSR0H
```

`DLABS` does not belong to the current psect and emits no initial data. Because it makes a fixed-address reservation, you must choose the banked start and length from the exact device’s implemented GPR map and check the final map file. Here 200 bytes consume all 80 bytes of bank 0, all 80 of bank 1, and 40 bytes of bank 2; consecutive FSR accesses use 0x2000–0x20C7. For the small, single-bank buffers a beginner writes, the traditional approach in §11.5 remains simpler.

The takeaway: **little variables** → **glue bytes together (§11.2)**; **big buffers that outgrow a bank** → **linear memory via the FSRs**. Same two registers, two levels of power.

11.7 What just happened

You broke the 8-bit ceiling two different ways. For a **number** bigger than a byte, you chained bytes and carried between them with `INCFSZ`. For **data** bigger than a register — arrays and buffers — you used the FSR/INDF pointer machinery, with auto-incrementing `moviw/movwi` moves, and you learned that the **linear memory** view lets a buffer span banks as if the 80-byte walls weren't there.

11.8 Common mistakes

Symptom	Cause	Fix
16-bit counter high byte never changes	tested the wrong condition, or used <code>incf</code> +separate check wrong	use <code>incfsz</code> on the low byte, <code>incf</code> the high byte only when it skips
High byte increments every count	logic inverted (carried when <i>not</i> zero)	<code>incfsz</code> skips the “done” goto <i>only</i> on wrap — carry belongs after the skip
Bytes read as a garbled 16-bit value	mixed up which byte is low vs. high	pick one convention (counter = low, counter+1 = high) and keep it
FSR walk hits non-buffer data after ~80 bytes <code>moviw FSR0++</code> won't assemble	buffer outgrew one bank in the traditional region wrong increment syntax	reserve it with <code>DLABS</code> and use its linear symbol (§4.9.12, §9.6.2) it's <code>FSRn++/FSRn--</code> / <code>++FSRn/--FSRn</code> (data sheet §41)
Pointer reads zero everywhere	<code>FSR0</code> points at an <code>INDF</code> address, or was never loaded	load <code>FSR0L/FSR0H</code> with a real address first (§9.6)

(Appendix C decodes the exact message wording.)

11.9 Try it yourself

1. **Count faster to see the carry.** Preload counter with 0xFA in the low byte before the loop, so the first rollover happens in six counts. Watch the high byte tick.
 2. **Make it 24-bit.** Extend counter to DS 3 and add a second carry stage (`incfsz counter+1` guarding `incf counter+2`). Confirm it rolls over at 16,777,215.
 3. **Read the buffer back.** After filling `buf` in §11.5, re-point `FSR0` at `buf` and use `moviw FSR0++` in a loop to read each byte into `W`; watch the values 0...7 appear.
 4. **Sum a buffer.** Walk `buf` with `moviw FSR0++`, adding each byte into a running total with `addwf` — your first indirect-addressing algorithm.
-

11.10 Reference bridge

- **Data sheet §9.6 “Indirect Addressing”** — the FSR/INDF model, traditional and linear data, the EEPROM read window, program Flash reads, and reserved address ranges.
- **Data sheet §41** — the `MOVIW`, `MOVWI`, `ADDFSR`, and `INCFSZ` instruction details.
- **User’s Guide §4.7** — the `low()`/`high()` operators for splitting an address.
- **User’s Guide §4.9.12 and Embedded Engineers guide §5.3** — DLABS, fixed physical reservation, and its automatically generated linear symbol.

Next chapter (Part IV): you’ve been writing straight-line code with the occasional loop. Chapter 12 broadens your toolkit with the **directives you’ll actually use** — `EQU`, `DB/DW` data tables, `ORG`, conditional assembly, and the macro basics — so your programs can define constants, embed lookup tables, and adapt at build time.

PART IV

Building Real Programs

Chapter 12 — Directives You'll Actually Use

***What you'll build:** a program with a **DB lookup table** — a table of squares baked into program memory — that you read back by index at run time. Along the way you'll pick up the everyday directives that make assembly readable and adaptable: named constants (EQU/SET), data definition (DB/DW/DS), and build-time decisions (IF/ELSE). These are the tools that turn magic numbers into names and hard-coded data into tables.*

12.1 The idea: directives talk to the assembler, not the chip

You met the instruction-vs-directive split back in Chapter 4: an instruction becomes machine code the CPU runs; a **directive** is an order to the *assembler* that shapes the build. The User's Guide is blunt about it — “with the exception of PAGESEL and BANKSEL, these directives do not generate instructions” (§4.9). You've already used a handful (PROCESSOR, PSECT, CONFIG, END, DS). This chapter adds the rest of the everyday set. There are dozens in the reference (§4.9, Table 4-6); you need about eight.

12.2 Named constants: EQU and SET

Magic numbers are the enemy of readable assembly. EQU gives a number a name (User's Guide §4.9.16):

```
LED_PIN    EQU    1           ; RC1 is bit 1 - name it once
MAXCOUNT  EQU    100
BAUD       EQU    9600
```

Now you write `bcf LATC, LED_PIN` instead of a bare 1, and if the LED ever moves you change one line. Key rules from §4.9.16:

- **EQU is define-once.** “EQU is legal only when the symbol has not previously been defined.” Try to redefine an EQU symbol and you get an error.
- **It reserves no memory.** EQU just names a value; it’s like the pre-processor’s #define.
- **No location counter.** You can’t use \$ in an EQU expression (they’re processed separately from the code stream).

When you *need* a value that changes as assembly proceeds, use **SET** instead — “defines or re-defines symbol value” (§4.9, Table 4-6):

```
row    SET    0
row    SET    row+1          ; legal - SET can be reassigned; now 1
```

Rule of thumb: EQU for real constants (pin numbers, limits), SET for build-time counters you deliberately step.

12.3 Reserving vs. initializing memory

Two different jobs, two different directives — and beginners mix them up constantly:

Directive	Job	Goes where
DS n	reserve n uninitialized address units	current psect (normally data memory for variables)
DB / DW	initialize constant bytes / 16-bit values	program memory

DS — reserve RAM (you’ve used this)

DS n “reserves, but does not initialize, the specified amount of space” (§4.9.13). The unit depends on the psect: in a data psect (space=1) it reserves **bytes**; that’s every DS 1 / DS 2 you’ve written for variables. In a program psect it advances the location counter without placing initialized bytes in the HEX file. For variables, keep it in a data psect.

DB/DW — bake constant data into the program

DB “initializes bytes of program memory” (§4.9.9); DW does 16-bit words (§4.9.14). Arguments can be numbers, character constants, or strings:

```
PSECT myData,class=STRCODE,delta=2,noexec
msg:  DB  "Hello",0          ; a nul-terminated string (you add the 0)
sq:   DW  0,1,4,9,16        ; five 16-bit values
```

Two facts that trip people up (both from §4.9.9):

- **Strings are not auto-terminated.** DB "Hello" stores exactly five bytes — no trailing zero. If you want a C-style nul terminator, add it yourself: DB "Hello",0.
- **On mid-range, a DB byte costs a whole word.** Program memory here is 14 bits wide (delta=2), so DB 'X',1,2 stores each byte in the low 8 bits of a word with the upper byte zeroed — the HEX shows 0058 0001 0002. You get one program word per byte. That’s normal; just don’t expect bytes to pack two-per-word like they would on a PIC18.
- **DB/DW can’t make RAM variables.** “The DB directive cannot be used to create objects in data memory. For that, use the DS directive” (§4.9.9). DB is for *constants in flash*; DS is for *variables in RAM*.

XC8 4.00 width note. The 2024 guide documents DDW for 32-bit constants, but the XC8 4.00 release notes still list it as unsupported (known issue XC8-1817). XC8 4.00 adds DQW for 64-bit constants (XC8-2722). This chapter sticks to verified DB and DW; do not rely on DDW without testing the exact installed tool version.

12.4 The program — a DB lookup table you read at run time

Let’s bake a table of squares ($0^2 \dots 7^2$) into program memory with DB, then read an entry by index. Reading program-memory data uses the FSR trick from Chapter 11: point an FSR into the 0x8000+ program-Flash window by setting bit 7 of FSR0H (data sheet §9.1.1.2, §9.6.3).

The data-sheet example says its HIGH directive sets that mapping bit for a program-memory label, but XC8 4.00 pic-as high(label) only extracts

the numerical high byte. Therefore the code below sets the bit explicitly with `| 0x80`; the generated disassembly must show `FSR0H` loaded with a value whose bit 7 is set.

```

; -----
; table.S - a DB lookup table of squares, read by index
; -----

PROCESSOR 16F17146
#include <xc.inc>
; --- paste your Chapter 5 CONFIG block here ---

INDEX      EQU 3                ; which entry to look up (3 → 3*3 = 9)

PSECT udata_shr
result: DS 1                    ; the looked-up value lands here

PSECT resetVec,class=CODE,delta=2
resetVec:
goto main

PSECT code,delta=2
main:
; point FSR0 at the squares table in PROGRAM memory
movlw low(squares)
movwf FSR0L
movlw high(squares) | 0x80 ; explicit program-flash mapping bit for XC8
↪ pic-as movwf FSR0H

moviw INDEX[FSR0]             ; W ← squares[INDEX] (relative-offset read)
movwf result                  ; stash it (common RAM: no BANKSEL)
goto $

; --- the lookup table, baked into program memory -----
PSECT data                    ; provided STRCODE program-data psect on PIC16
squares:
DB 0, 1, 4, 9, 16, 25, 36, 49 ; square(0) .. square(7)

END resetVec

```

Build and watch it:

```

pic-as -mcpu=16f17146 -mdfp=/path/to/PIC16F1xxx_DFP/VERSION/xc8 \
      -Wl,-presetVec=0h -Wa,-a -Wl,-Map=table.map table.S

```

In the simulator, put a Watch on `result`, run to the `goto $`, and confirm `result = 9` ($= 3^2$). Change `INDEX` to 5, rebuild, and it becomes 25. You

just stored constant data in flash and indexed into it — the foundation of every font table, sine table, and state-machine table you'll ever write.

***The classic alternative: RETLW tables.** Older code builds lookup tables from a chain of RETLW instructions reached by a computed jump (BRW/ADDWF PCL) — the data sheet shows this pattern in §9.1.1.1. The DB + FSR method above is the modern, readable equivalent; know that RETLW tables exist because you'll meet them in other people's code.*

12.5 Build-time decisions: conditional assembly

IF/ELSIF/ELSE/ENDIF let you include or exclude code **when you build**, not at run time (User's Guide §4.9.31). The argument is a constant expression; if it's non-zero, the block is assembled:

```
DEBUG    EQU    1                ; flip to 0 for a production build

    IF DEBUG
        bsf     LATC,0           ; light a debug pin
    ELSE
        nop                    ; production: do nothing
    ENDIF
```

This is how one source file produces different builds — debug vs. release, or one board variant vs. another — with no run-time cost, because the un-taken branch never becomes machine code. Two cautions from §4.9.31:

- Both branches are still *scanned*, so they must be syntactically valid even when not assembled.
- **Don't put EQU inside an IF.** EQU (and other directives) are processed regardless of the condition, so an EQU in a false branch still takes effect. Keep constant definitions outside conditional blocks.

Fail the build on purpose: ERROR and MESSG

You can make the assembler stop with your own message when something's misconfigured (§4.9.17, §4.9.38):

```
IF BAUD > 115200
    ERROR "BAUD rate too high for this clock" ; halts assembly
ENDIF
```

ERROR halts the build; MESSG prints an advisory but lets it continue. These turn silent mistakes into loud, build-time failures — a cheap, powerful safety net.

12.6 A quick tour of the rest

You’ll reach for these occasionally; know they exist (all User’s Guide §4.9):

Directive	Use
ORG <i>n</i>	move the location counter <i>within the current psect</i> (relative, not absolute — §4.9.42; Chapter 10)
RADIX	change the default number base
REPT <i>n</i> / IRP / IRPC	repeat a block <i>n</i> times / once per list item / once per character
TITLE / SUBTITLE	set listing-file headers
#include < <i>xc.inc</i> >	pull in the device register definitions (you use this every file)

MACRO/ENDM, GLOBAL, and EXTRN are directives too — but they’re the heart of the *next* chapter, so we’ll give them full treatment there rather than a one-line mention.

12.7 What just happened

You expanded from “instructions only” to the directive toolkit real programs lean on: EQU/SET to name values, DS to reserve RAM vs. DB/DW to bake constants into flash, a working DB lookup table read by index, and IF/ELSE/ERROR to make build-time decisions and catch mistakes before they ship. None of these emit instructions — they shape *what gets built* — and that’s exactly why they make code cleaner and more adaptable.

12.8 Common mistakes

Symptom	Cause	Fix
"symbol already defined" on a constant	redefined an EQU	use SET if you need to reassign (§4.9.16)
DB "text" behaves as if longer/shorter than expected	strings aren't auto-terminated	add your own ,0 for a nul terminator (§4.9.9)
Lookup table reads garbage	FSR points at banked/reserved data instead of 0x8000+ program Flash	load FSR0H with high(label) \ 0x80 and verify bit 7 in disassembly (§9.1.1.2)
DB used for a variable, value never changes	DB writes flash (read-only at run time)	use DS in a space=1 psect for RAM variables (§4.9.13)
Code in a false IF branch still assembled	it was an EQU/directive, processed regardless	move constant definitions outside the IF (§4.9.31)
Byte table twice the expected size in flash	mid-range stores one word per DB byte	expected on 14-bit cores (delta=2); not a bug (§4.9.9)

(Appendix C decodes the exact message wording.)

12.9 Try it yourself

1. **Name the blink pin.** Go back to Chapter 7's `blink.s` and replace every literal 1 (the RC1 bit) with `LED EQU 1`. Confirm it still builds and blinks — and that changing one line would move the LED.
2. **Extend the table.** Add $8^2 \dots 10^2$ to the squares table, set `INDEX` to 9, and confirm `result = 81`.
3. **Store a string.** Define `DB "PIC",0` in a code psect, then read the three characters back into RAM one at a time with `moviw FSR0++`. Watch 'P','I','C' appear.
4. **Guard a constant.** Add `IF INDEX > 7 / ERROR "index out of range" / ENDIF` above the lookup, set `INDEX` to 9, and watch the build fail with your message. Then extend the table so it passes.

12.10 Reference bridge

- **User’s Guide §4.9** — the full directive table (Table 4-6) and every directive’s detail; you can now navigate all of it.
- **User’s Guide §4.9.9 / §4.9.14 / §4.9.13** — DB, DW, and DS specifics.
- **User’s Guide §4.9.16 / §4.9.31** — EQU/SET and conditional assembly.
- **User’s Guide §4.7** — expression operators, including what `pic-as low()` and `high()` actually extract from a symbol value.
- **Data sheet §9.1.1 / §9.6.3** — reading constants from program memory (RETLW and FSR methods).
- **XC8 4.00 release notes, “What’s New” and “Known Issues”** — DQW addition (XC8-2722) and the still-unsupported DDW directive (XC8-1817).

Next chapter: you’ve written everything in one file so far. Chapter 13 introduces **macros** (MACRO/ENDM) to package repeated instruction sequences, and **GLOBAL/EXTRN** to split a program across multiple source files and link them together — the first step toward organizing a real, growing project.

Chapter 13 — Macros & Multiple Source Files

***What you'll build:** the Chapter 7 blink again — but split across **two source files** with a **shared macro library**. A reusable delay routine lives in its own file and is linked in; a pin-setup macro lives in an include file any source module can share. You'll learn the two distinct ways code gets reused: **macros** (textual, compile-time) and **linked modules** (compiled, GLOBAL/EXTRN) — and why they're different.*

13.1 The idea: two very different kinds of reuse

As a program grows, one giant file becomes painful. There are two tools for breaking it up, and the single most important thing in this chapter is understanding that **they are not the same thing**:

	Macro	Linked module
What it is	a named block of source text	a separately-assembled .o object file
When it happens	assembly time — expanded at each invocation	link time — combined by the linker
How it's shared	#include the file holding it	GLOBAL/EXTRN + list both files at build
Cost	code is <i>duplicated</i> at each use	code exists <i>once</i> , called via call
Good for	short, repeated instruction patterns	real subroutines used from many places

A macro is copy-paste the assembler does for you. A linked module is a real routine that lives at one address and gets *called*. Use a macro for a two-line pattern you repeat; use a module for a delay routine, a math helper, an LCD driver.

13.2 Macros: patterns the assembler pastes in

A macro defines a sequence of source lines, optionally with arguments, that expand wherever you invoke it (User’s Guide §4.9.37). Here’s a genuinely useful one — configure any port pin as a digital output — saved in a shared include file:

```
; =====
; macros.inc - shared macro library
; =====

; CONFIG_OUTPUT port, bit, initial : set the latch, then make the pin a
; ⇨ digital output
CONFIG_OUTPUT MACRO port, bit, initial
    BANKSEL ANSEL&port          ; & concatenates: ANSEL + C → ANSELC
    bcf    BANKMASK(ANSEL&port), bit ; pin → digital
    BANKSEL LAT&port            ; set latch BEFORE changing direction
; ⇨ (no output glitch)
    IF initial
        bsf BANKMASK(LAT&port), bit
    ELSE
        bcf BANKMASK(LAT&port), bit
    ENDIF
    BANKSEL TRIS&port
    bcf    BANKMASK(TRIS&port), bit ; pin → output; macro leaves BSR here
ENDM
```

Two macro features on display:

- **Arguments.** port, bit, and initial are substituted at each use. CONFIG_OUTPUT C, 1, 1 sets up RC1 with an initial high latch value (LED off on the active-low Curiosity Nano). Loading the latch before clearing TRIS avoids a startup glitch.
- **& token concatenation.** Inside a macro body, & glues tokens together, then vanishes from the expansion (User’s Guide §4.9.37). So ANSEL&port with port = C becomes the real register name ANSELC. (Because & has this special meaning in macros, use the word and for a bitwise AND inside one — the & operator is unavailable there.)

LOCAL — unique labels per expansion. If a macro contains a label and you use the macro twice, you’d get “duplicate symbol” errors — the label would be defined twice. The LOCAL directive fixes this by generating a unique label for each expansion (User’s Guide §4.9.36):

```
SHORT_WAIT MACRO reg
    LOCAL again          ; 'again' becomes a unique symbol each time
    BANKSEL reg
again:
```

```

    decfsz BANKMASK(reg),f
    goto    again
ENDM

```

Always LOCAL *any label inside a macro*.

Because a macro is **pasted in every time you invoke it**, its instructions are *duplicated* in your program. That's fine for a few lines; it's the wrong tool for a big routine — which is what modules are for.

13.3 Splitting code across files: GLOBAL and EXTRN

Put the reusable delay routine in its own file. To let another file call it, the *defining* file must **export** the symbol, and the *using* file must **import** it. The two directives (User's Guide §4.9.30, §4.9.20):

- **GLOBAL name** — makes a locally defined symbol public, or references an external one when there is no local definition. The definition needs GLOBAL; a consumer may use GLOBAL again or the stricter EXTRN form below.
- **EXTRN name** — import-only; declares a symbol defined *elsewhere*. It's an error to use EXTRN on a symbol defined in the same file (§4.9.20) — which makes your intent explicit.

Here's the delay routine as a standalone module:

```

; =====
; delay.S - a reusable ~0.75 s busy-wait, its own module
; =====
PROCESSOR 16F17146
#include <xc.inc>

GLOBAL delay           ; EXPORT: make 'delay' callable from other files

PSECT    udata_shr
d1: DS   1
d2: DS   1

PSECT    code,delta=2
delay:
    movlw 0xFF
    movwf d1
douter:
    movlw 0xFF
    movwf d2

```

```
dinner:
    decfsz    d2,f
    goto      dinner
    decfsz    d1,f
    goto      douter
    return
```

Notice `delay`'s private counters `d1/d2` live *with it*, in its own file — `main.S` never sees them. That encapsulation is the whole point of splitting into modules.

13.4 The main file — using both

```
; =====
; main.S - blinks RC1 using a shared macro + external delay
; =====
PROCESSOR 16F17146
#include <xc.inc>
#include "macros.inc" // pull in the macro library (textual)
; --- paste your Chapter 5 CONFIG block here ---

    EXTRN    delay          ; IMPORT: 'delay' is defined in delay.S

    PSECT    resetVec,class=CODE,delta=2
resetVec:
    goto     main

    PSECT    code,delta=2
main:
    CONFIG_OUTPUT C, 1, 1    ; RC1 digital output, latch high (LED initially
↪    off)
    BANKSEL  LATC
loop:
    bcf      LATC,1          ; LED on (active-low)
    fcall    delay           ; call the EXTERNAL routine (page-safe - Ch 9)
    bsf      LATC,1          ; LED off
    fcall    delay
    goto     loop

    END        resetVec
```

Two things to note:

- `#include "macros.inc"` makes the macro definition part of this preprocessed source module; the assembler then expands `CONFIG_OUTPUT` at its invocation. The include file is **not** a separate compiled module; macros aren't linked.
 - `fcall delay` (not plain `call`) because `delay` is in another module and could land in a different program-memory page — `fcall` handles the paging for you (Chapter 9).
-

13.5 Building a multi-file project

Single-step — list all the source files

The `pic-as` driver assembles and links in one go; just name every source file (User's Guide §3.1):

```
pic-as -mcpu=16f17146 -mdfp=/path/to/PIC16F1xxx_DFP/VERSION/xc8 \
      -Wl,-presetVec=0h -Wl,-Map=blink2.map main.S delay.S
```

The driver assembles `main.S` and `delay.S`, then links them — resolving `main.S`'s `EXTRN delay` to the `delay` exported by `delay.S`. (Note `macros.inc` is **not** on the command line — it's pulled in by `#include`, not linked.)

Multi-step — assemble once, link once (incremental builds)

For bigger projects you assemble each file to an object (`.o`) with `-c`, then link the objects (User's Guide §3.2). A make utility (or MPLAB X) then only re-assembles files that changed:

```
pic-as -mcpu=16f17146 -mdfp=/path/to/PIC16F1xxx_DFP/VERSION/xc8 -
c main.S # -> main.o
pic-as -mcpu=16f17146 -mdfp=/path/to/PIC16F1xxx_DFP/VERSION/xc8 -
c delay.S # -> delay.o
pic-as -mcpu=16f17146 -mdfp=/path/to/PIC16F1xxx_DFP/VERSION/xc8 \
      -Wl,-presetVec=0h main.o delay.o # link
```

Either way the LED blinks exactly as in Chapter 7 — but now `delay` is a reusable module and the pin setup is a one-line macro you can drop into any project.

13.6 Bundling modules into a library: the archiver

Once you have a handful of reusable `.o` modules, you can package them into a single **library archive** (`.a`) with the librarian, `xc8-ar` (User's Guide §7.1). The linker pulls only the modules it actually needs out of the archive.

```
xc8-ar -r mylib.a delay.o          # create mylib.a containing delay.o (-  
r = replace/add)
```

Then link your program against the library instead of the loose object:

```
pic-as -mcpu=16f17146 -mdfp=/path/to/PIC16F1xxxx_DFP/VERSION/xc8 \  
-Wl,-presetVec=0h main.o mylib.a
```

The common `xc8-ar` options (§7.1, Table 7-1): `-r` add/replace a module, `-d` delete, `-p` list modules, `-t` list with symbols, `-x` extract. One rule that bites people: **module order matters** — if module A calls a symbol defined in module B, A must come *before* B in the archive, because the linker searches in order (§7.1). For a beginner, one library of independent helper routines is plenty; the ordering rule matters once modules call each other.

13.7 What just happened

You reused code two fundamentally different ways. A **macro** (`CONFIG_OUTPUT`) is source text the assembler pastes in at each call — shared by `#include`, duplicated at every use, perfect for short patterns. A **module** (`delay.S`) is a separately-assembled routine that exists once and is `fcall`-ed — shared by `GLOBAL/EXTRN` and combined by the linker, perfect for real subroutines. You built the project both single-step and incrementally, and packaged a module into a `.a` library. That's the organizational backbone of every non-trivial assembly project.

13.8 Common mistakes

Symptom	Cause	Fix
Linker: “undefined symbol delay”	forgot to list <code>delay.S/delay.o</code> at link, or no <code>GLOBAL</code>	export with <code>GLOBAL</code> delay in <code>delay.S</code> ; include it in the build (§4.9.30)
“duplicate symbol” from a macro used twice	a label inside the macro isn’t <code>LOCAL</code>	wrap internal labels with <code>LOCAL</code> (§4.9.36)
<code>EXTRN</code> delay errors “defined in this module”	you <code>EXTRN</code> ’d a symbol you also define here	use <code>EXTRN</code> only for symbols from <i>other</i> files (§4.9.20)
Macro won’t expand / <code>&</code> appears in output	<code>&</code> misuse, or invoked a macro that wasn’t <code>#included</code>	check the <code>#include</code> ; <code>&</code> only concatenates inside a macro body (§4.9.37)
Cross-file <code>call</code> gives fixup overflow	callee in another page, plain <code>call</code> used	use <code>fcall</code> for cross-module calls (Chapter 9)
Library link can’t resolve a symbol	archive module order wrong	put referencing modules before defining ones (§7.1)

(Appendix C decodes the exact message wording.)

13.9 Try it yourself

1. **Reuse the macro.** Add a second `CONFIG_OUTPUT` `C, 2, 0` and blink `RC2` as well (external LED + resistor). One macro, two pins.
2. **Prove duplication.** Build the single-step version with `-Wa,-a`, open `main.lst`, and find where `CONFIG_OUTPUT` expanded into real instructions — see the macro become code.
3. **Break the link.** Remove `delay.S` from the build command and read the linker’s “undefined symbol” error. Restore it.
4. **Grow the library.** Write a second module `blinkio.S` exporting a `led_on/led_off` pair, archive both it and `delay.o` into `mylib.a`, and link `main.o mylib.a`. Confirm with `xc8-ar -t mylib.a` that both modules are inside.

13.10 Reference bridge

- **User's Guide §4.9.37 / §4.9.36** — `MACRO/ENDM`, arguments, & concatenation, and `LOCAL`.
- **User's Guide §4.9.30 / §4.9.20** — `GLOBAL` and `EXTRN` for cross-file symbols.
- **User's Guide §3.1-3.2** — single-step vs. multi-step (incremental) builds with `pic-as`.
- **User's Guide §7.1** — the archiver/librarian `xc8-ar` and its options.

Next chapter: every program so far has run top-to-bottom, polling in loops. Chapter 14 introduces **interrupts** on the PIC16F17146 — letting a hardware event (a timer overflow) stop your main code, run a service routine, and resume — including the crucial matter of saving and restoring context so the interruption is invisible to the code it interrupts.

Chapter 14 — Interrupts on the PIC16F17146

***What you'll build:** a blinking LED again — but this time **your main code does nothing**. A hardware timer (TMR0) overflows on its own, interrupts the CPU, and a short service routine toggles the LED. You'll learn how an interrupt hijacks the processor, why the PIC16F17146 saves your registers for you, and the one line you must never forget (clearing the flag).*

14.1 The idea: let hardware interrupt you

Every program so far has *polled* — spun in a loop asking “is it time yet?” That wastes the CPU and scales badly once you have several things to watch. An **interrupt** flips the model: you set up a hardware event, go do something else (or nothing), and when the event fires the processor **automatically stops**, jumps to a service routine, runs it, and resumes exactly where it left off — as if the interruption never happened.

On the PIC16F17146, here's precisely what the hardware does when an enabled interrupt fires while interrupts are globally on (data sheet §12.5):

1. finishes/flushes the current instruction,
2. **clears GIE** (so the ISR isn't itself interrupted),
3. pushes the current PC onto the stack,
4. **automatically saves your critical registers** to shadow registers (§12.9),
5. loads the PC with the **interrupt vector, 0x0004**.

Your Interrupt Service Routine (ISR) runs, and a **RETFIE** instruction reverses it all: pops the return address, restores the saved registers, and sets GIE back — resuming your main code untouched (§12.5).

***Remember the vectors (Chapter 4).** Reset sends the chip to **0x0000**; an interrupt sends it to **0x0004** (data sheet §9.1, §12.5). That's why we've always kept the reset code tiny and jumped away — to leave 0x0004 free for the ISR.*

14.2 Turning an interrupt on

Three switches control whether an interrupt reaches the CPU (data sheet §12.5):

- **GIE** — Global Interrupt Enable, INTCON bit 7. The master switch (§12.10.1).
- **PEIE** — Peripheral Interrupt Enable, INTCON bit 6. A second gate for *most* peripherals.
- **The specific enable bit** — e.g. TMR0IE for the Timer0 interrupt.

INTCON is a **core register** — it appears at offset 0x0B in *every* bank (Chapter 8), so you can touch GIE and PEIE without any BANKSEL.

The PEIE subtlety worth knowing

The generic rule is “GIE + PEIE + the specific enable.” But there’s an exception the data sheet is explicit about (§12.10.2, PIE0 note): the interrupt sources in the **PIE0** register — TMR0IE, IOCIE, INTE — **do not require PEIE**. Only sources in PIE1 through PIE6 need it. Since our timer’s enable (TMR0IE) lives in PIE0, we enable it with just **GIE + TMR0IE** — no PEIE needed. (Setting PEIE anyway would be harmless, but it’s good to know why we don’t have to.)

Register	Holds	Needs PEIE?
PIE0 (TMR0IE, IOCIE, INTE)	core-ish sources	No (§12.10.2)
PIE1-PIE6 (all other peripherals)	peripheral sources	Yes (§12.10.8)

14.3 The gift of automatic context saving

On older, *classic* mid-range PICs, the first thing every ISR had to do was manually copy `w` and `STATUS` into temporary variables (and restore them before returning) — because the ISR would clobber them and wreck the interrupted code. The Embedded Engineers guide (§7.3) shows that “manual context switch” dance.

You don’t need it here. The enhanced mid-range core saves and restores your context *in hardware*. On interrupt entry it automatically stashes, and on RETFIE restores, these registers (data sheet §12.9):

- WREG

- STATUS (except the TO and PD bits)
- BSR
- the FSR registers
- PCLATH

They live in **shadow registers** in Bank 63, and are readable/writable if you ever need them. The practical upshot is liberating: **inside your ISR you can use W, change banks with BANKSEL, and alter STATUS — the hardware restores the listed CPU context.** No save/restore boilerplate is needed for those registers. This does **not** restore application RAM, peripheral registers, table state, or any other temporary you modify. Save other machine state when required, and give every RAM object shared with `main` an explicit ownership/atomicity rule.

14.4 The rule you must never break: clear the flag

When TMR0 overflows it sets its flag bit, **TMR0IF** (PIR0 bit 5). That flag is what triggers the interrupt — and the data sheet says it “must be cleared by software” (§12.10.9). If your ISR doesn’t clear it, the instant you `RETFIE` and re-enable interrupts, the still-set flag fires the interrupt *again*, immediately, forever. Your program appears frozen inside the ISR.

Every ISR must acknowledge the source it handled at the data-sheet-specified point, and its flag must be clear before exit. For TMR0 the acknowledgement is simply clearing **TMR0IF**, and doing it early is appropriate:

```
BANKSEL PIR0
bcf     BANKMASK(PIR0),5      ; clear TMR0IF - or the interrupt re-fires
↪      endlessly
```

(The flag is a *request* bit and gets set by hardware regardless of the enable bits — so you also clear it once *before* enabling the interrupt, to start from a clean slate. Other peripherals can require a read, write, or ordered sequence instead of a simple `bcf`; always use that peripheral’s chapter and errata.)

One vector means you must dispatch

Every interrupt source on this device reaches the same 0x0004 vector. This example enables only TMR0, so the ISR can service it directly. Once

you enable more than one source, qualify a request with both its flag and its enable bit, dispatch all enabled pending sources deterministically, and decide what happens if several are pending together. Keep every path bounded — no busy waits or unbounded loops inside an ISR.

Interrupts also create concurrency. An 8-bit load/store is indivisible on this core, but a multi-byte value shared between `main` and the ISR is not read or written atomically as a whole. Use a brief GIE-masked critical section, a stable double-read, or a single-writer handoff protocol rather than assuming the automatic context save protects shared RAM.

14.5 Setting up TMR0 to overflow periodically

We'll run Timer0 as a free-running 16-bit counter off the system clock, so it rolls over `0xFFFF` → `0x0000` at a steady rate and sets `TMR0IF` each time (data sheet §22). Two control registers:

- **T0CON1** — clock source and prescaler. We pick `CS = 010` (**FOSC/4**), `ASYNC = 0` (synchronous), `CKPS = 0000` (**1:1** prescale). That's `0b0100_0000 = 0x40` (§22.5.2).
- **T0CON0** — enable and mode. We set `EN = 1` (on) and `MD16 = 1` (**16-bit**), postscaler `OUTPS = 0000` (1:1). That's `0b1001_0000 = 0x90` (§22.5.1).

How fast does it overflow? Our Chapter 5 config runs the chip at **1 MHz**, so $\text{FOSC}/4 = \mathbf{250\text{ kHz}}$. A 16-bit counter overflows every 65536 counts: $65536 \div 250000 \approx \mathbf{0.26\text{ s}}$. So the LED toggles about every quarter-second — a brisk $\sim 1.9\text{ Hz}$ full blink cycle. Increase `CKPS` (prescaler) to slow it down. Configure `T0CON1` and the disabled mode value in `T0CON0` first, clear the pending flag, enable the source, then write `T0CON0 = 0x90` to start the timer and set GIE last. Writing either Timer0 control register clears both prescaler and postscaler (§22.2.3–22.2.4).

14.6 The whole program — a timer-driven blink

```
; -----  
; tmr0blink.S - LED blinks from a TMR0 interrupt; main() idles  
; -----
```



```

PROCESSOR 16F17146
#include <xc.inc>
; --- paste your Chapter 5 CONFIG block here ---

; --- reset vector (0x0000) -----
PSECT resetVec,class=CODE,delta=2
resetVec:
    goto    main

; --- interrupt vector (0x0004) -----
; Positioned at 0x0004 by: -Wl,-pintVec=0004h
PSECT intVec,class=CODE,delta=2
isr:
    ; NO manual context save needed - hardware did it ($12.9)
    BANKSEL PIR0
    bcf     BANKMASK(PIR0),5      ; clear TMR0IF (MUST do this)
    BANKSEL LATC
    movlw   0x02                  ; mask for RC1 (bit 1)
    xorwf   LATC,f                ; toggle the LED
    retfie                        ; restore context + re-enable GIE

; --- main -----
PSECT code,delta=2
main:
    ; RC1 as a digital output (Chapter 7)
    BANKSEL ANSEL
    bcf     BANKMASK(ANSEL),1
    BANKSEL LATC
    bsf     LATC,1                ; preload LED off BEFORE making the pin an
    ↪ output
    BANKSEL TRISC
    bcf     TRISC,1

    ; TMR0: configure FOSC/4, 1:1 prescale, 16-bit, but leave it disabled
    BANKSEL T0CON1
    movlw   0x40
    movwf   BANKMASK(T0CON1)      ; CS=FOSC/4, ASYNC=0, CKPS=1:1
    movlw   0x10
    movwf   BANKMASK(T0CON0)      ; EN=0, MD16=1, OUTPS=1:1

    ; enable the TMR0 interrupt (PIE0 source → no PEIE needed)
    BANKSEL PIR0
    bcf     BANKMASK(PIR0),5      ; clear any pending TMR0IF
    BANKSEL PIE0
    bsf     BANKMASK(PIE0),5      ; TMR0IE = 1
    BANKSEL T0CON0
    movlw   0x90
    movwf   BANKMASK(T0CON0)      ; start TMR0 only after its flag/enable are
    ↪ ready

```

```
    bsf      INTCON,7          ; GIE = 1 last (INTCON is mirrored in every
    ↪      bank)

idle:
    goto     idle              ; main does nothing - the ISR runs the show

    END      resetVec
```

Build it, placing the ISR at the interrupt vector:

```
pic-as -mcpu=16f17146 -mdfp=/path/to/PIC16F1xxx_DFP/VERSION/xc8 \
    -Wl,-presetVec=0h -Wl,-pintVec=0004h \
    -Wa,-a -Wl,-Map=tmr0blink.map tmr0blink.S
```

On the Curiosity Nano the yellow LED blinks ~1.9 Hz while `main` sits in `idle` doing absolutely nothing — the timer and the ISR carry the whole workload. In the simulator, set a breakpoint on `xorwf LATC,f` and watch it hit every ~65536 timer counts; note that `WREG` is preserved across the interrupt even though the ISR loaded it with `0x02` (that’s automatic context saving at work).

14.7 What just happened

You inverted the program’s structure. Instead of `main` babysitting the LED, a hardware timer raises an interrupt, the CPU vectors to **0x0004**, your ISR clears the flag and toggles the pin, and `RETFIE` restores everything. You leaned on the enhanced mid-range core’s **automatic context saving** for the documented CPU registers, enabled the interrupt with just **GIE + TMR0IE** (no `PEIE`, because `TMR0` is a `PIE0` source), and acknowledged `TMR0` by clearing its flag before exit. This is the foundation of every responsive embedded program — timers, buttons, serial bytes, all serviced by interrupts while `main` does higher-level work.

14.8 Common mistakes

Symptom	Cause	Fix
Program seems frozen, LED solid	ISR didn't clear TMR0IF — it re-fires instantly	bcf BANKMASK(PIR0),5 in the ISR (§12.10.9)
Interrupt never fires	GIE or the specific enable not set	set INTCON GIE (bit 7) and PIE0 TMR0IE (bit 5) (§12.5)
A <i>PIE1</i> –6 peripheral interrupt never fires	forgot PEIE for that source	set INTCON PEIE (bit 6) — but PIE0 sources don't need it (§12.10.2)
Main code corrupted after an interrupt	ISR changed RAM/peripheral state not covered by the shadow registers	hardware restores only the §12.9 list; preserve every other shared temporary explicitly
ISR at wrong place / chip resets on interrupt	ISR psect not at 0x0004	position it with -Wl,-pintVec=0004h (§12.5)
Wrong handler runs after adding another source	ISR assumes every entry is TMR0	test each source's enable and flag; dispatch all pending enabled sources
Torn multi-byte value shared with <code>main</code>	interrupt occurred between byte accesses	use a short critical section, stable read, or single-writer handoff
Timer never overflows	wrote T0CON0 enable before configuration, or EN=0	configure while disabled; start with T0CON0 = 0x90 after flag/enable setup (§22.2.3)

(Appendix C decodes the exact message wording.)

14.9 Try it yourself

1. **Change the rate.** Set CKPS in T0CON1 to 1:16 (0b0100, so T0CON1 = 0x44) and confirm the blink slows by ~16×. Compute the new period from FOSC/4.
2. **Do work in main too.** Replace the `idle` loop with a slow software counter in common RAM and watch it keep counting *while* the ISR blinks the LED — proof the two run “at once.”
3. **Prove the flag rule.** Delete the `bcf BANKMASK(PIR0),5` from the ISR, run, and watch the program hang. Restore it. Feel why the rule is absolute.

4. **Confirm auto-save.** In the ISR, add `movlw 0x99` before `retfie`. Put a watch on `WREG`, set a breakpoint in `main` after the first interrupt, and confirm `WREG` is *not* `0x99` — the hardware restored it.
-

14.10 Reference bridge

- **Data sheet §12 “INT - Interrupts”** — the interrupt logic, `INTCON/PIE/PIR`, automatic context saving (§12.9), and the full enable sequence (§12.5).
- **Data sheet §22 “TMR0”** — Timer0 modes, clock/prescaler (`T0CON0/T0CON1`), and the overflow interrupt (§22.3.2).
- **Embedded Engineers guide §7** — the mid-range interrupt example, including the *manual* context switch you don’t need on this enhanced core (useful when you meet classic PICs).
- **PIC16F17126/46 silicon errata DS80001009E** — checked for the target revision; it lists no TMR0 or automatic-context-save anomaly. Recheck the current errata for the silicon you actually program.

Next chapter: the PIC16F17146 has a single interrupt vector and saves context for you. Chapter 15 moves *up* a core to **PIC18**, whose interrupt system differs — vectored/prioritized interrupts, an expanded register set, and its own conventions — so you can carry your interrupt skills onto the larger 8-bit family.

Chapter 15 — Moving Up a Core: PIC18 Interrupts

What you'll learn: how your hard-won PIC16F17146 skills transfer up to the current- generation **PIC18-Q** family — where the instruction set is wider, data memory is easier to reach, and interrupts are **vectored and prioritized** instead of funneled through a single address. We anchor on a modern **PIC18F57Q43** (the same VIC design used across today's Q-series parts), and every example here is built and verified.

A note on sourcing. The book's main target is the PIC16F17146. This chapter's device is the **PIC18F57Q43**; its facts below are confirmed against the PIC18F27/47/57Q43 data sheet **DS40002147H** (especially §11), silicon errata **DS80000870M**, and PIC18F-Q DFP **1.30.487**. The listings assemble without source diagnostics on pic-as v4.00. **When you build for a different PIC18, re-confirm its vectors and config bits against that device's data sheet** — the confirm-before-you-code habit from Chapter 5 never stops.

15.1 Why move up — and what stays the same

The PIC18 core is the top of the 8-bit PIC line: a **16-bit-wide instruction set**, more instructions, an expanded register set, and (on modern parts) extended data memory and a vectored interrupt controller (User's Guide §2.1). The reassuring news first: **most of the workflow you learned still applies**. Source files, psects, CONFIG, #include <xc.inc>, W, file registers, BANKSEL, labels, macros, GLOBAL/EXTRN — all identical. What changes is a handful of core-specific details. Learn those and you can read and write PIC18 assembly today.

15.2 The assembler-level differences

These are device-independent — they apply to *any* PIC18 — and you can rely on them from the XC8 guides:

Topic	Mid-range (16F17146)	PIC18
Program memory addressing	word-addressable → delta=2	byte-addressable → delta=1 (default) (User's Guide §4.9.48.4)
Code alignment	none (reloc=1)	word-aligned → reloc=2 on code psects (§4.9.48.16; Emb Eng §3.6)
Bank-free RAM	16 B common RAM	Access bank — a larger fast region via udata_acs (Emb Eng §3.5)
Move memory→memory	not available	movff / movffl — copy across ordinary PIC18 data memory, no bank select (§4.1.5)
Program paging	PAGESEL needed	PAGESEL accepted but does nothing — PIC18 handles it differently (§4.1.2)
Instruction operands	,w/,f	adds an access operand ,c/,a (access bank) vs ,b (banked) (§4.1.1)

The delta/reloc swap

This trips up everyone porting code. On mid-range you wrote `class=CODE,delta=2`. On PIC18 you write `class=CODE,reloc=2` — **because PIC18 program memory is byte-addressable** (so delta stays at its default of 1) but **instructions must be word-aligned** (so reloc=2):

```
PSECT resetVec,class=CODE,reloc=2      ; PIC18: reloc=2, NOT delta=2
resetVec:
    goto start
```

The Access bank — PIC18's "common RAM," bigger

On mid-range you dodged banking with 16 bytes of common RAM. PIC18 gives you a larger **Access bank** reached without `BANKSEL`. You place variables there with the `udata_acs` psect and use the access operand ,c (or ,a) on instructions (Emb Eng §3.5):

```
PSECT udata_acs
max: DS 1
tmp: DS 1
...
    clrf    max,c                ; ',c' = access bank, no BANKSEL needed
```

movff — copy anywhere to anywhere

The single most convenient PIC18 instruction: `movff src,dst` moves one file register to another **across ordinary PIC18 data memory, no bank selection at all** (User's Guide §4.1.5). On devices with extended data memory, the assembler can select the longer `movffl` form. The mid-range `movf/movwf` two-step through W becomes one instruction:

```
movff    PORTA,tmp            ; tmp = PORTA, W untouched, no banking
```

15.3 PIC18 interrupts: two models

Mid-range gave you exactly one interrupt vector (0x0004). PIC18 offers **two priority levels** and, on modern parts, a **vectored interrupt controller (VIC)**. Which model you get is chosen by the `MVECEN` configuration bit (data sheet §11.3):

- **Legacy mode (`MVECEN = OFF`)** — two *fixed* vectors. The data sheet is explicit: `IVTBASE` defaults to **0x000008**, so the **high-priority vector is at 0x0008** and the **low-priority vector is at 0x0018** (eight instruction words higher) (§11.3.3). You write one ISR per priority level at those addresses.
- **Vectored mode (`MVECEN = ON`)** — a **table** of vectors, **one ISR per interrupt source**. The hardware reads the table and jumps straight to the right handler, so you don't poll flags to find the source. This is the modern approach and what we use below.

In vectored mode the hardware computes each ISR address from the table (§11.3.3):

Interrupt Vector Address = IVTBASE + (2 × Vector Number).

Every interrupt source has a fixed **vector number**. On the PIC18F57Q43, **Timer0 is vector number 31** (verified from the device's interrupt-source table). Enabling priorities uses the **IPEN** bit (in `INTCON0`), and the global enable splits into **GIEH** (high) and **GIEL** (low).

15.4 The vectored interrupt controller, by example

Here is a complete, build-verified PIC18F57Q43 timer-interrupt program — the PIC18 counterpart to Chapter 14's blink. Timer0 overflows, the VIC vectors to our ISR, and the ISR toggles a pin.

```

PROCESSOR 18F57Q43
#include <xc.inc>

CONFIG "FEXTOSC = OFF"
CONFIG "RSTOSC = HFINTOSC_1MHZ"    // internal osc, 1 MHz
CONFIG "CLKOUTEN = OFF"
CONFIG "PR1WAY = ON"
CONFIG "CSWEN = ON"
CONFIG "FCMEN = ON"
CONFIG "MCLRE = EXTMCLR"
CONFIG "PWRTS = PWRT_OFF"
CONFIG "MVECEN = ON"                // vectored interrupt controller
CONFIG "IVT1WAY = ON"
CONFIG "LPBORN = OFF"
CONFIG "BOREN = SBORDIS"
CONFIG "BORV = VBOR_2P45"
CONFIG "ZCD = OFF"
CONFIG "PPS1WAY = ON"
CONFIG "STVREN = ON"
CONFIG "LVP = ON"
CONFIG "XINST = OFF"                // XC8 requires the extended set off
CONFIG "WDTCPSS = WDTCPSS_31"
CONFIG "WDTE = OFF"
CONFIG "WDTCCS = WDTCCS_7"
CONFIG "WDTCCS = SC"
CONFIG "BBSIZE = BBSIZE_512"
CONFIG "BBEN = OFF"
CONFIG "SAFEN = OFF"
CONFIG "DEBUG = OFF"
CONFIG "WRTB = OFF"
CONFIG "WRTC = OFF"
CONFIG "WRTD = OFF"
CONFIG "WRTSAF = OFF"
CONFIG "WRTAPP = OFF"
CONFIG "CP = OFF"
GLOBAL __Livt

PSECT resetVec,class=CODE,reloc=2
resetVec:
    goto    start

```



```

; Timer0 is interrupt vector number 31 on PIC18F57Q43.
PSECT ivt,class=CODE,reloc=2,ovrld
ivtbase:
    ORG     31*2
    DW      tmr0Isr shr 2

PSECT textISR,class=CODE,reloc=4
tmr0Isr:
    bcf     BANKMASK(PIR3),PIR3_TMR0IF_POSN,c
    movlw   1
    xorwf   BANKMASK(LATA),f,c
    retfie  f

PSECT code
start:
    ; IVTLOCK/IVTBASE are in bank 4, not the Q43 Access Bank.
    bcf     BANKMASK(INTCON0),INTCON0_GIE_POSN,c
    BANKSEL IVTLOCK
    movlw   0x55
    movwf   BANKMASK(IVTLOCK),b
    movlw   0xAA
    movwf   BANKMASK(IVTLOCK),b
    bcf     BANKMASK(IVTLOCK),IVTLOCK_IVTLOCKED_POSN,b
    movlw   low highword __Livr
    movwf   BANKMASK(IVTBASEU),b
    movlw   high __Livr
    movwf   BANKMASK(IVTBASEH),b
    movlw   low __Livr
    movwf   BANKMASK(IVTBASEL),b
    movlw   0x55
    movwf   BANKMASK(IVTLOCK),b
    movlw   0xAA
    movwf   BANKMASK(IVTLOCK),b
    bsf     BANKMASK(IVTLOCK),IVTLOCK_IVTLOCKED_POSN,b

    BANKSEL ANSELA                ; RA0 as a digital output
    clrf    BANKMASK(ANSELA),b
    clrf    BANKMASK(LATA),c
    clrf    BANKMASK(TRISA),c

    ; Timer0 registers are in bank 3.
    BANKSEL T0CON0
    clrf    BANKMASK(T0CON0),b
    bsf     BANKMASK(T0CON0),T0CON0_MD16_POSN,b
    movlw   0x40
    movwf   BANKMASK(T0CON1),b
    bsf     BANKMASK(IPR3),IPR3_TMR0IP_POSN,b
    bcf     BANKMASK(PIR3),PIR3_TMR0IF_POSN,c
    bsf     BANKMASK(PIE3),PIE3_TMR0IE_POSN,c
    bsf     BANKMASK(T0CON0),T0CON0_EN_POSN,b
    bsf     BANKMASK(INTCON0),INTCON0_IPEN_POSN,c

```

```

    bsf      BANKMASK(INTCON0),INTCON0_GIEH_POSN,c
idle:
    goto     idle
    END      resetVec

```

Build it, linking the vector table at the legacy base 0x8:

```

pic-as -mcpu=18F57Q43 -mdfp=/path/to/Microchip.PIC18F-Q_DFP/1.30.487/xc8 \
-Wl,-presetVec=0h -Wl,-pivt=08h -Wa,-a -Wl,-Map=vic.map vic.S

```

Five PIC18-specific ideas in that program:

1. **A vector table, not a fixed address.** Each source has a slot; Timer0's is #31. `ORG 31*2` places the entry at the right offset *within the table* (each vector is 2 bytes). This is the one place `ORG` is genuinely recommended — and it's still psect-relative, exactly as Chapter 10 warned.
2. **DW tmr0Isr shr 2.** The table stores the ISR address **shifted right by 2**, because the hardware forms the jump target by shifting left — which is *why* the ISR sits on a 4-byte boundary (`reloc=4`).
3. **retfie f — the fast return.** The Q43 saves WREG, STATUS, and BSR on its fast stack; the `f` operand restores them on the way out (data sheet §9.1.3.4; User's Guide §4.1.6). Protect any additional registers or shared multi-byte state that an ISR changes; the fast return is not a blanket save of every peripheral or file register.
4. **IVTBASE locates the table.** The hardware finds the table via the IVTBASE registers, loaded through the device-defined `0x55/0xAA` unlock sequence, using the linker symbol `__Livt` (our table's address). The table is *relocatable*; you could keep several and switch them at runtime (data sheet §11.3.1).
5. **ovrld on the table psect.** Vector entries from different files overlay into one table at the right offsets regardless of assembly order — that's how a big project adds ISRs incrementally.

Verified. This program assembles on `pic-as v4.00` with the PIC18F-Q DFP, and the map file shows the `ivt` psect linked at **address 0x8** with length **0x40** (`__Livt = 0x8`) — exactly the table layout the vector-number math predicts ($31 \times 2 + 2 = 0x40$).

The explicit `BANKSEL/BANKMASK` operands matter on the Q43. Its Access Bank maps physical GPR `0x500-0x55F` and SFR `0x460-0x4FF`; `IVTLOCK/IVTBASE` (`0x459-0x45F`) and `Timer0` (`0x31A-0x31B`) are outside it (data sheet §9.4.2 and DFP 1.30.487). Marking those registers `,c` would address the wrong Access Bank location or trigger a fixup diagnostic.

15.5 What actually changed, at a glance

Coming from Chapter 14, here’s the whole delta:

	16F17146 (mid-range)	PIC18F57Q43 (PIC18)
Interrupt vector	single, fixed at 0x0004	vector table (MVECEN=ON), or dual fixed vectors 0x08/0x18 (MVECEN=OFF)
Priorities	none	high / low (IPEN, GIEH/GIEL)
Find the source	poll flag bits in the ISR	hardware vectors straight to the ISR
Context save	automatic (§12.9)	Q43 fast stack saves WREG/STATUS/BSR; retfie f restores them
Acknowledge source	follow the peripheral rule	TMR0IF must be cleared in software
Code psect flags	class=CODE,delta=2	class=CODE,reloc=2
Escape banking	common RAM (udata_shr)	Access bank (udata_acs) + movff

Every ISR must acknowledge its source exactly as that peripheral specifies. For this example, that means clearing PIR3.TMR0IF in software; do not generalize that operation to flags that are hardware-cleared or read-only on another peripheral.

15.6 What just happened

Moving to a modern PIC18 keeps 90% of what you know and swaps a few core-specific details: a byte-addressable program memory (reloc=2, not delta=2), a roomier Access bank plus the whole-memory movff, and — the headline — a **vectored, prioritized interrupt system** where a table of ISR addresses (found via IVTBASE, using **Address = IVTBASE + 2×vector#**) replaces the single 0x0004 vector. The habits stay identical: keep the reset code tiny, let the hardware save context, and acknowledge each interrupt source exactly as its peripheral requires.

15.7 Common mistakes (porting from mid-range)

Symptom	Cause	Fix
Code psect won't align / build error on PIC18	used <code>delta=2</code> (a mid-range habit)	PIC18 code uses <code>class=CODE, reloc=2</code> (§4.9.48)
ISR runs from the wrong address	ISR psect not <code>reloc=4</code> , or forgot <code>shr 2</code> in the table	ISR on a 4-byte boundary; store <code>DW isr shr 2</code>
Interrupts never vector correctly	IVTBASE not loaded / unlock sequence skipped	load IVTBASE from <code>__Lvt</code> via the <code>0x55/0xAA</code> unlock (§11.3.1)
Build error mentioning the extended instruction set	<code>XINST</code> left on	set <code>CONFIG "XINST = OFF"</code> (XC8 requires it off)
Used the wrong vector number	assumed a source's IRQ number	confirm it in your device's interrupt-source table (Timer0 = 31 on the Q43)
Interrupt re-fires forever	TMR0 flag was not cleared	clear <code>PIR3.TMR0IF</code> ; use each peripheral's documented acknowledge rule

15.8 Try it yourself

1. **Diff the two blinks.** Put Chapter 14's `tmr0blink.S` beside the Q43 program above and list every line that differs. Sort each difference into "assembler syntax" vs. "hardware/interrupt model."
2. **Read the map.** Build the example and open `vic.map`; find the `ivt` psect at link address **0x8** and confirm its length is **0x40** (the gap the `ORG 31*2` created plus the 2-byte entry).
3. **Add a second vector.** Add a second source's ISR at its own table slot (another `ORG n*2 / DW isr shr 2` in the `ovrld ivt` psect, using that source's vector number) and confirm both vectors coexist in the map.
4. **Legacy mode.** Set `MVECEN = OFF`, put an ISR at `0x0008`, and confirm from the data sheet (§11.3.3) why that address is the high-priority vector.

15.9 Reference bridge

- **PIC18F57Q43 data sheet §11 “Vectored Interrupt Controller”** — IVTBASE, the address calculation (§11.3.3), priorities, and the full interrupt-source/vector-number table.
- **PIC18F27/47/57Q43 silicon errata DS80000870M** — revision-specific restrictions to recheck before hardware deployment; revision M adds no VIC/TMR0 restriction to this example.
- **Embedded Engineers guide §3 / §8** — Microchip’s worked PIC18 assembly examples (Access bank, movff, reloc=2, the IVTBASE unlock).
- **User’s Guide §4.1** — PIC18 instruction deviations: access operands, movff/movffl, retfie f, and why PAGESEL is a no-op.

Next chapter: interrupts introduced the idea of routines with their own private state. Chapter 16 tackles **the compiled stack** — how assembly on PIC allocates local variables for subroutines (there’s no hardware data stack), why the linker overlays them, and how to declare a routine’s locals so several functions can safely share memory.

Chapter 16 — The Compiled Stack

***What you'll learn:** how assembly routines get **local variables** — the assembly equivalent of a C function's `auto` and parameter variables — even though a PIC has no hardware data stack. You'll meet the **compiled stack**: memory the linker hands out and cleverly **reuses** across routines that never run at the same time. This is how large assembly programs keep their RAM footprint small without dangerous manual sharing.*

16.1 The idea: locals without a data stack

In C, every function gets private locals for free — `int x`; inside a function exists only while that function runs, and its memory is recycled for the next call. Under the hood, C on a big CPU uses a **hardware data stack**: push locals on entry, pop on exit.

The PIC has a stack, but it's a *return-address* stack only — it holds PC values for `CALL/ RETURN` (Chapter 9's §9.5), not your variables. There's no push/pop for data. So how does an assembly routine get recyclable locals?

The answer is the **compiled stack** — and the name is slightly misleading, so read carefully (User's Guide §5.5; Embedded Engineers guide §6):

*“Not to be confused with a software stack... a compiled stack is a memory area designated for the **static** allocation of local objects that should only consume memory for the duration of the routine with which they are associated.”*

There's **no stack pointer and no push/pop**. Each routine's locals get *fixed* addresses, assigned by the **linker**. The magic is that the linker can give **two routines the same addresses** for their locals — *provided they're never active at the same time*. Memory that a helper routine uses is quietly reused by another helper that never overlaps with it.

Why bother? Big programs have dozens of routines each needing a few local bytes. Give each its own permanent RAM and you run out fast. The compiled stack lets non-overlapping routines share the same

bytes — big RAM savings, and because it’s static there’s **no speed or code-size penalty** (no pointer math) (Emb Eng §6).

***The one hard rule: not reentrant.** Because the addresses are static, a routine using compiled-stack locals **cannot be recursive**. As a safe assembly convention, do not place the same compiled-stack routine in both a main-line and interrupt call graph: an interrupt could preempt an active instance and corrupt its fixed locals. Use a separate root and separate routines/state for each asynchronous call graph (Emb Eng §6; User’s Guide §5.5).*

16.2 The tools: the FN directives

The linker can only reuse memory safely if it knows **who calls whom** — the program’s *call graph*. You describe that graph with four directives, all starting FN (Emb Eng §6.1; User’s Guide §5.5):

Directive	Used	Says
FNCONF psect,?au_,?pa_	once per program	put the stack in this psect; prefix auto symbols with ?au_, parameters with ?pa_
FNROOT routine	once per call-graph root	this routine is the top of a call tree (typically main, and one per ISR)
FNSIZE routine,autos,params	per routine with locals	this routine needs <i>autos</i> bytes of locals and <i>params</i> bytes of parameters
FNCALL caller,callee	per unique call	<i>caller</i> calls <i>callee</i> (an edge in the graph)

From these, the linker builds the call graph, creates a symbol for each routine’s local block (?au_<routine> for autos, ?pa_<routine> for parameters), assigns addresses, and **overlaps blocks wherever the graph proves it’s safe** (Emb Eng §6.2).

16.3 A worked example

Here's the heart of Microchip's compiled-stack example (Emb Eng §6) — a main that repeatedly calls `add(a,b)` and `incr(val,amount)`. It's PIC18 (note `movff, ,c` access operands from Chapter 15) — built and verified here on a **PIC18F57Q43** — but the *stack concepts* are identical on every core.

```

;--- configure the stack: hold it in udata_acs; prefixes for autos/params
FNCONF udata_acs,?au_,?pa_

;--- add: needs 4 bytes of parameters, no autos -----
FNSIZE  add,0,4                ; two 2-byte 'int' parameters
GLOBAL  ?pa_add                ; linker-made symbol for add's parameter block
add:
    movf   BANKMASK(?pa_add+2),w,c
    addwf  BANKMASK(?pa_add+0),f,c
    movf   BANKMASK(?pa_add+3),w,c
    addwfc BANKMASK(?pa_add+1),f,c
    return                                ; result left in the parameter memory

;--- main: the call-graph root, needs 4 bytes of autos -----
GLOBAL  ?au_main
result  EQU  ?au_main+0        ; readable aliases for main's autos
incval  EQU  ?au_main+2
FNROOT  main                  ; main is the top of a call graph
FNSIZE  main,4,0              ; 4 bytes of autos (result + incval), no params
FNCALL  main,add               ; main calls add
FNCALL  main,incr              ; main calls incr
main:
    clrf   BANKMASK(result+0),c
    clrf   BANKMASK(result+1),c
    ...
    call   add                  ; (parameters loaded into ?pa_add first)
    ...

```

Read the pattern:

- **FNSIZE add,0,4** reserves 4 bytes for `add`'s parameters; the linker names that block `?pa_add`, and you reach each byte as `?pa_add+0`, `?pa_add+1`, ... (Emb Eng §6.1).
- **FNSIZE main,4,0** gives `main` 4 bytes of autos, block `?au_main`. The `EQU` lines just make `result/incval` friendlier names for `?au_main+0` and `?au_main+2`.
- **FNCALL main,add** / **FNCALL main,incr** tell the linker `main`'s call edges — *the* information that lets it decide what can overlap.
- **Return values** live in the parameter memory: a routine writes its result where its parameters were. So size a routine's params as the *larger* of its parameters and its return value (Emb Eng §6.1).

You invent the calling convention yourself — here, the first `int` parameter is `?pa_add+0/+1` (low/high byte), the second is `?pa_add+2/+3`. Nothing enforces this but your own consistency. The linker-defined symbols contain full physical addresses (for this Q43 example, in `0x500–0x55F`), so byte-oriented instructions use `BANKMASK( ... ),c`. In contrast, `movff` operands use the full addresses and must not be masked (User's Guide §4.1.1/§4.1.5).

Build the verified Q43 source with the matching pack and explicitly place the stack psect:

```
pic-as -mcpu=18F57Q43 -mdfp=/path/to/Microchip.PIC18F-Q_DFP/1.30.487/xc8 \
-Wl,-presetVec=0h -Wl,-pudata_acs=COMRAM -mcallgraph=full \
-Wa,-a -Wl,-Map=cstack.map cstack.S
```

16.4 Seeing the overlap in the map file

Build with a call graph (`-mcallgraph=full`) and the map file shows the linker's work (Emb Eng §6.2):

```
Call graph: (fully expanded)
*main size 0,4 offset 0
*   add size 4,0 offset 4
    incr size 2,0 offset 4
```

This tiny listing is the whole point of the chapter:

- **Indentation = call depth:** `main` calls `add` and `incr`.
- **The offset for `add` and `incr` is the same (4).** That means **they share the same memory** — the linker proved it's safe because `add` and `incr` never call each other, so they're never active simultaneously.
- **A *** marks a *critical-path* node — memory at a unique location that adds to total RAM. Un-starred routines' blocks fully overlap others and cost **zero** extra RAM.

The payoff, in Microchip's words (Emb Eng §6.2): the program needed **10 bytes** of locals but the linker allocated only **8** — 2 bytes reused — “without the programmer having to employ the dangerous practice of sharing objects between routines.” In the verified Q43 map, the call-graph offsets are 0 and 4, while the corresponding physical symbols are `?au_main = 0x500` and `?pa_add = ?pa_incr = 0x504` because `udata_acs`

was placed in the Q43 COMRAM class. No special build option turns stack analysis on: “a compiled stack is automatically created if FN-type directives are detected” (Emb Eng §6.3).

16.5 On the PIC16F17146

The example above is PIC18, which conveniently has a roomy Access bank for the stack. Our mid-range target differs in two ways (Emb Eng §6.1):

- **Where the stack lives.** Mid-range parts have only 16 bytes of common RAM, usually needed elsewhere, so you put the compiled stack in **banked GPR** — e.g. `FNCONF udata,?au_,?pa_` — and access its objects with the `BANKSEL/BANKMASK` you learned in Chapter 8.
- **How you touch it.** No `movff` on mid-range (Chapter 8), so you move locals through `W` with `movf/movwf`, selecting the stack’s bank first.

Everything else — `FNCONF/FNROOT/FNSIZE/FNCALL`, the `?au_/?pa_` symbols, the call-graph overlap — works identically. And remember the reentrancy rule from §16.1: give each **interrupt** routine its **own FNROOT**, so its locals never share bytes with main-line routines that an interrupt could preempt (User’s Guide §5.5).

16.6 What just happened

You gave assembly routines proper local variables. The **compiled stack** is a linker-managed, statically-addressed region where each routine’s locals live only “logically” for its duration — and the linker **overlaps the locals of routines that never run together**, cutting RAM use with no runtime cost. You drive it with four `FN` directives that describe your call graph, read the `?au_/?pa_` symbols the linker creates, and verify the overlap in the map file’s call graph. The price of admission is discipline: keep the `FN` directives accurate, and never share a compiled-stack routine between main and an interrupt.

16.7 Common mistakes

Symptom	Cause	Fix
Locals of two routines corrupt each other	a missing FNCALL hid a real call, so the linker wrongly overlapped them	add an FNCALL for every unique call (Emb Eng §6.2)
“undefined symbol ?pa_add”	forgot to GLOBAL the linker-created stack symbol	GLOBAL ?pa_add in each module that uses it (Emb Eng §6.1)
Data corrupted only when an interrupt fires	a routine is called from both main and the ISR	give the ISR its own FNROOT; don’t share compiled-stack routines (§5.5)
Return value clobbered	params sized smaller than the return value	size params as the larger of parameters and return value (Emb Eng §6.1)
No overlap happening / RAM higher than expected	FNROOT/FNSIZE/FNCALL missing or wrong	check the call graph in the map file against your real call structure
Warning about the FNCONF psect	stack psect not placed	add -Wl,-p<psect>=<class> (e.g. -Wl,-pudata_acs=COMRAM) (Emb Eng §6.3)

(Appendix C decodes the exact message wording.)

16.8 Try it yourself

1. **Find the overlap.** Build Microchip’s example with the exact command in §16.3, open the map, and confirm add and incr share offset 4 while main sits at offset 0.
2. **Break the graph.** Delete FNCALL main,incr, rebuild, and study how the call graph and the overlap change — then explain why that omission would be dangerous in real code.
3. **Add a third helper.** Write a dbl routine (needs 2 param bytes) that main also calls, add its FNSIZE/FNCALL, and check whether the linker overlaps it with add/incr or gives it its own slot. Explain the result from the call graph.

4. **Port the idea to mid-range.** Sketch (on paper) the `FNCONF` `udata, ...` version for the PIC16F17146: which directive changes, and where would `BANKSEL/BANKMASK` appear when reading `?pa_add`?
-

16.9 Reference bridge

- **Embedded Engineers guide §6** — the full compiled-stack example, the `FN` directives, and the annotated call graph in the map file.
- **User's Guide §5.5** — a summary of compiled stacks and the `FN` directives.
- **User's Guide §4.9.24-4.9.29** — the individual `FNCALL/FNCONF/FNROOT/FNSIZE` directive references (plus `FNARG`, `FNINDIR` for advanced call graphs).

Next chapter (Part V): you've now generated map files a dozen times and peeked at call graphs and symbol tables. Chapter 17 finally reads the **map file** properly — memory ranges, psect placement, linker classes, and the linker-defined symbols — so you can see exactly where every byte of your program landed and diagnose placement problems yourself.

PART V

Toolchain Mastery and Going Further

Chapter 17 — The Linker & Map Files

***What you'll build:** nothing new to run — instead you'll finally read the **map file** the linker has been quietly producing since Chapter 4. You'll open your blink's map, find where every psect and symbol landed, spot the free memory that's left, and learn to diagnose “won't fit / can't place” errors yourself. The map file is the linker showing its work; this chapter teaches you to read it.*

17.1 The idea: the linker places, the map reports

Recall the pipeline (Chapter 10): the assembler turns each source file into an object file full of **relocatable psects**; the **linker** then gathers all the psects, drops each into a memory range (its **class**), assigns concrete addresses, resolves every symbol, and emits the final HEX. The **map file** is the linker's report of exactly where everything ended up — “information relating to the memory allocation of psects and the addresses assigned to symbols within those psects” (User's Guide §6.3).

When a build succeeds, the map tells you how much memory you used and where. When it *fails* with a placement error, the map is the first place you look. Either way, it's the ground truth about your program's memory — and after this chapter it won't be mysterious.

17.2 Getting a map file

You've been generating one all along with `-Wl,-Map`:

```
pic-as -mcpu=16f17146 -mdfp=/path/to/Microchip.PIC16F1xxxx_DFP/1.31.465/xc8 \
-Wl,-presetVec=0h -Wl,-Map=blink.map blink.S
```

In MPLAB X a map file is produced **by default** (User's Guide §6.3.1). Two useful facts from that section:

- The map is written **by the linker**, so if the build stops before linking (e.g. an assembly error), you get no map.
 - A map **is** produced even when the linker reports errors — a partial map that often points straight at the cause of a “can’t place” failure. (Only a *fatal* linker abort suppresses it.)
-

17.3 What’s in a map file

The sections appear in this order (User’s Guide §6.3.2):

1. Assembler name & version
2. The linker command line
3. Object-code version and **machine type**
4. **Psect summary by module** (which file contributed which psect)
5. **Psect summary by class** ← *where each psect landed*
6. Segment summary (*ignore this one*)
7. **Unused address ranges** ← *free memory*
8. **Symbol table** ← *where each global symbol is*
9. Per-function and per-module info

We’ll focus on the four that matter to a beginner (4, 5, 7, 8). First, a sanity habit: **check the version and machine type at the top**. “Always confirm the assembler version number... to ensure you are using the assembler you intended,” and that Machine type reads 16F17146 (§6.3.2.1). A surprising number of “impossible” bugs are really “built for the wrong chip.”

17.4 The psect listing — and the columns that matter

The heart of the map is the psect listing, under this header (User’s Guide §6.3.2.2):

Name	Link	Load	Length	Selector	Space	Scale
------	------	------	--------	----------	-------	-------

Here’s Microchip’s example (§6.3.2.2), which teaches every column at once:

Name	Link	Load	Length	Selector	Space	Scale	
ext.o	text	3A	3A	22	30	0	
	bss	4B	4B	10	4B	1	
	rbit	50	A	2	0	1	8

Decoding the columns:

- **Link** — the address the psect is *accessed at* at run time. This is the one you usually care about. `text` is linked at `0x3A`.
- **Load** — where it sits in the output HEX; usually equal to Link (§6.3.2.2).
- **Length** — size, in the psect’s own units.
- **Space** — the memory space: **0 = program memory**, **1 = data memory** (§6.3.2.2). *This is the column that resolves apparent overlaps.*
- **Scale** — address units per byte; blank when 1, **8 for bit psects** (§6.3.2.2).
- **Selector** — ignore it on PIC devices.

***The “overlap” that isn’t — why Space matters.** In the example, `text` is at `0x3A` and `bss` at `0x4B` — but glance again: `text` has **Space 0** (program memory) and `bss` has **Space 1** (data memory). They’re in different memory spaces (Harvard architecture, Chapter 1), so program address `0x3A` and data address `0x4B` have nothing to do with each other. **Never compare two addresses without checking their Space.** This single habit prevents a whole category of map-reading mistakes. (The `rbit` psect’s Scale of 8 means its addresses are in bit units — another reason two numbers might look like a clash when they aren’t.)*

The same psects are then re-listed **grouped by class** (TOTAL Name Link Load Length, §6.3.2.3) — same data, sorted by `CODE`, `BANK0`, `COMMON`, etc. This is often the easiest view for “where did my code psect go?”

17.5 Unused ranges and the symbol table

Free memory (§6.3.2.5)

After the psect listings comes **UNUSED ADDRESS RANGES** — the memory still available in each class. When you hit a “can’t find space” / “won’t fit” error, **this is the section to study**. Pay attention to the **Largest block** column: it shows the biggest *contiguous* free chunk (taking paging boundaries into account), which is what actually determines whether a psect

fits (§6.3.2.5). A class can show plenty of total free bytes yet still fail to place a psect if that free space is fragmented into pieces all smaller than the psect.

Where your symbols went (§6.3.2.6)

The **Symbol Table** alphabetically lists the program's **global** symbols — global labels, global EQU/SET values, and linker-defined symbols — each with the psect it's in and its value (usually an address). Two things to remember (§6.3.2.6):

- **Only GLOBAL symbols appear.** A plain local label won't be here (Chapter 4). If you want to find a symbol in the map, declare it GLOBAL.
- A psect shown as (**abs**) means the symbol isn't tied to a psect — typical of EQU-defined constants (they're just values, not memory).

(The assembler *list* file, by contrast, shows *local* symbols too, but only for its own module — a useful complement, §6.3.2.6.)

17.6 Hands-on: read your blink's map

Rebuild Chapter 7's blink asking for a map, then open blink.map:

```
pic-as -mcpu=16f17146 -mdfp=/path/to/Microchip.PIC16F1xxxx_DFP/1.31.465/xc8 \  
-Wl,-presetVec=0h -Wl,-Map=blink.map blink.S
```

Walk it top to bottom and find:

1. **Header** — confirm Machine type is 16F17146 and the version is the toolchain you meant.
2. **resetVec** — in the psect-by-class listing, under class CODE, confirm its **Link address is 0x0000** (that's what -Wl,-presetVec=0h forced). This is the reset vector the chip jumps to.
3. **The code psect** — find where the linker placed your main/loop instructions in the CODE class (an address it chose automatically). Note its **Length** — that's your program size in words.
4. **Your variables** — if you used udata_shr counters (from other chapters), find them in a data class with **Space 1**. Confirm they're in data memory, not program.
5. **UNUSED ADDRESS RANGES** — see how much program flash and RAM you have left. For a blink, almost all of it.

6. **Symbol table** — remember most of the blink’s labels are local, so they *won’t* appear. Add `GLOBAL main` to the source, rebuild, and watch `main` show up with its address. That’s the direct way to make a symbol visible in the map.

Everything the linker did to your program is right there, in plain text.

17.7 Diagnosing placement problems

The map turns cryptic linker errors into findable facts:

- **“can’t find space for psect X”** → open `UNUSED ADDRESS RANGES`, look at the `Largest block` for X’s class. If the largest free block is smaller than X, X won’t fit even if total free looks ample — the space is fragmented (§6.3.2.5).
 - **“a symbol is at the wrong address”** → find it in the Symbol Table and check its psect; then check where that psect was placed in the psect-by-class listing.
 - **“two things seem to collide”** → compare their **Space** first. Different Space = no collision (§6.3.2.2).
 - **“built, but behaves like the wrong chip”** → check `Machine type` at the top (§6.3.2.1).
-

17.8 What just happened

The map file stopped being noise. You know its running order, you can read the psect listing’s **Link/Load/Length/Space/Scale** columns — especially **Space (0 = program, 1 = data)**, which keeps you from mistaking cross-space addresses for overlaps — and you can find free memory in **Unused Address Ranges** and any global symbol in the **Symbol Table**. Most importantly, when the linker complains it “can’t place” something, you now know exactly where to look.

17.9 Common mistakes

Symptom	Cause	Fix
No map file produced	build failed before the linker ran (assembly error)	fix the assembly error; the map is a <i>linker</i> product (§6.3.1)
Two addresses look like they overlap	compared them without checking Space	different Space = different memory; compare within a space (§6.3.2.2)
A symbol you expected isn't in the map	it's a local label	declare it GLOBAL to list it (§6.3.2.6)
"can't find space" despite free memory	the free space is fragmented	check Largest block, not total free (§6.3.2.5)
A bit-object address looks wrong/huge	its Scale is 8 (bit units)	read the Load column (byte-converted) for bit psects (§6.3.2.2)
Wrong-chip behavior	built for the wrong device	verify Machine type at the map's top (§6.3.2.1)

(Appendix C decodes the exact message wording.)

17.10 Try it yourself

1. **Map the blink.** Build `blink.S` with `-Wl,-Map=blink.map`, and write down `resetVec`'s Link address, the code psect's address and Length, and the largest free block of program memory.
2. **Make a symbol appear.** Add `GLOBAL main`, rebuild, and find `main` in the Symbol Table. Note its address and confirm it matches the code psect's placement.
3. **Force a fit failure.** Add a DS-reserved buffer far larger than a bank in a `udata` psect, build, and read the "can't find space" error — then find in `UNUSED ADDRESS RANGES` why the largest block was too small.
4. **Prove the Space rule.** Find a program-memory psect (Space 0) and a data psect (Space 1) with numerically close addresses, and explain in one sentence why they don't conflict.

17.11 Reference bridge

- **User's Guide §6.3 "Map Files"** — every section of the map, in order, with the column definitions.

- **User's Guide §6.1-6.2** — what the linker does: operation, psects, and relocation.
- **User's Guide §5.3-5.4** — the linker classes psects are placed into, and linker-defined symbols (the `__L/__H` names you saw in Chapter 15's `IVTBASE` code).

Next chapter: you can see where your program lives in memory; now let's get it *onto the chip*. Chapter 18 covers the **HEX file** and the **Hexmate** utility — merging, checksums, filling unused memory — and then actually programming the PIC16F17146 so your blink runs on real silicon.

Chapter 18 — Utilities, Hex Files & Programming the Chip

What you'll build: the finish line. You'll take the timer-blink from Chapter 14, understand the **HEX file** the build produces, meet the **Hexmate** utility, and then actually **program the PIC16F17146** so your assembly runs on real silicon and blinks a real LED. Everything so far has led here: code on a chip.

18.1 The idea: the HEX file is the deliverable

Every build you’ve run produced a `.hex` file. That file — not the `.o`, not the `.map` — is what gets burned into the chip. It’s the portable, tool-independent image of your program’s memory. Understanding it demystifies the last step from “it assembled” to “it runs.”

The format is **Intel HEX** — plain ASCII text, a series of *records*, one per line, each beginning with a colon (User’s Guide §7.2.2). Open any .hex in a text editor and you’ll see lines like:

```
:04000000FEEFFF020
```

That's not random. Every record has a fixed structure (§7.2.2):

```

:  LL      0000      TT      DD...DD      CC
|  |        |        |        |        |
|  |        |        |        |        |  Checksum (1 byte)
|  |        |        |        |        |  Data / Argument (LL bytes)
|  |        |        |        |        |
|  |        |        |        |        |  Record Type
|  |        |        |        |        |
|  |        |        |        |        |  Address Offset (2 bytes)
|  |        |        |        |        |
|  |        |        |        |        |  Data Length (1 byte)
|  |        |        |        |        |
|  |        |        |        |        |  Record Mark (colon)

```

Decoding the example above: 04 bytes of data, at address offset 0000, record type 00 (a data record), the four data bytes FE EF FF F0, and checksum 20. The **checksum** is the 8-bit two's complement of the sum

of every byte in the record — sum $0 \times 04 + 00 + 00 + 00 + FE + EF + FF + F0 = 0 \times 3E0$, and two's complement of the low byte gives 0×20 (§7.2.2). It exists so a programmer can catch a corrupted line before writing it to your chip.

There are six record types; the two you'll always see are **00 (Data)** and **01 (End-of-file)** (§7.2.2). Types **04/02** extend the address range for large devices. A HEX file using only types 0 and 1 is Microchip's **INHX8M** format; add type 4 records and it's **INHX32** (§7.2.2.1).

18.2 The gotcha: HEX addresses are byte addresses

Here's the fact that confuses everyone reading a mid-range HEX file (User's Guide §7.2.2):

"HEX files always use byte addresses... Some devices use word addressing, where each unique device address might contain more than one byte of data."

The PIC16F17146 is **word-addressed** — like all baseline, mid-range, and enhanced mid-range PICs, its program memory uses **2 bytes per address** (§7.2.2 table). But the HEX file addresses *bytes*. So **a device program-memory address does not equal its HEX-file address** — the HEX address is (roughly) *twice* the device word address. If you go hunting in a .hex for the code at device address $0x0004$ (the interrupt vector), you'll find it near HEX offset $0x0008$. This is the same delta/word-vs-byte theme from Chapter 10, now visible in the output file. (Hexmate's -addressing option can do the mapping for you; §7.2.2.)

18.3 Hexmate: massaging the HEX file

Hexmate is a post-link utility that manipulates Intel HEX files. The driver runs it automatically at the end of every build, but you can also call it standalone (xc8-ar's sibling) for jobs like (User's Guide §7.2.1):

- **Merging** several HEX files into one — e.g. dropping a bootloader in with your application.
- **Checksums / CRC** — computing a hash over your code so the running program can verify it hasn't been corrupted.

- **Filling unused memory** — putting a known value (or a “trap” instruction) in every empty location, so a crashed program that wanders into blank flash does something predictable.
- **Format conversion** — INHX8M ↔ INHX32, standardizing record lengths.
- **Finding/validating** — checking record checksums, mapping addresses.

You rarely need Hexmate by hand as a beginner — but knowing it exists explains how bootloaders, serial numbers, and code-integrity checksums get *into* a HEX file after the linker is done.

18.4 Programming the PIC16F17146 Curiosity Nano

Now the moment. The Curiosity Nano has an **on-board debugger** that programs the target over USB — no separate programmer needed (CNANO guide §3.1). There are two ways to flash your .hex.

Method 1 — from your IDE (the normal way)

1. Plug the board in over USB; the on-board debugger enumerates automatically.
2. **VS Code:** run **Debug: Add Configuration → MPLAB Debugger** (Chapter 3), select the connected Curiosity Nano as the tool, and start a session — programming happens as the session launches. (*MPLAB X IDE: press **Make and Program Device**.*)
3. The tool assembles, links, and writes the HEX to the chip.

During programming, the board’s **Power/Status (PS) LED** “blinks slowly during programming/debugging” (CNANO §3.1). When it settles, your code is running.

Method 2 — Drag-and-drop (no IDE needed)

The on-board debugger also appears as a **USB mass-storage drive** (CNANO §3.1). To program:

1. Build your project to a .hex (command line or IDE).
2. **Drag the .hex file onto the Curiosity Nano drive.**
3. Watch the PS LED: it “**blinks slowly for 2 sec**” on success, or “**blinks rapidly for 2 sec**” on failure (CNANO §3.1, Table 3-1).

Drag-and-drop is a delightfully simple way to flash a board from any machine, no toolchain installed — great for demos and sharing.

18.5 Run your blink on real silicon

Let's ship the Chapter 14 timer-blink. Build it to a HEX:

```
pic-as -mcpu=16f17146 -mdfp=/path/to/Microchip.PIC16F1xxx_DFP/1.31.465/xc8 \
-Wl,-presetVec=0h -Wl,-pintVec=0004h -Wl,-Map=tmr0blink.map tmr0blink.S
```

This produces `tmr0blink.hex`. Program it by either method above. On success, the **yellow user LED (RC1)** blinks at ~2 Hz — driven entirely by the TMR0 interrupt, on a physical chip, from assembly *you* wrote and now fully understand, byte by byte from the reset vector up.

Take a moment. That blink is the whole book working at once: config bits bring the chip up, the reset vector at 0x0000 jumps to your code, psects placed everything in memory, banking reached the port registers, the timer and interrupt controller run the show, the linker produced the HEX, and the on-board debugger burned it in. You built all of it from first principles.

***A note on config bits and LVP.** Keep LVP = ON for the Curiosity Nano's normal low-voltage ICSP path. The low-voltage programming interface cannot itself clear the LVP configuration bit; disabling LVP requires high-voltage entry. Do not apply an external high-voltage programmer to the assembled board without following the board guide's isolation precautions: high voltage on MCLR can damage the on-board debugger connection.*

18.6 What just happened

You closed the loop from source to silicon. The **HEX file** is your program as Intel-HEX records — data records, an end record, and a checksum per line — and you learned the mid-range trap that its **byte addresses aren't the chip's word addresses**. **Hexmate** is the utility that merges, fills, and checksums HEX files after linking. And the **Curiosity Nano's on-board debugger** flashed your code two ways — MPLAB X or drag-and-drop — with the PS LED reporting success. Your assembly runs on hardware.

18.7 Common mistakes

Symptom	Cause	Fix
Data in the HEX is at “the wrong address”	HEX is byte-addressed; mid-range is word-addressed	HEX addr $\approx 2 \times$ device addr; use -addressing to map (§7.2.2)
Board won’t accept a new program	wrong device/HEX, cabling, or programming state; LVP may have been disabled by external HV programming	keep LVP = ON; verify the exact HEX/device and board status before considering external tools
Drag-and-drop: PS LED blinks <i>rapidly</i>	programming failed	rebuild; check the HEX is valid and for the right device (CNANO §3.1)
No .hex produced	build stopped before Hexmate/linker ran	fix earlier assembly/link errors first
Programmed OK but nothing happens	config bits wrong (oscillator/MCLR), or LED polarity	revisit Chapter 5 config and Chapter 7’s active-low LED
Merged bootloader + app overlap	overlapping regions in the two HEX files	use Hexmate merging and check the address map (§7.2)

(Appendix C decodes the exact message wording.)

18.8 Try it yourself

1. **Read your HEX.** Open `tmr0blink.hex` in a text editor. Find the first : ... 00 ... data record and the final :00000001FF end record. Verify one record’s data length byte matches its data.
2. **Flash both ways.** Program the board from your IDE (VS Code MPLAB Debugger, or MPLAB X), then rebuild and program the *same* HEX by drag-and-drop. Watch the PS LED confirm each.
3. **Map an address.** Pick the device address of your ISR (near 0x0004) and predict its HEX-file byte offset (roughly double). Find it in the HEX.

4. **Fill the blanks (stretch).** Read Hexmate’s fill option in User’s Guide §7.2 and describe how you’d fill unused flash with a goto reset so a crashed program restarts instead of running blank memory.
-

18.9 Reference bridge

- **User’s Guide §7.2 “Hexmate”** — the HEX file specification, record types, formats, and every Hexmate option (merge, CRC, fill, find/replace).
- **Curiosity Nano guide §3.1** — the on-board debugger, drag-and-drop programming, and the PS LED status table.
- **Chapter 5** — the config bits (especially LVP) that keep programming reliable.

Next chapter: you’ve mastered the enhanced mid-range PIC16F17146 top to bottom. Chapter 19 (the optional finale) looks *down* the range to the **baseline** 12-bit cores — including a plain PIC10 without interrupts and the interrupt-capable enhanced-baseline PIC16F570 — where memory is scarce and every instruction counts.

Chapter 19 — Baseline and Enhanced-Baseline Devices (Optional Finale)

What you’ll learn: how your skills scale down to the 12-bit PIC cores. We distinguish plain baseline **PIC12** (the PIC10F200, with no interrupt controller), enhanced baseline **PIC12E**, and interrupt-capable enhanced baseline **PIC12IE** (the PIC16F570 used in the worked example). Memory is scarce and one call quirk shapes code placement on all three.

Sourcing note. Plain-baseline facts are checked against the PIC10F200/202/204/206 data sheet **DS40001239F**, errata **DS80194G**, and the PIC10-12Fxxx DFP 1.8.184 (ARCH=PIC12). The runnable example is the interrupt-capable enhanced-baseline PIC16F570, checked against data sheet **DS40001684F**, errata **DS80000624B**, the Embedded Engineers guide §9, and PIC16Fxxx DFP 1.7.162 (ARCH=PIC12IE). Always repeat that exact-device check.

19.1 The idea: the same skills, a smaller box

The family uses **12-bit-wide instructions** — narrower than mid-range’s 14 bits — and appears in PIC10, PIC12, and some PIC16 part numbers (User’s Guide §2.1). Source structure, PSECT, CONFIG, W, file registers, DS, and macros remain familiar, but capabilities must be read from the exact device rather than inferred from the marketing part number.

DFP architecture	Meaning	Concrete anchor here
PIC12	plain baseline	PIC10F200: 33 instructions, two-level return stack, no interrupt controller
PIC12E	enhanced baseline without interrupts	added core features, but no interrupt vector

DFP architecture	Meaning	Concrete anchor here
PIC12IE	enhanced baseline with interrupts	PIC16F570: 36 instructions, four-level return stack, vector at 0x0004, automatic context switching

Three things shrink or vanish:

- **Fewer instructions and less memory** — you budget both carefully.
- **Interrupt capability varies.** The plain PIC10F200 has no interrupt vector and uses polling; the PIC12IE PIC16F570 has Timer0, ADC, comparator-change, and pin-change interrupts plus automatic W/STATUS/FSR/BSR context switching (PIC16F570 data sheet §8.11–§8.12).
- **A call quirk** that dictates where callable routines can start — the one genuinely new idea, next.

19.2 The baseline call restriction

Here’s the wrinkle that catches everyone. When a baseline device executes a `call`, the target address is assembled unusually (Emb Eng §9.1):

*“bits 0 through 7 of the PC register are loaded from the instruction operand, bits 9 and 10 are loaded from the PA bits in the STATUS register but, importantly, **bit location 8 is unconditionally cleared.**”*

Clearing bit 8 means a `call` can only land in an address where bit 8 is 0 — i.e. the **first 256 locations of a page**. So on baseline devices:

- **A callable routine’s entry point must be in the first 256 words of a page** (Emb Eng §9.1).
- The **same restriction applies to any instruction that modifies PCL** (computed jumps).
- **goto is not restricted** — jumps reach anywhere in a page.

Note what this does *not* say: a routine can still be as long as you like and fill the whole page — only its **entry label** must sit in the low 256. Once execution is inside the routine, it runs on normally to the page’s end (Emb Eng §9.1).

19.3 The ENTRY class: letting the linker handle it

You don't want to hand-place every subroutine in the low 256 words. The assembler solves this with a special linker class, **ENTRY** (Emb Eng §9.1). Recall from Chapter 10 that a class is a range of addresses; **ENTRY** is defined with an *extra* address field that means "start here, but you may extend to there":

```
-AENTRY=00h-0FFh-01FFh, ...
```

That reads: a psect placed in **ENTRY** **must start within 0x00-0xFF** (the callable region) but its body **may run on up to 0x1FF** (the page end). Put your callable routines in a psect with `class=ENTRY` and the linker guarantees their entry points are reachable by `call` — without limiting the routine's length.

Routines that are only ever *jumped* to (like `main`) don't need this — they go in the ordinary `CODE` class and can live anywhere, using the upper halves of pages that **ENTRY** can't (Emb Eng §9.1).

19.4 A baseline example

Microchip's PIC16F570 example writes increasing values to `PORTA` (Emb Eng §9). This is an **enhanced-baseline PIC12IE** example, chosen because it demonstrates the **ENTRY** rule. It is not evidence that all baseline devices share the PIC16F570's interrupts, memory map, or instructions.

```
PROCESSOR 16F570
#include <xc.inc>

CONFIG "FOSC = EXTRC_CLKOUT" // illustrative: requires external RC hardware
CONFIG "WDTE = OFF"
CONFIG "CP = OFF"
CONFIG "IOSCFS = 8MHz"
CONFIG "CPSW = OFF"
CONFIG "BOREN = OFF"
CONFIG "DRTEN = OFF"

PSECT udata_shr ; common memory (baseline has little of it)
counter: DS 1
loc: DS 1
```

```

; Physical Reset fetch is 0x7FF (factory OSCCAL); it rolls over here.
PSECT userEntry,class=CODE,delta=2
userEntry:
    ljmp    main                ; ljmp: page-safe jump (Chapter 9)

PSECT code
main:
    movlw   0
    tris    PORTA              ; baseline 'tris' instruction: 0 → all outputs
    clrf    counter
    movf    counter,w
loop:
    fcall   increment          ; fcall: page-safe call (Chapter 9)
    movwf   PORTA              ; write directly to PORTA - no LATx on baseline
    goto    loop

; a CALLABLE routine → must go in the ENTRY class
PSECT entryCode,class=ENTRY,delta=2
increment:
    movwf   loc
    incf    loc,w
    return

    END        userEntry

```

Three baseline-specific details in that listing:

1. **increment is in class=ENTRY** because main *calls* it (fcall increment). That's what guarantees its entry point lands in a page's low 256. main itself is only *jumped* to (ljmp main), so it stays in the ordinary code psect (Emb Eng §9.1).
2. **No LATx on this PIC16F570.** Write PORTA directly and set direction with the dedicated tris instruction. Recheck the registers on any other device.
3. **Use long pseudo-instructions where page placement is not guaranteed.** ljmp main and fcall increment are page-safe. The raw goto loop is a local branch within the same small psect; verify final placement in the list/map before making that assumption in larger code.

The EXTRC_CLKOUT setting matches Microchip's toolchain example and requires suitable external RC hardware; choose oscillator settings for your actual board before expecting the HEX to run. Errata DS80000624B's Sleep/wake issue affects silicon A1 when DRT is enabled; this example sets DRTEN = OFF. The errata also clarifies the enable conditions for comparator-change and interrupt-on-change flags—recheck those notes before adding such an ISR.

There is an important reset-path exception here. PIC16F570 hardware begins at **0x7FF**, the last implemented program word, where Microchip programs an oscillator-calibration `MOVLW` value. After that instruction, the PC rolls over to **0x000**, our `userEntry` psect (data sheet §4.1/§4.7.1). Do not erase or replace the factory calibration word. Microchip’s guide calls the psect at 0 `resetVec`; this book uses `userEntry` so it is not confused with the physical Reset fetch.

Build it just like any other project (Emb Eng §9.2):

```
pic-as -mcpu=16F570 -mdfp=/path/to/Microchip.PIC16Fxxx_DFP/1.7.162/xc8 \
  -Wl,-puserEntry=0h -Wa,-a -Wl,-Map=incPort.map incPort.S
```

19.5 Seeing ENTRY in the map file

Open the map (Chapter 17’s skill) and you’ll see the two classes do their jobs (Emb Eng §9.1):

TOTAL	Name	Link	Load	Length	Space		
CLASS CODE							
	userEntry	0	0	5	0		
	code		1F5	1F5	B	0	<-
upper half of a page: fine, only jumped to							
CLASS ENTRY							
	entryCode	5	5	3	0	<- low 256 of a page: callable	

code landed at 0x1F5 — the *top* of a page, which is fine because nothing calls into it. `entryCode` landed at 0x5 — safely inside the entry region, so `fcall` increment works. The linker enforced the baseline rule for you; the map file proves it.

A cost to know. *The linker finds ENTRY psects harder to place than plain CODE ones, because of the low-256 restriction (Emb Eng §9.1). So mark a routine ENTRY only if it’s actually called or reached via a PCL-modifying instruction — leave everything else in CODE to make the most of those upper page halves.*

19.6 What just happened (and a look back)

You scaled all the way down to the 12-bit baseline core and found the same discipline with tighter limits. The plain PIC10F200 has 33 instructions and no interrupt controller; the worked PIC16F570 is PIC12IE, has 36 instructions and interrupts, and writes PORTA via tris. Both share the key placement rule: **callable routines must start in the low 256 words of a page**, which the ENTRY class handles for you.

And with that, you’ve traveled the entire 8-bit PIC range: **baseline (12-bit)** here, the **enhanced mid-range (14-bit) PIC16F17146** through the heart of the book, and **PIC18 (16-bit)** in Chapters 15–16. The core skills never changed — reset vector, psects, banking, the build pipeline, reading a data sheet before you write a line — only the size of the box around them. That’s the real lesson: learn one PIC deeply and you can meet any of them.

19.7 Common mistakes

Symptom	Cause	Fix
call to a routine jumps to the wrong place	routine’s entry point is above the low 256 of its page	put callable routines in class=ENTRY (Emb Eng §9.1)
Linker “can’t place” an ENTRY psect	too many callable routines competing for the low 256	move non-called code to CODE; keep ENTRY for true entry points
bsf LATA,0 won’t assemble for PIC16F570	this device has no LATx register	write PORTA directly; set direction with tris
Assumed the PIC16F570 has no ISR	confused plain PIC12 with PIC12IE	PIC16F570 vectors at 0x0004; PIC10F200 has no interrupt controller
Paging bug on a call/goto	used raw instructions across a page	use fcall/ljmp (Chapter 9)
Reset behaves unexpectedly	assumed Reset starts at 0	PIC16F570 fetches 0x7FF (factory calibration), then rolls to user entry 0; other devices differ

(Appendix C decodes the exact message wording.)

19.8 Try it yourself

1. **Read the classes.** Build the PIC16F570 example and confirm in the map that `entryCode` is in `ENTRY` (low address) while `code` sits high in a page.
 2. **Force the failure.** Move `increment` into the plain code psect, re-build, and see whether the linker (or the running program) objects to a call landing above the low 256.
 3. **Port a routine.** Take Chapter 13's `delay` and adapt it for PIC16F570: `class=ENTRY` if it's called, `tris/PORT` instead of `LAT`, and page-safe transfers where final placement is not guaranteed.
 4. **Datasheet drill.** Compare PIC10F200 and PIC16F570: program-memory size, reset behavior, return-stack depth, instruction count, and interrupt capability.
-

19.9 Reference bridge

- **Embedded Engineers guide §9** — the full baseline example, the call entry-point restriction, and the `ENTRY` class.
- **User's Guide §2.1** — the baseline vs. enhanced-baseline core descriptions (including which have interrupts).
- **User's Guide §5.3** — the `ENTRY` linker class definition and the other predefined classes.
- **PIC10F200 data sheet DS40001239F §4.7-§4.8** — plain-baseline call restriction and stack; its feature/instruction tables establish the 33-instruction, non-interrupt anchor.
- **PIC16F570 data sheet DS40001684F §4.6-§4.7 and §8.11-§8.12** — the same call rule plus the `PIC12IE` interrupt vector and automatic context switching.

Where to next: that's the last core chapter. The **appendices** are your working references — the instruction-set and directive quick-cards, a guide to decoding the assembler's error and warning messages (Appendix C), the MPLAB X option reference, an MPASM→XC8 migration cheat sheet, a glossary, and a bridge from assembly into XC8 C. Keep them beside you as you write your own PIC assembly from scratch.

Appendix A — Instruction Set Quick Reference

This appendix gives the complete **50-instruction PIC16F17146** quick card, then compact delta cards for the plain-baseline PIC10F200, enhanced-baseline PIC16F570, and PIC18F57Q43. The exact device data sheet remains authoritative for encodings and device-specific extensions.

Operand key (data sheet §41, Table 41-1): f = file register address (0x00-0x7F) · w = Working register · b = bit number (0-7) · k = literal/constant · d = destination ($,w \rightarrow$ result to W, $,f \rightarrow$ result to f ; default is $,f$) · n = FSR number (0-1) · mm = pre/post inc-dec mode.

Status flags: c = Carry · dc = Digit Carry · z = Zero · to = Time-Out · pd = Power-Down.

A.1 Byte-oriented operations

Mnemonic	Operation	Flags
ADDWF f,d	$W + f \rightarrow \text{dest}$	C, DC, Z
ADDWFC f,d	$W + f + \text{Carry} \rightarrow \text{dest}$	C, DC, Z
ANDWF f,d	$W \text{ AND } f \rightarrow \text{dest}$	Z
ASRF f,d	arithmetic right shift f	C, Z
LSLF f,d	logical left shift f	C, Z
LSRF f,d	logical right shift f	C, Z
CLRF f	$0 \rightarrow f$	Z
CLRW	$0 \rightarrow W$	Z
COMF f,d	complement $f \rightarrow \text{dest}$	Z
DECF f,d	$f - 1 \rightarrow \text{dest}$	Z
INCF f,d	$f + 1 \rightarrow \text{dest}$	Z
IORWF f,d	$W \text{ OR } f \rightarrow \text{dest}$	Z
MOVF f,d	$f \rightarrow \text{dest}$ (use $,w$ to load W)	Z
MOVWF f	$W \rightarrow f$	—
RLF f,d	rotate f left through Carry	C
RRF f,d	rotate f right through Carry	C
SUBWF f,d	$f - W \rightarrow \text{dest}$	C, DC, Z

Mnemonic	Operation	Flags
SUBWFB <i>f</i> , <i>d</i>	<i>f</i> − <i>W</i> − Borrow → <i>dest</i>	C, DC, Z
SWAPF <i>f</i> , <i>d</i>	swap nibbles of <i>f</i> → <i>dest</i>	—
XORWF <i>f</i> , <i>d</i>	<i>W</i> XOR <i>f</i> → <i>dest</i>	Z

Byte-oriented skip

Mnemonic	Operation	Flags
DECFSZ <i>f</i> , <i>d</i>	<i>f</i> − 1 → <i>dest</i> , skip next if 0	—
INCFSZ <i>f</i> , <i>d</i>	<i>f</i> + 1 → <i>dest</i> , skip next if 0	—

A.2 Bit-oriented operations

Mnemonic	Operation	Flags
BCF <i>f</i> , <i>b</i>	0 → bit <i>b</i> of <i>f</i>	—
BSF <i>f</i> , <i>b</i>	1 → bit <i>b</i> of <i>f</i>	—
BTFSC <i>f</i> , <i>b</i>	skip next if bit <i>b</i> of <i>f</i> is clear	—
BTFSS <i>f</i> , <i>b</i>	skip next if bit <i>b</i> of <i>f</i> is set	—

A.3 Literal operations

Mnemonic	Operation	Flags
ADDLW <i>k</i>	<i>W</i> + <i>k</i> → <i>W</i>	C, DC, Z
ANDLW <i>k</i>	<i>W</i> AND <i>k</i> → <i>W</i>	Z
IORLW <i>k</i>	<i>W</i> OR <i>k</i> → <i>W</i>	Z
MOVLB <i>k</i>	<i>k</i> → BSR (bank select)	—
MOVLP <i>k</i>	<i>k</i> → PCLATH (page select)	—
MOVLW <i>k</i>	<i>k</i> → <i>W</i>	—
SUBLW <i>k</i>	<i>k</i> − <i>W</i> → <i>W</i>	C, DC, Z
XORLW <i>k</i>	<i>W</i> XOR <i>k</i> → <i>W</i>	Z

A.4 Control operations

Mnemonic	Operation	Cycles	Flags
BRA k	relative branch (PC+1+k)	2	—
BRW	relative branch (PC+1+W)	2	—
CALL k	call subroutine (k = 0-2047)	2	—
CALLW	call subroutine at PCLATH:W	2	—
GOTO k	jump to address	2	—
RETURN	return from subroutine	2	—
RETLW k	return, putting k in W	2	—
RETFIE	return from interrupt	2	—

CALL/GOTO carry an **11-bit** address (reach 2048 words = one page); the upper bits come from PCLATH (Chapter 9).

A.5 Inherent operations

Mnemonic	Operation	Flags
NOP	no operation	—
SLEEP	enter Sleep mode	TO, PD
CLRWDT	clear Watchdog Timer	TO, PD
RESET	software device reset	—
TRIS f	load TRIS register from W (legacy)	—

A.6 Indirect / FSR operations

Mnemonic	Operation	Flags
ADDFSR n, k	$FSRn + k \rightarrow FSRn$ (k = -32...31)	—
MOVIW ++FSRn / --FSRn	pre-inc/dec FSRn, then INDFn $\rightarrow$ W	Z
MOVIW FSRn++ / FSRn--	INDFn $\rightarrow$ W, then post-inc/dec FSRn	Z
MOVIW k[FSRn]	$*(FSRn + k) \rightarrow W$, FSRn unchanged (k = -32...31)	Z
MOVWI ++FSRn / --FSRn	pre-inc/dec FSRn, then W $\rightarrow$ INDFn	—

Mnemonic	Operation	Flags
MOVWI FSRn++ / FSRn--	W → INDFn, then post-inc/dec FSRn	—
MOVWI k[FSRn]	W → *(FSRn + k), FSRn unchanged	—

That’s the full 50 (data sheet §41, Table 41-3). Everything else you type is either a *directive* (Appendix B) or an assembler *pseudo-instruction* (below).

A.7 Assembler conveniences (not part of the 50)

Some generate one or more real instructions; the *MASK() forms are expression macros that emit no instruction (User’s Guide §4.1):

Pseudo-op	Expands to / does
BANKSEL obj	emits movlb to select obj’s data bank (Chapter 8)
BANKMASK(obj)	expression macro: trims an operand to its bank offset; emits no instruction
PAGESEL lbl	emits movlp to select lbl’s program page (Chapter 9)
PAGEMASK(lbl)	expression macro: trims an operand to the call/goto field; emits no instruction
fcall lbl	page-safe call pseudo-instruction; XC8 4.00 may emit page select/restore code even when the final target shares a page (Chapter 9)
ljmp lbl	page-safe jump pseudo-instruction; may expand to page-selection code plus goto (Chapter 9)

A.8 Cross-core delta cards

These cards highlight what changes; they are not substitutes for the full instruction tables.

Core/device anchor	Instruction-set shape	Important differences from PIC16F17146
Plain baseline PIC12: PIC10F200	33 single-word, 12-bit instructions	two-level return stack; RETLW is the return form; no RETURN, RETFIE, MOVLB, or interrupt controller
Enhanced baseline PIC12IE: PIC16F570	36 single-word, 12-bit instructions	adds MOVLB, RETURN, and RETFIE; four-level return stack and interrupt support
PIC18: PIC18F57Q43	standard 16-bit PIC18 set plus device extensions	explicit access operand ,c/,a vs ,b; MOVFF/MOVFLL, LFSR, multiply, conditional branches, PUSH/POP, table read/write, fast CALL/RETURN/RETFIE modes

Transfer rules also change: baseline CALL and PCL-modifying instructions can enter only the first 256 words of a 512-word page (Chapter 19); enhanced mid-range CALL/GOTO use PCLATH and 2048-word pages; PIC18 uses byte-addressed program memory and does not need PAGE-SEL.

Data-sheet wording note. DS40002343F Table 41-3 prints RETFIE k, but its detailed instruction entry says **Operands: None** and shows RETFIE. Use the operand-free form on the PIC16F17146; PIC18 fast return is the separate retfie f syntax described in Chapter 15.

A.9 Reference

- **Data sheet §41 “Instruction Set Summary”** — the authoritative table (Table 41-3) and per -instruction detail (syntax, operands, cycles, encoding, status).
- **User’s Guide §4.1** — the assembler’s instruction deviations, the ,w/,f operand styles, and the pseudo-instructions above.
- **PIC10F200 data sheet DS40001239F §10** and **PIC16F570 data sheet DS40001684F §13** — the two 12-bit-core instruction tables.
- **PIC18F27/47/57Q43 data sheet DS40002147H §44** — the Q43 standard and extended instruction tables.

Appendix B — Directive Reference Card

Directives are orders to the **assembler**, not CPU opcodes. `BANKSEL`, `BANKISEL`, and `PAGESEL` can generate instructions; `DB/DW/DQW` generate initialized data; `DS` reserves storage; many others only control assembly or linking (User's Guide §4.9). `#include` is a C preprocessor directive, included here because every preprocessed `.s` file needs it.

B.1 The directives you'll use constantly

Directive	Purpose	Chapter
<code>PROCESSOR dev</code>	name the target device (must match <code>-mcpu</code>)	4
<code>#include <xc.inc></code>	pull in the device's register/SFR/bit definitions	4
<code>CONFIG "SET = VAL"</code>	set a configuration bit	5
<code>PSECT name, flags</code>	declare/resume a program section (see B.3)	4, 10
<code>END [label]</code>	end of source; optional entry-point label	4
<code>EQU</code>	define a constant (define-once)	12
<code>SET</code>	define/ redefine a value	12
<code>DS n</code>	reserve <code>n</code> uninitialized units (RAM variables)	6, 12
<code>DB / DW / DQW</code>	initialize constant bytes / words / 64-bit values in program memory	12
<code>GLOBAL sym</code>	export a symbol / reference an external one	13
<code>EXTRN sym</code>	import a symbol defined in another module	13
<code>MACRO ... ENDM</code>	define a macro (with args, & concatenation)	13

Directive	Purpose	Chapter
LOCAL lbl	unique label per macro expansion	13
IF/ELSIF/ELSE/ENDIF	conditional assembly	12
BANKSEL obj	select obj's data bank (emits code)	8
PAGESEL lbl	select lbl's program page (emits code)	9

B.2 The rest of the directives (reference)

From User's Guide §4.9, Table 4-6, with the XC8 4.00 release-note update. You'll rarely need most of these as a beginner, but here's the map:

Data & storage: DABS (absolute storage), DLABS (linear-memory absolute storage), ORG (move location counter within current psect), ALIGN (align to a boundary).

Symbols & linkage: PUBLIC (make symbols accessible), SIGNAT (function signature).

Source inclusion and conditionals: assembler INCLUDE, IF, ELSIF, ELSE, ENDIF, and the END directive already shown above.

Messages & build control: ERROR (halt with a message), MESSG (advisory message), WARN (warning), ERRORLEVEL (enable/disable message numbers), RADIX (default number base), ASMOPT (optimizer control — no effect in the PIC assembler).

Repetition: REPT (repeat n times), IRP (repeat per list item), IRPC (repeat per character).

Listing file: LIST / NOLIST, EXPAND / NOEXPAND (macro expansion), COND / NOCOND (conditional code in listing), TITLE / SUBTITLE, PAGELEN, PAGEWIDTH.

Compiled-stack / call-graph (Chapter 16): FNCONF, FNROOT, FNSIZE, FN-CALL, FNARG, FNINDIR, FNADDR, FNBREAK; CALLSTACK reports remaining hardware call-stack depth to an optimizer (the current PIC assembler performs no such optimization).

Bank/page for indirect: BANKISEL (bank select for indirect access on some devices).

Debug/intermediate (usually tool-generated, not hand-written): LINE, FILE, DEBUG_SOURCE.

XC8 4.00 compatibility note. Release notes add `DQW` for 64-bit program-memory constants. Although the 2024 assembler guide documents `DDW`, XC8 4.00 still lists `DDW` as unsupported known issue XC8-1817. Do not depend on `DDW` without testing the installed compiler version.

B.3 PSECT flags (the ones that matter)

`PSECT name, flag, flag, ...` — the flags define the section’s attributes (User’s Guide §4.9.48, Table 4-10). The four you’ll actually use on the PIC16F17146:

Flag	Meaning	Rule for our chip
<code>class=NAME</code>	which linker class (memory range)	<code>CODE</code> for program memory, <code>BANKn/COMMON/RAM</code> for data
<code>delta=n</code>	bytes per address unit	<code>delta=2</code> for code (word-addressable); 1 (default) for data
<code>space=n</code> <code>reloc=n</code>	which memory space alignment boundary	0 = program , 1 = data default 1 on mid-range (PIC18 code needs 2)

Other flags you may meet (Table 4-10): `global` (default) / `local` (concatenate across modules or not), `ovrld` (overlay same-named contributions; useful for relocatable vector tables), `abs` (with `ovrld`, makes symbols truly absolute), `bit` (holds bit objects, scale 8), `size=/limit=` (bounds), `pure` (read-only), `with=` (co-locate with another psect), and `optim=` (allowed optimizations; no effect in hand-written PIC assembler code).

Common psect declarations, ready to copy

```
PSECT resetVec,class=CODE,delta=2      ; reset/interrupt code, positioned with
↪ -Wl,-p ...
PSECT code                             ; assembler-provided ordinary
↪ program-code psect
PSECT udata                            ; RAM variables (any GPR bank) - BANKSEL
↪ to use
PSECT udata_shr                        ; RAM variables in common RAM (no BANKSEL)
PSECT udata_bankn                     ; RAM variables in a specific bank n
```

(The assembler-provided psects code, udata, udata_shr, udata_bankn, data come pre-wired to the right class once you #include <xc.inc>; User’s Guide §5.2.)

B.4 Number formats (quick reminder)

Default radix is **decimal** (User’s Guide §4.5). Other bases:

Base	Write it	Note
Decimal	255 or 255D	default
Hex	0xFF or 0FFh	must start with a digit — 0FFh, not FFh
Binary	11111111B	uppercase B suffix
Octal	377o / 377q	—

B.5 Reference

- **User’s Guide §4.9 “Assembler Directives”** — Table 4-6 (all directives) and each directive’s detailed description.
- **User’s Guide §4.9.48 “Psect Directive”** — Table 4-10, the complete flag list.
- **User’s Guide §5.2 / §5.3** — the assembler-provided psects and the linker classes they map to.

Appendix C — Decoding Error & Warning Messages

Every chapter’s “common mistakes” table points here. This appendix teaches you to **read** the assembler’s messages, **look them up**, and fix the fifteen problems that trip up beginners most. Learning to decode errors is not a side skill — it’s half of writing assembly.

C.1 How to read a message

The MPLAB XC8 assembler prints each message in a fixed shape (User’s Guide §8):

(1507) asmopt state popped when there was no pushed state (Assembler)

- **(1507)** — a **unique message number**. It’s printed with the message and is your lookup key.
- **The text** — the description; where you see a * in the guide, the tool substitutes a specific string (a filename, symbol, etc.) (§8).
- **(Assembler)** — the **application** that produced it. This tells you *which stage* failed.

That last part matters more than beginners expect. The message set is shared across the whole toolchain — it’s “the complete and historical message set covering all former HI-TECH C compilers” (§8), so **most messages in the guide are about C, not assembly**. Filter by the application tag:

Tag	Stage	Relevant to assembly?
(Preprocessor)	#include/#define handling	yes
(Assembler)	assembling your .s	yes — your main concern
(Linker)	placing psects, resolving symbols	yes
(Hexmate)	building the HEX	sometimes

Tag	Stage	Relevant to assembly?
(Driver)	coordinating options, device packs, and build stages	yes
(Code Generator), (Parser)	the C compiler	usually ignore for pure assembly

If a message is tagged (Code Generator) or (Parser), it is almost certainly about C and not your pure-assembly source. Driver messages still matter—for example, a missing external DFP is a driver failure before assembly begins.

C.2 How to look one up

1. Note the **number** the tool printed, e.g. (107).
2. Open the User’s Guide §8 “**Error and Warning Messages**” — messages are sorted numerically across §8.1–8.5.
3. Read the description and its example. The guide often shows the exact code that triggers it.

Two strategy notes straight from the guide (§8):

- **Fix errors in the order shown.** “You should attempt to resolve errors or warnings in the order in which they are displayed.”
- **One mistake can spawn several messages.** “One problem in your... source code can trigger more than one error message.” Fix the first, rebuild, and watch the rest often vanish.

You can also silence *advisory/warning* numbers with the `ERRORLEVEL` directive (Chapter 12) — but **you cannot disable an error**: attempting `ERRORLEVEL -2070` on an error itself produces message (1607) (§8.4).

C.3 The fifteen most common beginner problems

Symptom → what it means → fix. Cross-references point to the chapter that explains the concept. Where a message number is shown, it’s from the guide’s §8; where none is shown, identify the error by its text and the producing application.

Build won't even start

1. **#include treated as an illegal directive / SFR names undefined** (*Assembler/Preprocessor*) A lowercase `.s` file normally skips preprocessing, so `#include <xc.inc>` never supplies the selected device definitions. → Rename it to uppercase `.S`, and confirm `-mdfp` points to a complete installed/unpacked pack's `xc8` directory. (Chapters 3–4)
2. **(103) #error: *** (*Preprocessor*) Your own `#error` directive fired deliberately — often a config guard. → Find the `#error` and satisfy the condition it's checking. (Chapter 12)
3. **(111) redefining preprocessor macro "*"** (*Preprocessor*) A `#defined` name was redefined to something different. → `#undef` it first, or use a new name. (Chapter 12)

Assembler errors (your .s)

4. **A hex constant like FFh reported as an unknown symbol** (*Assembler*) Hex numbers must start with a digit, or the assembler reads them as identifiers. → Write `0FFh` or `0xFF`. (Chapter 4 §4.5)
5. **"duplicate symbol" / "symbol already defined"** (*Assembler*) Either you named something after a keyword (`and`, `goto`, `mod`), or a label inside a macro isn't `LOCAL` so it's defined on every expansion. → Rename the identifier, or wrap macro-internal labels with `LOCAL`. (Chapters 4, 13)
6. **(113) unterminated string in preprocessor macro body** (*Preprocessor*) A string in a macro is missing its closing quote. → Add the quote. (Chapter 13)
7. **EQU-defined symbol won't redefine** (*Assembler*) `EQU` is define-once. → Use `SET` for a value you intend to reassign. (Chapter 12 §12.2)
8. **movfw unknown instruction** (*Assembler*) That MPASM pseudo-instruction doesn't exist in XC8. → Write `movf f,w`. (Chapter 6 §6.4)
9. **"contradictory flags" on a PSECT** (*Assembler*) You re-declared a psect with different flags than its first declaration. → Flags propagate from the first PSECT; don't restate them differently. (Chapter 10)

Linker errors (placement & symbols)

10. **"fixup overflow"** (*Linker*) An operand's full address (with bank or page bits) is too big for the instruction's narrow address field. → For data: `BANKSEL` **and** `BANKMASK()` the operand. For calls/jumps across a page: `PAGESEL + PAGEMASK()`, or `fcall/ljmp`. The linker option `-Wl,--fixupoverflow=warn:lstwarn` deliberately truncates the value and is useful

for locating legacy cases; it does **not** select the correct bank/page and is not the fix. (Chapters 8, 9)

****11. “undefined symbol *” (Linker) A symbol you referenced isn’t defined in any linked module. → Make sure the defining file is in the build, and that the symbol is exported with GLOBAL**** (or imported with EXTRN). (Chapter 13)

****12. “can’t find space for psect *” (Linker) A psect won’t fit its class. → Open the map file’s UNUSED ADDRESS RANGES and check the Largest block**** — the free space may be fragmented into pieces all smaller than your psect. (Chapter 17 §17.7)

13. Code psect misbehaves / wrong bytes in HEX (Linker/Assembler) A class=CODE psect on our mid-range chip is missing delta=2 (program memory is word -addressable). → Add delta=2 to every code psect. (Chapters 4, 10)

Config & runtime (builds, but doesn’t run right)

14. CONFIG setting rejected: “invalid/unknown setting or value” (Assembler) A CONFIG token doesn’t match your Device Family Pack’s spelling. → Generate the exact tokens from MPLAB X’s **Configuration Bits** window for the PIC16F17146. (Chapter 5 §5.2)

15. Builds fine but the chip does nothing / hangs (no message — a runtime symptom) The most common non-error “error.” Usual causes: an interrupt source not acknowledged as its peripheral requires, so the ISR re-fires (Chapter 14); a pin left analog because ANSELx wasn’t cleared (Chapter 7); or the reset vector not placed at 0 (-w1, -presetVec=0h, Chapter 4). → Work the symptom back to its chapter; the simulator’s Watch window is your friend here.

C.4 A worked lookup

Say the build stops with:

```
main.S: (107) illegal # directive "indef" (Preprocessor)
```

Decode it: number **107**, producing app **Preprocessor**, and the substituted text `indef`. Look up (107) in §8.1 — it means the preprocessor doesn’t recognize a # directive, “probably a misspelling” (§8.1). And there it is: `#indef` should be `#undef`. Fix the typo, rebuild. That’s the whole loop: **read the number → find it in §8 → apply the fix.**

C.5 Reference

- **User’s Guide §8 “Error and Warning Messages”** — the full numbered catalog (§8.1 messages 0-499 through §8.5 messages 2000-2499). Your lookup table.
- **User’s Guide §4.9.18 “Errorlevel Directive”** — enabling/disabling advisory and warning numbers (Chapter 12).
- The **“Common mistakes” table** at the end of every chapter — the fastest route from a symptom to the concept behind it.

Appendix D — MPLAB X & pic-as Option Reference

The command-line driver is **pic-as**, and MPLAB X issues these same options for you based on Project Properties (User’s Guide §3, §3.4). Options are **case-sensitive** and start with a dash (§3.4). This card covers the ones a beginner meets; the full list is User’s Guide §3.4, Table 3-1.

Basic form (User’s Guide §3):

```
pic-as [options] files [libraries]
```

D.1 Options this book actually uses

Option	What it does	Seen in
-mcpu=16f17146	target device (must match PROCESSOR)	every build
-mdfp=<DFP>/xc8	required on modern standalone XC8 — path to the device pack (MPLAB X adds it automatically; without it you get “no device-support files”, err 2103/2104)	Ch 3
-Wl,-presetVec=0h	position the resetVec psect at address 0	Ch 4
-Wl,-pintVec=0004h	position the interrupt-vector psect at 0x0004	Ch 14
-Wl,-pfarCode=0800h	position a psect at a chosen address (page 1)	Ch 9
-Wl,-Map=file.map	produce a map file	Ch 17
-Wa,-a	produce an assembly list file (.lst)	Ch 4+

Option	What it does	Seen in
-Wl, -- fixupoverflow=warn:lstwarn	truncate overflowing fixups and report them in console/list output for migration diagnosis; does not select a bank/page	Ch 7, 8
-mcallgraph=full	print the compiled-stack call graph in the map	Ch 16
-Wl, -pudata_acs=COMRAM	place the compiled-stack psect	Ch 16
-c	assemble to an object (.o) without linking	Ch 13

The three “pass-through” prefixes (User’s Guide §3.4, Table 3-1)
— remember which stage each targets: -Wl, → to the **linker** (map, psect positioning, fixupoverflow) -Wa, → to the **assembler** (list file)
- Wp, → to the **preprocessor**

In MPLAB X, put these in the Custom linker/assembler options fields **without** the leading -Wl,/-Wa, — the IDE adds that prefix (Embedded Engineers guide §4.4).

D.2 The broader option set (from Table 3-1)

Option	Controls
-o file	output file name
-Dmacro=text	define a preprocessor symbol (e.g. -DDEBUG=1)
-Umacro	undefine a preprocessor symbol
-Idir	directory searched for #include headers
-llibrary / -Ldir	libraries to scan / where to find them
-mdfp=path	which Device Family Pack to use
--fill=options	fill unused memory (same as Hexmate -fill)
-mchecksum=specs	generate/place a checksum or hash
-mram= / -mrom= / -mreserve=	limit/reserve data or program memory ranges
-mserial=options	insert a serial number into the output

Option	Controls
-fmax-errors=n	how many errors before the build aborts
-mwarn=level / -w	warning threshold / suppress all warnings
-Werror, -Werror=num, -Wno-error	promote all warnings, one numbered warning, or undo promotion
-save-temps	keep intermediate files after the build
-v / --version / --help	verbose output / version / help
-mprint-devices	list supported devices

D.3 Long command lines (@file)

To store options and file names in a file (a simple alternative to a makefile), use @ immediately followed by the filename (User's Guide §3.3):

```
# xyz.xc8
-mcpu=16f17146 -Wl,-Map=proj.map -Wa,-a \
main.S isr.S

pic-as @xyz.xc8
```

Arguments are space-separated, may span lines with a trailing \, and blank lines are ignored (§3.3). MPLAB X supports these via **XC8 Linker** → **Additional options** → **Use response file to link**.

Reproducible pack-specific examples

Current standalone XC8 4.00 builds need a complete unpacked/installed pack path, not merely a chip-specific header subset:

```
pic-as -mcpu=16f17146 -mdfp=/packs/Microchip.PIC16F1xxxx_DFP/1.31.465/xc8 main.S
pic-as -mcpu=18f57q43 -mdfp=/packs/Microchip.PIC18F-Q_DFP/1.30.487/xc8 main.S
pic-as -mcpu=16f570 -mdfp=/packs/Microchip.PIC16Fxxx_DFP/1.7.162/xc8 main.S
```

The xc8 directory must contain generic files such as pic/include/pic.inc and device-selection metadata as well as the device header. MPLAB X normally supplies this path from its pack manager.

D.4 Reference

- **User's Guide §3.4 "Assembler Option Descriptions"** — Table 3-1 and every option's detail.
- **User's Guide §3.1-3.3** — single-step vs. multi-step builds and command files.
- **User's Guide §3.5** — how each Project Properties control maps to a pic-as option.

Appendix E — MPASM → XC8 Migration Notes

If you have older PIC assembly written for **MPASM** (Microchip’s legacy assembler), it won’t build as-is under the MPLAB XC8 PIC assembler (pic-as) — the syntax differs in several places. This appendix is a starter cheat-sheet of the changes you’ll hit most; every delta here is documented in the XC8 PIC Assembler User’s Guide sections cited.

Authoritative full reference. Microchip’s **MPLAB XC8 PIC Assembler Migration Guide DS50002973B** describes the nearest XC8 syntax and directives for MPASM code. For anything beyond the common cases below, use that guide and rebuild against the exact device pack.

E.1 The high-frequency changes

Concept	MPASM	XC8 (pic-as)	Source
Destination operand	,0 (W) / ,1 (file)	,w (W) / ,f (file)	§4.1.1
PIC18 access bit	,0 / ,1	,c/,a (access) / ,b (banked)	§4.1.1
Move file to W	movfw foo	movf foo,w (no movfw)	§4.1.4
Config bits	__CONFIG / config word	CONFIG "SETTING = VALUE"	§4.9.7
Include device defs	#include <p16f ... inc>	#include <xc.inc> (one header, all devices)	§3.3 (Emb Eng)
Reserve RAM	res/cblock	PSECT udata + DS n	§4.9.13, §5.2
Program sections	MPASM sections	PSECT name, class=..., delta=..., space=...	§4.8, §4.9.48
Bank select	banksel	BANKSEL (+ BANKMASK() on the operand)	§4.1.2–3

Concept	MPASM	XC8 (pic-as)	Source
Page select	pagesel	PAGESEL (+ PAGEMASK()); or use fcall/ljmp)	§4.1.2–3

E.2 Things that trip up MPASM porters

- **Operand address masking is now explicit.** MPASM often hid bank/page bits; XC8 makes you mask with BANKMASK()/PAGEMASK() or use fcall/ljmp, or you'll get *fixup overflow* errors. --fixupoverflow=warn:lstwarn truncates and reports legacy cases; it does not select the correct bank/page and is not the migration fix (User's Guide §4.1.3; Chapters 8–9).
- **Case sensitivity.** XC8 identifiers are case-sensitive (Fred ≠ fred); mnemonics and directives are not (User's Guide §4.3, §4.6). Legacy code that was sloppy about label case may break.
- **Hex constants need a leading digit.** FFh becomes 0FFh (User's Guide §4.5).
- **#include <xc.inc> replaces per-device headers.** Don't include pic16f....inc files or maintain your own — xc.inc provides the SFR/bit definitions for your selected device (Emb Eng §3.3).
- **The .s extension matters.** Use uppercase .s so the C preprocessor runs (enabling #include/#define); lowercase .s skips it (User's Guide §4.6.4).
- **PIC18 fast modes.** XC8 writes interrupt fast return as retfie f; a fast call uses call target,f. The data-sheet 0/1 forms are also accepted, but f is clearer (User's Guide §4.1.6–§4.1.7).
- **Current standalone builds require -mdfp.** Point it at the complete selected pack's xc8 directory; an extracted single-device header directory is not enough.

E.3 A tiny before/after

MPASM style:

```
banksel PORTA
movf     PORTA,0           ; W = PORTA
movwf    temp,1           ; temp = W
__CONFIG _WDT_OFF
```

XC8 (pic-as) style:

```
BANKSEL PORTA
movf     BANKMASK(PORTA),w ; W = PORTA
BANKSEL temp
movwf    BANKMASK(temp)    ; temp = W (MOVWF has no destination operand)
CONFIG "WDTE = OFF"
```

E.4 Reference

- **MPLAB XC8 PIC Assembler Migration Guide** — the complete MPASM→XC8 equivalence reference (User's Guide §1.2, Recommended Reading).
- **User's Guide §4.1** — instruction deviations (operand styles, `movfw`, masking).
- **User's Guide §4.9.7** — the `CONFIG` directive.
- **Chapters 8-9** — banking/paging and the masking that MPASM used to hide.

Appendix F — Glossary

Plain-language definitions of the terms used in this book. The chapter in brackets is where the term is introduced or explained in depth.

Access bank — On PIC18, a region of data memory reachable without bank selection (the PIC18 counterpart to mid-range common RAM). (*Ch 15*)

ALU (Arithmetic-Logic Unit) — The part of the CPU that performs arithmetic and bitwise operations, working through the W register. (*Ch 1*)

ANSEL (ANSELx) — The analog-select register for a port. A pin defaults to *analog* after reset; clear its ANSEL bit to make it a digital I/O. (*Ch 7*)

ARCH — The core-architecture value in a device's INI file (PIC12, PIC14E, PIC18, ...) that tells the assembler which core it's building for. (*Ch 2*)

Assembler — The tool (*pic-as*) that translates mnemonic instructions and directives into machine-code opcodes in an object file. (*Ch 1, 3*)

Bank / Banking — A core-specific division of data memory. PIC16F17146 uses 128-byte banks and BSR selection; PIC18 banks are 256 bytes; small baseline devices use their own STATUS/BSR rules. (*Ch 8, 15, 19*)

Baseline — A 12-bit-instruction PIC core. DFP PIC12 denotes plain baseline (for example, PIC10F200 without interrupts); PIC12E and PIC12IE are enhanced variants, with PIC12IE explicitly interrupt-capable (for example, PIC16F570). (*Ch 2, 19*)

BSR (Bank Select Register) — On enhanced mid-range/PIC18 and some enhanced-baseline devices, holds the current data-memory bank number; BANKSEL emits the core-appropriate selection code. (*Ch 8*)

Common RAM — On PIC16F17146, 16 bytes at bank offsets 0x70-0x7F mirrored into every bank, so they need no BANKSEL. Other devices have different shared/Access regions. (*Ch 6, 8, 15*)

Compiled stack — A linker-allocated region for routine-local variables, with the locals of non-overlapping routines reused to save RAM. No stack pointer; not reentrant. (*Ch 16*)

Config bits (Configuration Words) — Nonvolatile settings (oscillator, watchdog, MCLR, protection) read at start-up; set with CONFIG. Their number and fields are device-specific. (*Ch 5*)

Core — A CPU design shared across many PICs, distinguished mainly by instruction width (baseline 12-bit, mid-range/enhanced 14-bit, PIC18 16-bit). (*Ch 2*)

delta (psect flag) — Bytes per program-memory address. Mid-range code is word-addressable, so code psects use delta=2; data uses delta=1. (*Ch 10*)

DFP (Device Family Pack) — A package that teaches the tools a specific chip's registers, config bits, and memory. (*Ch 3*)

Directive — An order to the assembler (e.g. PSECT, EQU, CONFIG), not a CPU opcode. Some generate instructions (BANKSEL, PAGESEL), data (DW), or reserved storage (DS). (*Ch 4, 12; App B*)

EEPROM (Data EEPROM) — Small non-volatile data memory that survives power-off, accessed like a peripheral. (*Ch 1*)

Enhanced mid-range — The 14-bit core of our PIC16F17146: mid-range plus more instructions, automatic interrupt context saving, and linear memory. (*Ch 2*)

File register — Any addressable byte of data memory — including SFRs and W itself. (*Ch 6*)

FSR / INDF — The two 16-bit File Select Registers hold addresses; reading/writing INDF_n accesses the byte the FSR points to (indirect addressing). (*Ch 11*)

GIE (Global Interrupt Enable) — The master interrupt-enable bit; on PIC16F17146 it is INTCON bit 7, while PIC18 naming/priority gates can differ. (*Ch 14, 15*)

Harvard architecture — Separate program and data memories on separate buses, each with its own address 0. (*Ch 1*)

HEX file — The Intel-HEX program image (records of data + checksum) that a programmer burns into the chip. Uses byte addresses. (*Ch 18*)

Hexmate — The utility that manipulates HEX files (merge, checksum, fill, convert). (*Ch 18*)

INTCON — On PIC16F17146, the core Interrupt Control register at mirrored offset 0x0B (including GIE, PEIE, and INTEDG). Other cores/devices may use INTCON0/INTCON1 or different layouts. (*Ch 14, 15, 19*)

Interrupt / ISR — A hardware event that redirects the CPU to a service routine. PIC16F17146 and PIC16F570 use vector 0x0004; PIC18F57Q43 can use a vector table or legacy 0x08/0x18 vectors. Return is via the core-appropriate RETFIE. (*Ch 14, 15, 19*)

LAT (LATx) — The output-latch register; write here to drive a pin (safer than writing PORTx). (*Ch 7*)

Linear memory — A virtual FSR view (0x2000-0x2FEF) that stitches every bank's GPR into one contiguous run, for buffers larger than one bank. *(Ch 11)*

Linker — The tool that gathers psects, places each in its class's address range, resolves symbols, and emits the HEX. *(Ch 10, 17)*

Literal — A constant value baked into an instruction (e.g. the *k* in `movlw k`). *(Ch 6)*

Macro — A named block of source text the assembler pastes in wherever it's invoked (compile-time reuse). *(Ch 13)*

Map file — The linker's report of where every psect and global symbol landed. *(Ch 17)*

Mid-range — The classic 14-bit PIC core with banking and paging. *(Ch 2)*

Opcode — The numeric machine-code form of an instruction, stored in program memory. *(Ch 1)*

Page / Paging — On enhanced mid-range, `CALL/GOTO` address a 2048-word page and `PCLATH` supplies upper bits. Baseline pages and `CALL` entry restrictions differ; PIC18 does not need `PAGESEL`. *(Ch 9, 15, 19)*

PCLATH — The register supplying the high program-counter bits for `goto/call` (the "page register"). *(Ch 9)*

PEIE (Peripheral Interrupt Enable) — On PIC16F17146, `INTCON` bit 6 gates `PIE1-PIE6` sources (`PIE0` sources do not use it). Other devices differ. *(Ch 14)*

PIC18 — The 16-bit top of the 8-bit line; expanded registers, `movff`, vectored/prioritized interrupts on many parts. *(Ch 2, 15)*

PORT (PORTx) — Reading it returns the pin levels; writing it writes the latch (prefer `LATx` for output). *(Ch 7)*

PPS (Peripheral Pin Select) — Maps digital peripheral signals to I/O pins. Not needed for a software-driven LED (pins default to their data latch). *(Ch 7)*

Program memory (Program Flash) — Non-volatile memory holding your instructions and constants. *(Ch 1)*

Psect (program section) — A named, relocatable block of code or data the linker places into memory. *(Ch 10)*

Reset vector — The device-defined program address fetched after Reset. It is 0x0000 on the PIC16F17146 and PIC18F57Q43; PIC16F570 instead fetches its factory calibration word at 0x7FF and then rolls over to user entry 0. *(Ch 4, 15, 19)*

RETFIE — Return-from-interrupt: pops the return address and re-enables interrupts. Automatic context restoration is core-specific: PIC16F17146 and PIC16F570 switch shadow/secondary state; PIC18F57Q43 uses `retfie f` to restore its fast WREG/STATUS/BSR stack. (*Ch 14, 15, 19*)

Shadow registers — On PIC16F17146, Bank-63 copies of WREG, STATUS, BSR, PCLATH, FSR0, and FSR1 used for automatic interrupt context saving. Other cores save a different set or require an explicit fast-return mode. (*Ch 14, 15, 19*)

space (psect flag) — Which memory space a psect lives in: 0 = program, 1 = data (resolves the Harvard address overlap). (*Ch 10, 17*)

STATUS — A core status register holding ALU flags such as Z, C, and DC plus core-specific status/page fields. Its exact bits are device-specific. (*Ch 1, 11, 19*)

TRIS (TRISx) — The data-direction register: bit 0 = output, 1 = input. (*Ch 7*)

W (Working register) — The 8-bit accumulator most instructions read from or write to; also a file register named WREG. (*Ch 1, 6*)

Appendix G — From Assembly to C

You’ve learned to program the PIC16F17146 in pure assembly — the deepest way to understand a chip. In practice, most production firmware is written in **C** (with the MPLAB XC8 C compiler), dropping to assembly only where it’s genuinely needed. This appendix shows how your assembly knowledge fits into a C world, so you can move fluidly between them.

G.1 Why C, and why assembly still matters

C gives you loops, *if/else*, functions with real locals, and portability — the compiler handles banking, paging, and the compiled stack for you. For most work it’s faster to write and easier to maintain. So why did learning assembly matter?

- **You now understand what C compiles *to*.** Banking, the reset vector, config bits, interrupt vectors, the compiled stack — C hides them, but you know they’re there and can debug them.
 - **You can read the disassembly.** When a C program misbehaves at the register level, the Disassembly view is just the assembly you already read fluently.
 - **You can drop to assembly where it counts** — tight timing loops, a specific instruction sequence, or squeezing a few bytes — because the assembler and the C compiler **share the same assembly language** (User’s Guide §2).
-

G.2 The same tool underneath

This is the key connection: the assembler you’ve used **is the same assembler the XC8 C compiler uses**. As the User’s Guide puts it (§2), the internal assembler “is the same as that used by the MPLAB XC8 C Compiler tool, with the assembly language being common between both tools.” So the `movlw`, `BANKSEL`, `PSECT`, and `CONFIG` you know carry straight over.

Config bits are a good example of the overlap. In assembly you wrote:

```
CONFIG "WDTE = OFF"
```

In C, the equivalent pragma is:

```
#pragma config WDTE = OFF
```

Same settings, same values (the MPLAB X Configuration Bits window generates both forms).

G.3 Inline assembly inside C

When you need a specific instruction sequence inside a C function, XC8 lets you embed assembly with `__asm()` (XC8 C Compiler User's Guide DS50002737L §C language extensions):

```
void exact_nop(void) {  
    __asm("nop");  
}
```

Each `__asm("...")` injects assembly text into the compiler's output; multiple instructions can be separated with `\n`. The `asm()` spelling is also accepted, but the current C guide recommends `__asm()`. Inline assembly is not optimized by the assembler optimizer, and the compiler cannot infer all register/memory effects from an arbitrary text string. Keep snippets small. For an unlock sequence, use the exact device header symbols, banking/access operands, and data-sheet ordering—do not paste a generic PPS sequence across devices.

G.4 Mixing whole assembly modules with C

You can also keep entire routines in assembly and call them from C, or vice-versa. An assembly routine marked for external linkage can be declared `extern` in C and called like any C function, provided it follows the compiler's current ABI. Symbol decoration, argument/return locations,

preserved state, reentrancy, and stack model can vary by core and compiler version, so inspect the C-generated assembly and map rather than inventing a convention. The details are documented in the **MPLAB XC8 C Compiler User's Guide for PIC MCU (DS50002737L)**, which the assembler guide points to for “writing assembly code to be linked with C source code, or... in-line with C code” (User's Guide §2).

G.5 A suggested path forward

1. **Rebuild a book example in C.** Take the Chapter 14 timer-blink and write it in XC8 C — TMR0 setup, an interrupt function, a pin toggle. Compare the compiler's disassembly to the assembly you wrote; you'll recognize every instruction.
 2. **Read one map file from a C build.** The same psect/class/map vocabulary applies, although C emits additional compiler-managed sections and runtime support.
 3. **Drop to `__asm()` once.** Find a spot in a C program that needs an exact instruction sequence and write it inline. That's the sweet spot where your assembly fluency pays off.
 4. **Keep the data sheet habit.** Whether in C or assembly, confirm every register, bit, and config setting against the device data sheet before you rely on it — the discipline this whole book was built on.
-

G.6 Reference

- **User's Guide §2 “Assembler Overview”** — the shared assembler and assembly language between the PIC assembler and the XC8 C compiler.
 - **MPLAB XC8 C Compiler User's Guide for PIC MCU DS50002737L** — the authoritative guide to C, to mixing C with assembly, and to the calling convention.
-

That's the end of the book. You started not knowing what a file register was; you can now write, build, simulate, flash, and debug PIC assembly from the reset vector up — and read the official Microchip guides without getting lost. Go build something.